Universidad de Valladolid

Escuela de Ingeniería Informática de Valladolid

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Mención Computación

Hephaestus: motor de videojuegos en Elixir y su empleo en el estudio de sistemas distribuidos

Autor:
Alonso Sayalero Blázquez

Tutor:
Dr. César Llamas Bello

A mi madre, a mi padre y a mi hermana.

Resumen

Una de las preocupaciones prioritarias en la labor docente de la enseñanza universitaria consiste en preparar materiales y documentos que motiven al alumnado y le faciliten el aprendizaje mediante ejemplos amenos y cercanos a sus intereses.

En este trabajo se desarrolla una herramienta y un conjunto de módulos para facilitar la enseñanza de los sistemas distribuidos aplicando ejemplos y construcciones basadas en los juegos de rol por computadora. Para ello se ha diseñado un motor de videojuegos en un lenguaje moderno y que puede resultar interesante para el alumno como Elixir. Sobre este motor se han implementado, también, ejemplos para ilustrar casos usando Java como lenguaje vehicular. El material constituye un portfolio para que los alumnos construyan su propio código partiendo de ejemplos en la asignatura de Sistemas Distribuidos del Grado de Ingeniería Informática de la Escuela de Ingeniería Informática de Valladolid. El motor de videojuegos es altamente paratrizable y puede ejecutarse de manera distribuida, así como los escenarios, que pueden integrarse de modo distribuido. La implementación del motor de juegos se distribuye en forma de imagen de Docker para que el alumno pueda simular la construcción de un sistema distribuido en su propia máquina en el caso más reducido.

Palabras clave: Sistema Distribuido, Elixir, Java, juego por computadora, enseñanza.

Abstract

One of the primary concerns in university-level teaching is preparing materials and documents that motivate students and facilitate learning through engaging examples relevant to their interests.

This work presents the development of a tool and a set of modules designed to facilitate the teaching of distributed systems by applying examples and constructs based on computer role-playing games. For this purpose, a video game engine has been designed using a modern language that students may find interesting, such as Elixir. Examples have also been implemented on top of this engine to illustrate use cases, employing Java as the instructional language. The material serves as a portfolio, allowing students to build their own code based on examples provided in the Distributed Systems course of the Bachelor's Degree in Computer Engineering at the School of Computer Engineering of Valladolid. The video game engine is highly parameterizable and can be executed in a distributed manner, just like the scenarios, which can also be integrated in a distributed way. The game engine's implementation is distributed as a Docker image, enabling students to simulate the construction of a distributed system on their own machine in a scaled-down environment.

Keywords: Distributed System, Elixir, Java, computer game, teaching.

Índice general

1. Introducción **1**

1.1. Motivación 2

1.2. Objetivos 2

1.3. Arquitectura de alto nivel 3

1.4. Organización de la memoria 4

1.5. Sobre el uso de la IA 5

1.6. Agradecimientos 6

2. Planificación **7**

2.1. Metodología empleada 7

2.2. Entregables 9

2.2.1. Hephaestus 10

2.2.2. Juego didáctico 13

2.2.3. Prácticas para alumnos 14

2.2.4. Documentación web 16

2.2.5. Distribución 16

2.3. Organización temporal 17

2.4. Riesgos 19

2.5. Costes 23

2.5.1. Desarrollo 23

2.5.2. Producción 25

2.5.3. Costes totales 26

3. Estado del arte **29**

3.1. Antecedentes 29

3.1.1. Second Life 29

3.1.2. JominiEngine 31

3.1.3. Design patterns of successful role-playing games 31

3.2. Tecnologías empleadas 31

3.2.1. Lenguajes de programación 32

3.2.2. Gestor de bases de datos 35

3.2.3. Herramientas de documentación 35

3.2.4. Otras tecnologías 36

4. Análisis **39**

4.1. Motor Hephaestus 39

4.1.1. Análisis inicial 39

4.1.2. Requisitos 41

4.1.3. Casos de uso 43

4.1.4. Modelos 49

4.2. Juego didáctico 56

4.2.1. Análisis inicial 56

4.2.2. Requisitos 57

4.2.3. Casos de uso 58

4.2.4. Modelos 61

4.3. Otras partes del proyecto 62

4.3.1. Hephaestus 62

4.3.2. Juego didáctico 63

4.3.3. Prácticas 64

4.3.4. Documentación web 65

4.3.5. Distribución 66

5. Diseño **67**

5.1. Hephaestus 67

5.2. Base de datos 71

5.3. Juego didáctico 71

5.4. Prácticas 75

5.4.1. Introducción a las prácticas 75

5.4.2. Primera práctica 75

5.4.3. Segunda práctica 77

5.4.4. Tercera práctica 78

6. Pruebas **81**

6.1. Pruebas unitarias 82

6.2. Pruebas end-to-end 83

7. Conclusión **85**

7.1. Evaluación de los riesgos 85

7.2. Problemas en la implementación del código 86

7.2.1. Elixir 86

7.2.2. Docker 87

7.3. Desarrollo a futuro 87

7.3.1. Mejora en el proceso de pruebas 88

7.3.2. Ampliación del motor 88

7.3.3. Separación en motores más pequeños 88

7.3.4. Generalizar y aumentar la especialidad del motor 88

7.3.5. Entorno de desarrollo gráfico 89

7.4. Tiempo de desarrollo 89

A. Repositorio de código **93**

A.1. Ubicación del repositorio 93

A.2. Organización del repositorio 93

B. Prácticas para los alumnos **95**

B.1. Práctica 0 95

B.2. Práctica 1 99

B.3. Práctica 2 108

B.4. Práctica 3 116

Bibliografía **123**

Índice de figuras

1.1. Diagrama de bloques de la arquitectura del proyecto 4

2.1. Estructura de Descomposición de Productos (PBS) del proyecto 10

2.2. Diagrama de Gantt del proyecto 18

3.1. Algunos avatares dentro del juego. Imagen por HyacintheLuynes 30

4.1. Diagrama de casos de uso de Hephaestus (Base de datos y Enemigo) 50

4.2. Diagrama de casos de uso de Hephaestus (Conexión, Objeto y Personaje) 51

4.3. Diagrama de casos de uso de Hephaestus (Ubicación y NPC) 52

4.4. Diagrama de análisis Hephaestus completo 53

4.5. Diagrama de análisis de Hephaestus 55

4.6. Diagrama de casos de uso del juego didáctico 61

4.7. Diagrama de análisis juego didáctico 62

5.1. Diagrama de clases del motor Hephaestus 69

5.2. Diagrama de secuencia del caso de uso cambiar ubicación 70

5.3. Diagrama de secuencia del caso de uso herir 70

5.4. Diagrama entidad-relación de la base de datos 71

5.5. Diagrama de clases de Alfa 72

5.6. Diagrama de secuencia del caso de uso inspeccionar ubicación actual 73

5.7. Diagrama de secuencia del caso de uso iniciar sesión de juego 74

Índice de tablas

2.2. Riesgo R01 20

2.4. Riesgo R02 20

2.6. Riesgo R03 20

2.8. Riesgo R04 21

2.10. Riesgo R05 21

2.12. Riesgo R06 22

2.14. Riesgo R07 22

2.16. Riesgo R08 22

2.18. Tabla de costes de desarrollo 26

2.20. Tabla de costes en producción 26

4.1. Hephaestus. Requisitos funcionales 43

4.2. Hephaestus. Requisitos no funcionales 43

4.4. Caso de uso de Hephaestus CU-01 44

4.6. Caso de uso de Hephaestus CU-02 44

4.8. Caso de uso de Hephaestus CU-03 44

4.10. Caso de uso de Hephaestus CU-04 44

4.12. Caso de uso de Hephaestus CU-05 45

4.14. Caso de uso de Hephaestus CU-06 45

4.16. Caso de uso de Hephaestus CU-07 45

4.18. Caso de uso de Hephaestus CU-08 45

4.20. Caso de uso de Hephaestus CU-09 46

4.22. Caso de uso de Hephaestus CU-10 46

4.24. Caso de uso de Hephaestus CU-11 46

4.26. Caso de uso de Hephaestus CU-12 46

4.28. Caso de uso de Hephaestus CU-13 47

4.30. Caso de uso de Hephaestus CU-14 47

4.32. Caso de uso de Hephaestus CU-15 47

4.34. Caso de uso de Hephaestus CU-16 47

4.36. Caso de uso de Hephaestus CU-17 48

4.38. Caso de uso de Hephaestus CU-18 48

4.40. Caso de uso de Hephaestus CU-19 48

4.42. Caso de uso de Hephaestus CU-20 48

4.44. Caso de uso de Hephaestus CU-21 48

4.46. Caso de uso de Hephaestus CU-22 49

4.48. Caso de uso de Hephaestus CU-23 49

4.50. Caso de uso de Hephaestus CU-24 49

4.52. Caso de uso de Hephaestus CU-25 49

4.53. Juego didáctico. Requisitos funcionales 57

4.54. Juego didáctico. Requisitos no funcionales 58

4.56. Caso de uso del juego didáctico CU-01 58

4.58. Caso de uso del juego didáctico CU-02 58

4.60. Caso de uso del juego didáctico CU-03 59

4.62. Caso de uso del juego didáctico CU-04 59

4.64. Caso de uso del juego didáctico CU-05 59

4.66. Caso de uso del juego didáctico CU-06 59

4.68. Caso de uso del juego didáctico CU-07 60

4.70. Caso de uso del juego didáctico CU-08 60

4.72. Caso de uso del juego didáctico CU-09 60

4.74. Caso de uso del juego didáctico CU-10 60

4.76. Caso de uso del juego didáctico CU-11 61

7.1. Planificación en el tiempo del proyecto 92

Índice de Códigos

5.1. Snippet cliente Java entregado a los alumnos 76

5.2. Snippet codigo asíncrono Java entregado a los alumnos 78

6.1. Snippet test unitario Elixir 82

Capítulo 1

Introducción

Todos los alumnos que cursan alguna carrera universitaria o de grado superior se encuentran con una gran diversidad de métodos de aprendizaje. Durante esos años, los alumnos pueden enfrentar pruebas escritas, orales o prácticas de diversos estilos, algunos más antiguos, otros más modernos.

La docencia, al igual que otras muchas áreas, está en constante evolución, adaptándose a las necesidades del alumnado según las épocas y las tecnologías avanzan. Una de las últimas incorporaciones al mundo docente es la gamificación de los conceptos. Esta metodología consiste en emplear juegos, diseñados para el caso concreto, con el fin de que el alumno interiorice el concepto que se trata de explicar de una forma más sencilla.

A raíz de esta gamificación surge el problema de cómo implementarla en el ámbito educativo. Este proyecto trata de resolver ese problema para la asignatura de Sistemas Distribuidos de la Escuela de Ingeniería Informática de Valladolid.

Los sistemas distribuidos adquieren más importancia en un mundo en el que todo está conectado por la web y el usuario cada vez tiene menos tolerancia a la ralentización de los servicios a los que accede. Esto obliga a diseñar sistemas distribuidos que permitan un escalado horizontal sencillo y rápido, asegurando la disponibilidad y ajustando el coste a las necesidades puntuales de los usuarios.

Debido a la complejidad que supone todo lo mencionado, la enseñanza de estos tipos de sistemas acarrea una dificultad inherente al simple hecho de que algunas de las características de estos sistemas, como pueden ser la concurrencia o la tolerancia a fallos, son abstractas o se esconden detrás de múltiples capas de abstracción que dificultan el aprendizaje. Muchas veces esto lleva a una situación de creérselo por el simple hecho de que te lo cuentan, sin llegar a comprender los conceptos correctamente. Al hacer uso de un videojuego, algunos de estos conceptos pasan a afectar al alumno directamente. Para cumplir con este objetivo, el juego debe presentar situaciones, que obliguen a emplear alguno de los conceptos que se quieren enseñar para resolver el problema planteado dentro del mundo del videojuego.

Con todo esto en cuenta, la premisa del proyecto es crear un motor de videojuegos que permita arquitecturas tanto de sistemas distribuidos como de no distribuidos. Alrededor del motor, nombrado como **Hephaestus**, se crea un juego, diseñado para el aprendizaje de conceptos de los sistemas distribuidos, haciendo uso del motor y una serie de prácticas para la asignatura, basadas en lo que permite el videojuego.

1.1. Motivación

La motivación aparece a partir de los retos que plantea el proyecto. Dichos retos incluyen el diseño de un motor de videojuegos distribuidos, el diseño de un videojuego pensado para el aprendizaje de un alumno o el empleo de lenguajes de programación que siguen el paradigma funcional.

Todos estos retos suponen un aprendizaje que difícilmente se presentaría en otro contexto y, también, obligan a adoptar el paradigma funcional frente al imperativo, el cual sigue un mapa mental más natural, ya que la mayor parte del proyecto se realiza en Elixir, el cual sigue el paradigma funcional.

1.2. Objetivos

El proyecto consta de varios objetivos. El primero se trata de diseñar y programar un motor de videojuegos con las siguientes restricciones:

- Se debe poder utilizar para crear videojuegos tanto distribuidos como no distribuidos.
- Debe estar programado en Elixir.
- Debe estar especializado en juegos de tipo rol.

El lenguaje empleado, como ya se ha mencionado, es Elixir. Está elección se basa en la buena sinergia que tiene el lenguaje con este tipo de sistemas. Gracias a la máquina virtual BEAM o la tolerancia nativa a fallos, le hacen un lenguaje muy completo para diseñar sistemas distribuidos. Todas estas características y muchas más, que lo hacen idóneo para el proyecto, se detallan más adelante en este documento.

Los juegos de rol o RPGs, debido a su alto contenido narrativo, permiten ser jugados sin necesidad de desarrollar una interfaz gráfica, esto los hace especialmente adecuados para el proyecto, simplificando la arquitectura tanto del motor como del juego. El motor no tiene que implementar el renderizado de píxeles en pantalla, y el juego no tiene que tener en cuenta el aspecto visual del desarrollo de un videojuego, como puede ser la composición estética o los fallos ocasionados por las colisiones de objetos.

Una vez se cuenta con el motor, el siguiente objetivo es emplear dicho motor y crear un videojuego, el cual permita a los alumnos interiorizar los conocimientos vistos en la asignatura de Sistemas Distribuidos. El videojuego, al igual que el motor, cuenta con una serie de características que se deben cumplir:

- Se debe diseñar con el fin de enseñar conceptos de los sistemas distribuidos.
- Debido a la restricción del motor, debe estar escrito en Elixir.
- La longitud y complejidad del mismo tiene que estar adaptadas al contexto de los escenarios que se le plantean a los alumnos.

Como último objetivo del proyecto se debe proporcionar una serie de prácticas, ya mencionadas en el último punto del listado de características anterior, mediante las cuales el alumno pueda adquirir conocimientos nuevos sobre los sistemas distribuidos y pueda reforzar o comprender conceptos vistos en la teoría de la asignatura. Por ello, estas prácticas o escenarios cuentan también con una serie de restricciones:

- El número de prácticas no puede exceder las tres, debido a que estas forman parte de un conjunto más grande de prácticas que se realizan durante el curso.
- Se debe adaptar la dificultad de las mismas al nivel de los alumnos del curso.
- Se debe adaptar la longitud de las mismas para que, en conjunto con la dificultad, no supongan una inversión de tiempo requerido por el alumno superior a lo establecido por la guía docente.
- Deben estar diseñadas para poder resolverse mediante Java, aunque se pueden incluir partes a resolver en Elixir.

1.3. Arquitectura de alto nivel

Se muestra la arquitectura final del proyecto. Esta arquitectura es de alto nivel y por lo tanto no se centra en los detalles concretos del proyecto, estos se pueden ver en los Capítulos 4 y 5, dedicados al análisis y diseño del proyecto respectivamente.

El despliegue y la orquestación de los componentes del proyecto se realizan mediante Docker, esto permite la simulación de sistemas distribuidos en una única máquina y la posibilidad de desplegar la misma imagen en un sistema verdaderamente distribuido, es decir, en varias máquinas distintas. Todo ello sin necesidad de modificar ningún aspecto de la imagen Docker.

Tal y como se muestra en la Figura 1.1, el alumno interactúa, mediante un cliente Java, con el videojuego mediante el puerto 3000. La conexión entre el cliente y el juego se realiza mediante TCP/IP,

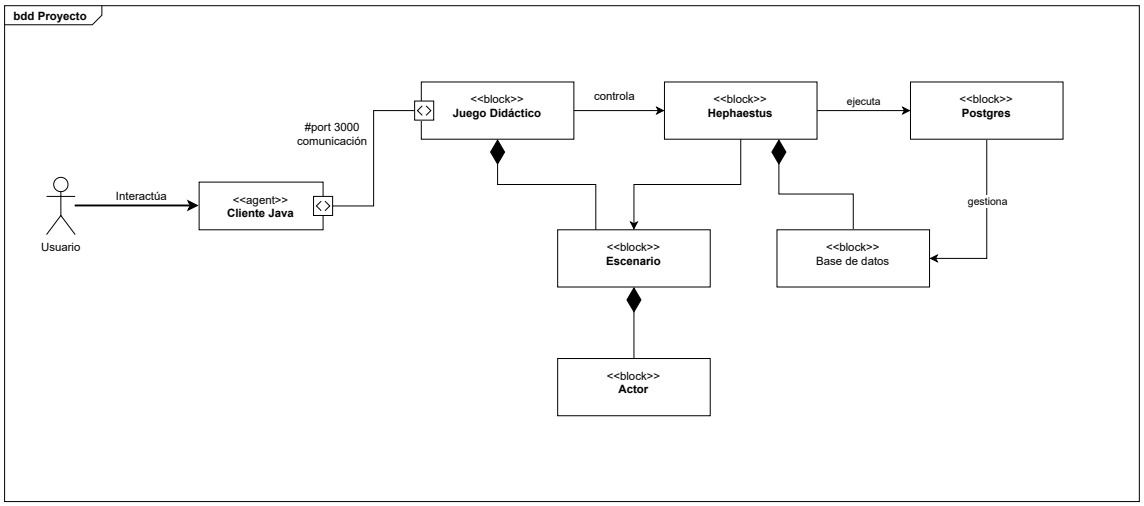


Figura 1.1: Diagrama de bloques de la arquitectura del proyecto

lo cual permite una comunicación segura hacia máquinas externas de ser necesario. El juego didáctico controla el motor Hephaestus a partir de las instrucciones que le llegan desde el cliente Java del jugador, este a su vez modifica y controla el escenario y la base de datos asociada al juego. Tras los cambios producidos por Hephaestus, el juego comunica el resultado de la ejecución al jugador de vuelta.

De esta forma se da una visión total de la arquitectura final del proyecto. Todos estos componentes se ejecutan mediante Docker de manera es necesario tener las imágenes Docker correspondientes tanto el motor, como al juego y una base de datos ya configurada y poblada con los datos necesarios.

1.4. Organización de la memoria

El documento presentado trata todos los aspectos relacionados con el proyecto que pudieran ser de relevancia, con el fin de mostrar todo el trabajo realizado durante el tiempo de ejecución del mismo y aclarar, en la medida de lo posible, cuestiones que pudiera dejar el hecho de mostrar únicamente el resultado final.

Siguiendo el orden mostrado en el índice de este documento, los capítulos que se pueden leer son los siguientes:

- **Introducción**. Introducción al proyecto, motivación que provoca la realización de dicho proyecto y objetivos del proyecto para que se pueda concluir satisfactoriamente. Dentro de los objetivos se incluyen algunas explicaciones sobre elementos centrales del proyecto, con el fin de brindar mayor claridad al porqué de la elección de los objetivos mostrados.
- **Planificación**. Se describe la metodología empleada durante la ejecución del proyecto, los artefactos producidos por la metodología empleada y cómo se ha organizado el trabajo a lo largo del tiempo

empleado. También se realiza el análisis de riesgos del proyecto y el análisis de costes, tanto aquellos a la hora de ejecutar el proyecto, como los recurrentes por ejecución a lo largo del tiempo.

- **Estado del arte**. Capítulo sobre otros proyectos con índole similar que hayan podido servir de inspiración. Se describen los proyecto referenciados de manera general y a partir de los aportes realizados a este proyecto. También se describen las tecnologías empleadas en el proyecto, las características de las mismas y los motivos para su uso.
- **Análisis**. Se centra en el análisis del proyecto. Se incluye un listado de los requisitos funcionales y no funcionales del proyecto, centrado en la pieza principal del mismo, el motor Hephaestus. También incluye una descripción escrita y gráfica de la arquitectura del motor y del sistema completo.
- **Diseño**. Se centra en el diseño software del sistema, incluyendo la descripción escrita y gráfica del diseño final del motor. También se incluye en el capítulo, una descripción del diseño del juego y las decisiones tomadas para dicho diseño, así como, una descripción de las prácticas diseñadas para reforzar conocimientos de la asignatura Sistemas Distribuidos.
- **Pruebas**. Capítulo dedicado a las pruebas de calidad realizadas sobre todo el proyecto, centrando la mayor parte sobre el motor.
- **Conclusión**. Capítulo final, últimas reflexiones sobre el proyecto y su ejecución, puntos de enfoque para el desarrollo futuro del mismo y problemas encontrados durante la implementación del código.

1.5. Sobre el uso de la IA

La inteligencia artificial (IA) es una herramienta cada más habitual en todos los trabajos modernos, en especial en el desarrollo de software, para el cual destaca en los momentos de escribir este documento. Por ese motivo es importante aclarar la influencia y alcance de su uso en el desarrollo de este proyecto.

Como autor de este documento y este proyecto declaro que el empleo de la IA ha sido meramente auxiliar, siendo empleada para ubicar y corregir errores de ortografía en este documento o encontrar errores dentro del código de una manera más rápida. Su ayuda se extiende a su vez a la búsqueda de la información necesaria para la realización del proyecto.

Añadir que tanto todo el código del proyecto, como este documento, ha sido escrito por mi, el autor, sin influencia de al inteligencia artificial y que todo el desarrollo de las ideas es proveniente de seres humanos, en concreto, una vez más, de mí, el autor, y de mi tutor, el Dr César Llamas Bello.

1.6. Agradecimientos

Este trabajo no podría haber salido adelante sin el apoyo emocional y económico de mi familia. Les agradezco a mis padres el haber estado ahí en todos los momentos en los que los necesite durante estos años de carrera, animándome a continuar en los momentos más duros. A mi hermana por apoyarme siempre, ya sea desde la lejanía o desde cerca. A mis amigos, por aguantar todo este tiempo cuando no podía quedar con ellos por estar haciendo este trabajo. Por último, agradecer a mi tutor, por guiarme cuando me encontraba perdido y permitirme dar forma a su idea inicial dándole un toque personal.

Capítulo 2

Planificación

Todo proyecto requiere de una planificación previa que estime el tiempo que conlleva la ejecución del mismo y el coste, tanto del desarrollo del proyecto como de la ejecución a lo largo del tiempo de vida que tenga el resultado final.

Una parte importante de la tareas de planificación es la elección de tecnologías base con las que se ejecuta el proyecto. Es poco común que una tecnología encaje perfectamente en los requisitos para realizar el proyecto, por tanto, estas tecnologías tienen que ser escogidas de acuerdo a un balance entre aspectos positivos que pueden aportar al proyecto y aspectos negativos que puedan suponer un retraso en el mismo. La elección tecnológica conlleva la aceptación de un cierto nivel riesgo que tendrá que tenerse en cuenta en la correspondiente lista de riesgos y nivel de tolerancia del proyecto.

2.1. Metodología empleada

En el desarrollo software existen múltiples aproximaciones metodológicas pensadas para organizar un proyecto, las más empleadas actualmente son el desarrollo en cascada, pionero en la planificación de proyectos de software; Scrum o desarrollo en espiral. Todas estas metodologías están diseñadas para proyectos en los que existe un producto final claro, que todas las funciones que tiene que realizar ya se han pensado y no van a sufrir modificaciones. Este proyecto, por el contrario, no cuenta con un estado final claro, evoluciona a lo largo del tiempo, en otras palabras, aunque el objetivo sea sencillo de decir, no consta de unos requisitos precisos, a priori, ni se conoce la dificultad de su construcción. Es previsible que algunos aspectos puedan cambiar a lo largo del desarrollo del proyecto a partir de los riesgos incurridos y los costes invertidos.

Para la situación del proyecto, lo más adecuado es emplear una metodología ágil basada en prototipos. Las metodologías ágiles ofrecen un catálogo de técnicas útiles en el caso de tener que desarrollar proyectos, mediante prototipado con objetivos cambiantes y poco definidos. La gestión del tiempo resulta más flexible

y se adapta a los resultados que se van obteniendo. Surge a principios de los 2000 a partir del manifiesto ágil, publicado en 2001, con la intención de cambiar la forma de hacer software [1]. De esta forma el desarrollo de software estaría centrado, según el mencionado manifiesto, en:

- Individuos e interacciones sobre los procesos y herramientas.
- Software funcional sobre extensa documentación.
- Colaboración con el cliente sobre negociaciones contractuales.
- Responder a los cambios sobre seguir un plan rígido.

A partir de entonces, múltiples metodologías han surgido siguiendo los principios de este manifiesto y poniéndolos en práctica de diferentes maneras.

Debido a las restricciones de personal y tiempo para el proyecto, la metodología empleada sigue algunos de los principios de las metodologías ágiles *Atern* y *Extreme Programming* tal y como se explican en el libro *Software project management* [2]. Los principios que se siguen en el proyecto son los siguientes:

- Entregas a tiempo. Se aplican ventanas de tiempo para realizar las entregas. Cada entrega cuenta con entregables con algún valor propio sin el resto del conjunto del proyecto, evitando el retraso de las fechas de entrega debido a que alguno de los productos sufre algún retraso.
- Desarrollo iterativo. Se crean prototipos los cuales van evolucionando en cada iteración; añadiendo, modificando o eliminando características del entregable; adquiriendo conocimiento sobre el sistema y el proyecto. De esta manera el proyecto se puede modificar añadiendo características que surgen de nuevas ideas derivadas del conocimiento proporcionado por los prototipos. Este punto es fundamental para el proyecto debido a la incertidumbre inicial sobre el producto final.
- Desarrollo incremental. El proyecto se divide en partes más pequeñas las cuales se implementan y entregan de manera secuencial. De esta forma añadir un nuevo componente al proyecto se simplifica al reducirse en añadir una nueva parte. Un aspecto importante a definir en este punto es si estos incrementos se consolidan de modo definitivo o si están sujetos a posibles cambios. En este caso están sujetos a posibles cambios como resultado de crear prototipos evolutivos.
- Comunicación y retroalimentación. La comunicación se realizará cara a cara siempre que sea posible, haciendo uso de la comunicación telemática en caso de no poderse dar lo anterior. Al estar compuesto el equipo del proyecto por dos personas, se realizará una reunión semanal, siempre que sea posible, en la que se pondrá en común con el tutor los prototipos que el alumno ha implementado en ese tiempo. En esa misma reunión se pondrá en común nuevas ideas para la consecución del proyecto y el tutor decidirá con el alumno la siguiente parte del proyecto a comenzar, en caso de haber terminado con la actual.

- Diseño simple. Se prefiere el diseño más simple que cumpla con los requisitos del cliente o del proyecto.
- Refactorización. Se evitará el código espagueti haciendo uso de la refactorización cuando sea necesario. Nunca en partes pequeñas del código y siempre sin miedo a eliminar grandes bloques del mismo si así se requiere.

Con estos principios se pretende dar forma a la idea inicial planteada para el proyecto, añadiendo nuevas ideas y partes durante la creación de las anteriores, hasta llegar al resultado final que se presenta mediante este documento.

2.2. Entregables

Tal y como se ha explicado en el apartado anterior, la metodología empleada para el desarrollo del proyecto hace uso de prototipos iterativos para desarrollar cada una de las partes que conforman el conjunto completo del proyecto, las cuales forman parte del desarrollo incremental de la metodología. Por lo tanto, se tiene que hacer una distinción entre los entregables que pertenecen a la parte incremental de la metodología y aquellos que pertenecen a la parte iterativa. A partir de este punto, en el documento se referirá a aquellos elementos que pertenecen a la parte incremental mencionada como entregables y a aquellos que pertenecen a la parte iterativa como prototipos.

El proyecto cuenta con cinco partes finales: El motor Hephaestus, un juego didáctico, un conjunto de prácticas para alumnos de segundo curso del Grado de Ingeniería Informática de la Universidad de Valladolid, cursando la asignatura de Sistemas Distribuidos, una página web con la documentación necesaria para usar Hephaestus como motor de un videojuego y todos los archivos necesarios para la distribución del proyecto.

En el diagrama de la Figura 2.1 se muestra el PBS (Project Breakdown Structure) del proyecto. Este diagrama muestra las diferentes partes que componen todo el sistema o proyecto, así como los entregables que conforman cada una de las partes. El empleo de partes se realiza con el fin de categorizar cada uno de los entregables en subconjuntos para facilitar su localización en cada contexto específico, pero hay que destacar que una parte del proyecto no es una separación de la metodología de planificación y, por lo tanto, a la hora de planificar el proyecto se tienen en cuenta los entregables como un conjunto total sin divisiones internas.

Al tratarse de una metodología iterativa cada entregable pasa por varias versiones antes de decantarse por la final. Cada una de estas versiones corresponde a una iteración de ese entregable y produce un prototipo. En la planificación de este proyecto, cada iteración se estima en una duración de una semana de trabajo, esto es debido a las restricciones laborales con las que cuenta el alumno a la hora de completar la

iteración. Por eso mismo, y dependiendo de la cantidad de iteraciones necesarias para llegar a la versión final de un entregable, la ejecución del proyecto se alarga sustancialmente a lo largo del tiempo.

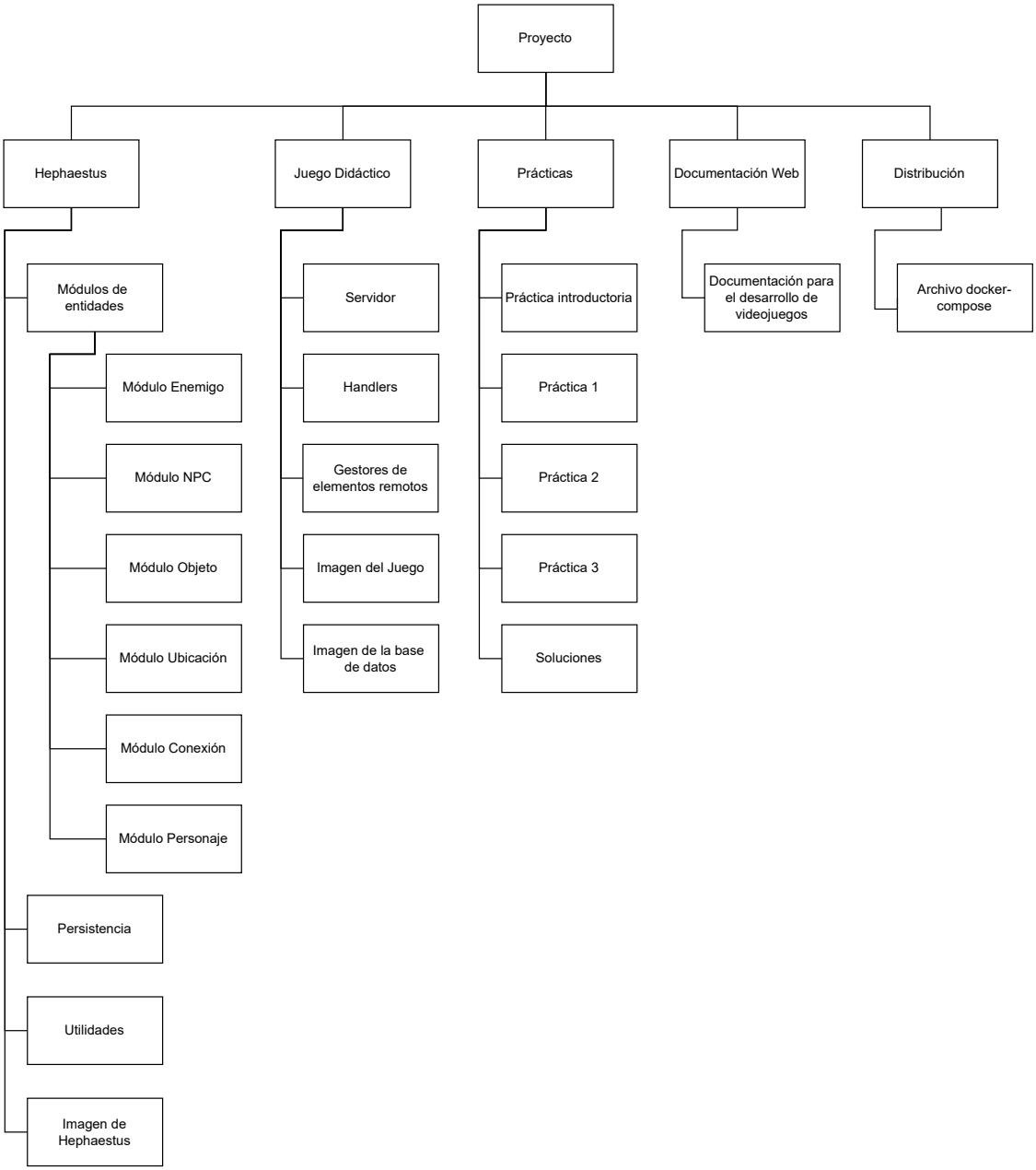


Figura 2.1: Estructura de Descomposición de Productos (PBS) del proyecto

2.2.1. Hephaestus

El motor de videojuegos Hephaestus, cuyas características a alto nivel se han detallado en la sección 1.2, es la pieza central del proyecto. El motor final se trata de un conjunto de piezas ya configuradas para su uso en un videojuego, a su vez, contiene utilidades básicas para el manejo del estado del juego, elección de

turnos, supervisión de procesos o enrutamiento de procesos distribuidos. Todos estos elementos forman el conjunto de entregables que conforman Hephaestus. Cada una de las piezas, ya preparadas para representar un elemento clásico de los juegos RPG, junto con las funciones auxiliares ya mencionadas y la imagen Docker del propio Hephaestus, se consideran un entregable de Hephaestus, dando lugar a los entregables de esta parte del proyecto.

Módulo Personaje

Primer entregable de Hephaestus. El personaje representa la entidad jugable del sistema y, a nivel técnico, actúa como un actor dentro del sistema del juego. El control del estado del mismo lo tiene el motor Hephaestus, el cual recibe órdenes por parte del juego didáctico. A nivel de código este entregable supone el primer contacto con un nuevo lenguaje de programación y varios conceptos nuevos, por lo tanto se planifican cinco prototipos para este entregable. De esta manera se asegura la correcta asimilación de todos los conocimientos necesarios. De este modo se sacrifica tiempo en realizar más prototipos de los que pueden ser necesarios con el fin de reducir el número de prototipos en los siguientes entregables, pues todos hacen uso de las mismas tecnologías.

Módulo Enemigo

A nivel técnico el enemigo es un actor, como ya se ha mencionado, por lo tanto se beneficia del conocimiento obtenido en el entregable anterior. Gracias a eso a la hora de planificar el tiempo y número de prototipos que pueden llegar a ser necesarios para implementar el entregable se reduce en gran medida. Con el fin de asegurar la correcta implementación en caso de fallo, al entregable se le asignan dos prototipos o iteraciones.

Módulo NPC

El NPC es, a nivel técnico, idéntico a los dos entregables anteriores, por consiguiente se le asignan el mismo número de iteraciones que al entregable del enemigo, dos. De la misma manera que en el entregable anterior, la segunda iteración establece un colchón en caso de error al realizar la implementación durante el primer prototipo.

Módulo Objeto

Al igual que los actores anteriores, a nivel técnico la implementación no difiere demasiado del resto de actores que se han planificado hasta este momento, por ello, al igual que en el entregable anterior, el periodo de tiempo previsto para la implementación del entregable es de dos iteraciones.

Módulo Ubicación

Una ubicación define los emplazamientos sobre los que transcurre el juego. Siguiendo la línea del resto de entregables anteriores, a nivel técnico la ubicación se implementa de la misma manera que los entregables vistos hasta este momento y, al igual que con el resto de entregables, se le planifican dos iteraciones para la ejecución del entregable.

Módulo Conexión

El último de los actores proporcionados y controlados por Hephaestus. Una conexión permite viajar entre ubicaciones y, a nivel técnico, este es último entregable con características parecidas a los ya tratados hasta ahora, por lo tanto es el último beneficiado por el trabajo realizado en el primer entregable y al que se le estiman dos prototipos para los trabajos de implementación.

Persistencia

Este entregable incluye todas las funciones necesarias para transferir el estado del juego al almacenamiento permanente de la base de datos. A nivel técnico, el manejo de base de datos es un problema ya resuelto a muchos niveles en el mundo de la informática. Por lo tanto, al no tratarse de un problema nuevo y simplemente necesitar de la adaptación correcta al lenguaje empleado, tres prototipos para la implementación del entregable, optando por una visión pesimista, son suficientes.

Utilidades

El último entregable de Hephaestus con componente de programación, a nivel técnico, supone un conjunto de funciones ya preparadas y separadas en módulos para su utilización. Con el fin de resolver el problema, se establece un límite de dos iteraciones para la implementación del entregable, en las cuales se prioriza las funciones necesarias para el resto de partes del proyecto.

Imagen Hephaestus

Con el fin de levantar un contenedor y que ejecute el programa correspondiente es necesario crear una imagen de dicho contenedor primero y por lo tanto el último entregable de esta parte del proyecto consiste en construir dicha imagen del motor Hephaestus. Elixir es un lenguaje que no está relacionado comúnmente con Docker, debido a que pierde una de sus fortalezas al emplear este entorno de ejecución, el hot swapping de código. Como esta característica no es de utilidad para el proyecto, el empleo de Docker no supone un problema, pero significa que la documentación que se puede encontrar para realizar

la labor de crear una imagen Docker ejecutando un programa escrito en Elixir es escasa. Por ese motivo, se le asignan dos iteraciones al entregable, permitiendo pulir la imagen final para que ocupe el menor espacio posible.

2.2.2. Juego didáctico

El juego didáctico es la siguiente pieza del proyecto después del motor. Ya que hace uso de él, el juego didáctico depende directamente de los actores y funciones que proporcione Hephaestus. Debido a la dependencia directa que tiene el juego respecto a Hephaestus, supone la evolución de este último a lo largo del desarrollo del juego. Se trata de un videojuego sencillo con dos partes distribuidas, los personajes jugables y los enemigos, de manera que se puede usar para enseñar el funcionamiento de estos sistemas. Cuenta con cuatro habitaciones por las que tiene que pasar el personaje, sin ser opcional ninguna, y presentan una oportunidad de enseñanza sobre algún concepto de los sistemas distribuidos y la programación de actores. La interfaz de juego está basada en un servidor TCP y comunicación mediante JSON-RPC (3.2) y, al igual que el motor, pasa por varios prototipos para obtener el entregable final.

Servidor

A nivel técnico el servidor supone la combinación de múltiples tecnologías con el fin de devolver la respuesta correcta al mensaje recibido. Para ello se necesita recibir el mensaje, deserializarlo, interpretar que acción se requiere para resolver el mensaje, solicitar al handler correcto el resultado de la acción, esperar dicho resultado, serializar el resultado y, por último, devolverlo. Toda esta secuencia es muy común en el ámbito web y por tanto no faltan recursos para solucionar el problema. Aún con toda la documentación, el hecho de preparar el servidor para soportar cargas de trabajo propias de producción supone un trabajo extra que hay que tener en cuenta a la hora de planificar su asignación de iteraciones. Teniendo todo esto en cuenta, y la posibilidad de emplear bibliotecas externas para esta labor se establecen dos iteraciones para la ejecución de esta parte del proyecto.

Handlers

En el entregable anterior se hace mención a los handlers. Los handlers son los encargados de resolver un mensaje concreto recibido por el servidor, dados los datos necesarios para la labor. A nivel técnico este entregable abarca toda al lógica necesaria para poder jugar al videojuego, por lo tanto es uno de los entregables con más contenido. Además, hay una gran variedad de escenarios que se necesitan tener en cuenta con el fin de cubrir todas las posibilidades. Pero, por otro lado, la sencillez del juego reduce la carga de trabajo en mayor medida. Teniendo todo en cuenta, la estimación de tiempo y prototipos para este entregable se establece en cuatro prototipos.

Gestores de elementos remotos

El siguiente entregable de esta parte del proyecto son los gestores de elementos remotos. Uno de los objetivos del juego es enseñar a los alumnos el funcionamiento de sistemas distribuidos y, para ello, el juego se trata como uno de estos sistemas. Este entregable no requiere de mucho tiempo ya que los gestores son una abstracción de lógica común que se repite en varios handlers y que se puede concentrar en un módulo propio. Hay que tener en cuenta además que los únicos actores que cuentan con máquinas propias son los enemigos y los personajes, por lo tanto solo son necesarios gestores para estos dos tipo de elementos remotos, el resto de actores se ejecutan en la misma máquina en la que corre el juego. Por estas razones, se planifican tres iteraciones para completar este entregable.

Imagen juego didáctico

Al igual que sucede con Hephaestus, el juego didáctico necesita una imagen propia para poder ser ejecutado en el entorno distribuido local de los alumnos. La única característica diferente respecto a la imagen del motor consiste en tener que indicar a Docker que se el código ejecutado se trata de un servidor y, por lo tanto, tiene que hacer uso de un puerto para poder comunicarse. Debido a que el conocimiento en creación de imágenes Docker obtenido en el entregable de la imagen de Hephaestus es completamente reutilizable para este entregable. La planificación de prototipos se limita a un único prototipo.

Imagen de la base de datos

El último entregable del juego didáctico, consiste en una imagen Docker de la base de datos empleada por el juego ya configurada con todo lo necesario para poder ser ejecutado. Al igual que con el entregable anterior, un único prototipo es suficiente para completar este entregable.

2.2.3. Prácticas para alumnos

A los dos apartados anteriores les sigue este, las prácticas para los alumnos del segundo curso de ingeniería informática en la Universidad de Valladolid. Estas prácticas se plantean como una serie de retos a resolver por el alumno haciendo uso de clientes escritos en Java. Todo ello con la excusa de finalizar el juego didáctico planteado en la sección anterior.

Las prácticas consisten en una serie de documentos que describen las labores que tendrá que realizar el alumno con el fin de completar la práctica correspondiente al documento, además de todas las instrucciones necesarias para poner en funcionamiento el juego en el estado correcto para comenzar la práctica. Además de la práctica escrita se incluye una serie de documentos de código Java con la posible solución de cada

una de las prácticas. Estos documentos no corresponden a una solución única, sino a una de las múltiples opciones mediante las cuales se pueden resolver.

Práctica introductoria

La primera práctica con la que se encuentra el alumno no se trata de una práctica per se. Consiste en un documento introductorio que incluye un resumen del resto de las prácticas. Al tratarse de un documento didáctico, supone un área nueva de conocimiento, por lo tanto requiere de un lenguaje y una estructura de documento diferente a lo acostumbrado. Por este motivo y por ser la primera de las prácticas se planifican dos iteraciones para este entregable, dando margen a poder realizar retoques al lenguaje empleado.

Práctica 1

Una vez se realiza la introducción, el siguiente entregable comprende la primera de las prácticas. Al tratarse de un nuevo documento didáctico, la novedad supuesta en el entregable anterior se considera subsanada en este. Por lo tanto la necesidad de iteraciones dedicadas a la finalización del entregable se reduce drásticamente. Teniendo esto en cuenta se planifican una única iteración.

Práctica 2

A continuación de la primera práctica se le muestra a los alumnos el funcionamiento interno de los actores en un sistema distribuido. Esto enmarca la segunda práctica y el segundo entregable del proyecto. Siendo igualmente un documento didáctico sigue la misma planificación del entregable anterior. Una única iteración se considera suficiente teniendo en cuenta el conocimiento adquirido con anterioridad.

Práctica 3

Este entregable consta de la última práctica a la que tendrán que hacer frente los alumnos. En esta última práctica se pretende enseñar al alumno un poco más sobre el funcionamiento interno de Hephaestus. Al igual que el resto de entregables vistos en esta sección hasta ahora, la tercera práctica se trata de un documento didáctico con las instrucciones necesarias para realizar la práctica. Por ello, se le planifican las mismas iteraciones que a las otras dos prácticas, un prototipo.

Soluciones

Al igual que se proporciona documentación para proponer unas prácticas de laboratorio a los alumnos, es también necesario proporcionar una posible solución al docente con el fin de facilitar la labor de corrección. Este entregable consta de tres soluciones, las cuales hacen uso de Java por primera vez en el proyecto y tienen que ser correctas. Por este motivo, se planifican dos iteraciones de las soluciones.

2.2.4. Documentación web

La siguiente parte del proyecto consiste en una página web orientada hacia desarrolladores externos, en este caso no para mejorar el motor sino para desarrollar videojuegos empleando el motor. Con el objetivo de documentar el funcionamiento de todas las partes del motor que sean relevantes para un desarrollador de videojuego, se emplea la biblioteca *docsify* [3] escrita en Javascript. Esta biblioteca permite escribir documentos Markdown y servirlos como páginas estáticas web, simplificando el proceso de documentación. El empleo de markdown como lenguaje de documentación permite la portabilidad de toda la documentación al formato físico de ser necesario, pues markdown cuenta con la posibilidad de convertir el fichero en un PDF.

Documentación para el desarrollo de videojuegos

El entregable supone un esfuerzo de selección y documentación exhaustivo. El objetivo de este entregable no es documentar todo el código de Hephaestus, sino seleccionar las partes que son necesarias para el desarrollo de un videojuego y describir sus capacidades y el funcionamiento. Al tratarse de una labor de documentación extensa y un proceso arduo de selección, siendo mitigada por el hecho de no ser una labor de una complejidad elevada, se planifican dos iteraciones para concluir este entregable del proyecto.

2.2.5. Distribución

El último punto, fundamental en el desarrollo de un proyecto, es como se va a distribuir el mismo. Durante la planificación del proyecto ya se han mencionado las imágenes Docker a distribuir por cada una de las partes del proyecto, las cuales se pueden encuadrar también en esta sección del mismo. Como parte de la disciplina de la distribución existe también otros elementos que permiten la ejecución del código en las máquinas soportadas. Por lo tanto, además de las imágenes Docker vistas en otras partes de este proyecto, se requiere de un documento que permita iniciar el sistema de manera reproducible y sencilla en una máquina local.

Archivo docker-compose

La tecnología que permite a Docker ser reproducible en cualquier entorno que tenga instalado su motor se denomina Docker Compose. Con el fin de que los alumnos tengan todos el mismo punto de inicio para realizar las prácticas, ya configurado y preparado para estas mismas, se tiene que crear un fichero docker-compose que reproduzca la misma secuencia de inicio de contenedores para todos los alumnos. Al contrario que ocurre con la creación de las imágenes, vistas en las partes del proyecto correspondientes a Hephaestus y al juego didáctico, la documentación para la creación de archivos docker-compose está ampliamente documentada y estandarizada. Por este motivo, una iteración es suficiente a la hora de planificar la implementación del entregable.

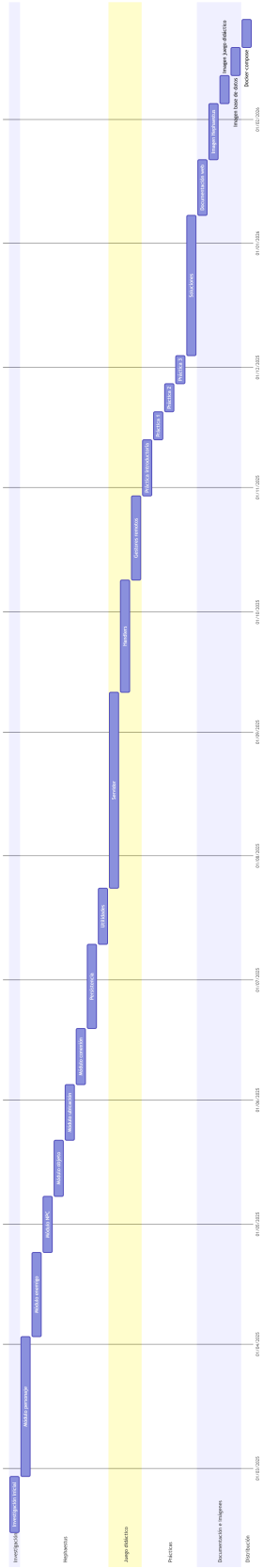
2.3. Organización temporal

Tal y como se ha mencionado en la Sección 2.1, el proyecto está dividido en varios entregables, los cuales a su vez están divididos en varios prototipos. Cada uno de estos prototipos requiere de una retroalimentación con el fin de corregir problemas que puedan haber surgido o, establecer el nuevo camino por el que avanzar una vez el prototipo está finalizado.

En la medida de lo posible, cada una o dos semanas se organiza una reunión entre el tutor y el alumno para que este último ponga en conocimiento común el proceso seguido y el estado del proyecto. En estas reuniones se establece el estado del prototipo actual y se determina si se da por terminado el entregable o se tiene que realizar un nuevo prototipo mejorando lo presentado en el anterior.

La figura 2.2 presenta el diagrama de Gantt parcial del proyecto, en él no se incluyen las reuniones acometidas durante el desarrollo del mismo. Se tiene que aclarar también sobre el diagrama, el hecho de que se toman en cuenta los parones por vacaciones de la Universidad de Valladolid como extensiones del tiempo empleado por los entregables que se ven afectados por dichos parones.

Una vez finalizada la planificación temporal del proyecto se puede calcular una estimación total de horas que van a ser necesarias emplear para la elaboración del mismo. En total se dedican 46 semanas a la investigación y desarrollo del proyecto, el tiempo medio semanal estimado es de 10 horas, debido a las circunstancias laborales del alumno. Por lo tanto la estimación del tiempo total, en horas, que se dedica al proyecto es una aproximación de 460 horas.


Investigación
Investigación inicial
Hipótesis
Análisis previo
Análisis crítico
Análisis del juego
Análisis conceptual
Análisis conceptual
Análisis conceptual

2.4. Riesgos

Un riesgo es una situación potencial que puede acarrear un retraso en el proyecto o lo puede acelerar. Todo riesgo tiene un posibilidad de ocurrir y un impacto sobre el proyecto. Es natural que cualquier proyecto que sea ejecutado esté expuesto a una serie de riesgos que modifiquen los tiempos de ejecución del mismo ya que toda planificación de un proyecto está basada en estimaciones y sujeta a incertidumbres. Todo proyecto de desarrollo software requiere de un análisis de riesgos que se puedan producir durante la ejecución del mismo. Los riesgos se dividen en diversos tipos dependiendo del área al que afecten:

- Estimación. Fallos a la hora de estimar los tiempos de desarrollo o el tamaño del software, entre otros muchos casos.
- Organizativos. Reestructuraciones en mitad del proceso de desarrollo, problemas económicos o derivados de estos.
- Personas. Falta de personal, enfermedades o personal sin el entrenamiento adecuado son algunos de los riesgos que pertenecen a esta categoría.
- Requisitos. Principalmente cambios de requisitos que supongan grandes cambios en el diseño.
- Tecnológicos. Bases de datos que no consiguen procesar todas las transacciones que se esperan, por ejemplo.
- Herramientas. Fallos de hardware o de herramientas de generación de código forman parte de este tipo de riesgos.

El análisis de riesgos realizado cuenta con los siguientes elementos, los cuales identifican, en la medida de lo posible, todas las variables para la prevención y recuperación de los riesgos [2] [4].

- **Probabilidad**: Nivel de posibilidad de que el riesgo suceda. Puede ser baja, media o alta.
- **Impacto**: Nivel del impacto que tiene el daño producido en caso de que el riesgo suceda sobre la ejecución del proyecto. Puede ser muy bajo, bajo, medio, alto o muy alto.
- **Plan de mitigación**: Lista de acciones que se pueden tomar para reducir la probabilidad de que el riesgo suceda.
- **Plan de contingencia**: Lista de acciones que se pueden llevar a cabo una vez el riesgo ha sucedido para reducir el impacto sobre los tiempos de ejecución del proyecto.

Las siguientes tablas describen los riesgos identificados en el análisis de riesgos del proyecto. Aparte de los elementos descritos anteriormente, cada tabla incluye también una descripción del riesgo al que se refieren.

Riesgo R01	**Falta de disponibilidad**
Descripción	Durante el desarrollo pueden surgir imprevistos, ya sean de salud o de otra índole, que impidan la participación de alguno de los dos integrantes del proyecto, el impacto será mayor si es el alumno el que sufre un imprevisto que le impida avanzar con el proyecto.
Probabilidad	Alta
Impacto	Medio
Plan de mitigación	▪ Evitar situaciones que puedan poner en riesgo la salud o la integridad física.
Plan de contingencia	▪ Aprovechar periodos de parada o con baja carga de trabajo para recuperar el tiempo que se pudiera haber perdido. ▪ Considerar tomar vacaciones en el trabajo para dedicarle más tiempo al proyecto.

Tabla 2.2: Riesgo R01

Riesgo R02	**Fallos de hardware**
Descripción	Todo equipo electrónico puede llegar a fallar, lo que puede suponer retrasos y perdida de información.
Probabilidad	Baja
Impacto	Muy alto
Plan de mitigación	▪ Disponer de un dispositivo de repuesto. ▪ Realizar copias de seguridad de todo lo relacionado con el proyecto de forma habitual. ▪ No mantener las copias de seguridad en el mismo equipo que el que se usa para trabajar en el proyecto, de ser posible almacenarlas en la nube. ▪ Instalar sistemas SAI para evitar perder datos ante un corte de luz.
Plan de contingencia	▪ Recuperar las copias de seguridad, de existir, en otro dispositivo. ▪ En caso de archivos corruptos intentar usar programas de recuperación para este tipo de archivos.

Tabla 2.4: Riesgo R02

Riesgo R03	**Error en la estimación de tiempo de desarrollo o tamaño de software**
Descripción	La novedad para el alumno de muchas de las tecnologías empleadas pueden provocar en la incorrecta estimación tanto del tiempo que puede llevar la ejecución de un prototipo como el tamaño de dicho prototipo. Una mala estimación del tamaño conlleva también un retraso en el proyecto.
Probabilidad	Alta
Impacto	Muy alto
Plan de mitigación	▪ Realizar análisis en profundidad de las tareas para obtener estimaciones más precisas. ▪ Dar prioridad a las partes del proyecto que resulten críticas. ▪ Emplear tiempo extra de tareas sencillas o cuya estimación haya sido positiva para aprender en mayor profundidad las tecnologías empleadas.
Plan de contingencia	▪ Reorganizar el proyecto eliminando partes no importantes de manera que su objetivo principal no se vea afectado. ▪ En última instancia, retrasar el final del proyecto para poder incluir todos los aspectos básicos del mismo.

Tabla 2.6: Riesgo R03

Riesgo R04	**Falta de conocimiento sobre las tecnologías**
Descripción	Aunque se inicia el proyecto sabiendo que una de las tecnologías supone un nuevo reto para el alumno (Elixir), existe el riesgo de subestimar la complejidad de dicha tecnología o sobrestimar el conocimiento sobre el resto de tecnologías que puedan llegar a ser empleadas en el proyecto.
Probabilidad	Media
Impacto	Alto
Plan de mitigación	▪ Estudio de la documentación oficial de las tecnologías empleadas. ▪ Seleccionar tecnologías conocidas para las partes del proyecto en las que no haya que emplear *Elixir*. ▪ Contar con un conjunto de tecnologías que puedan reemplazar alguna de las ya empleadas.
Plan de contingencia	▪ Cambiar tecnologías las cuales supongan una mayor complejidad a la esperada con alguno de los sustitutos investigados. ▪ Redistribuir los tiempos de desarrollo para priorizar las tecnologías que conlleven un mayor tiempo debido al escaso conocimiento. ▪ Emplear herramientas o bibliotecas que simplifiquen las partes menos importantes.

Tabla 2.8: Riesgo R04

Riesgo R05	**Cambios bruscos en los requerimientos que obliguen a un rediseño**
Descripción	Debido a la naturaleza del proyecto, pueden llegar a surgir cambios en los requerimientos de alguno de sus componentes que obliguen a realizar un rediseño completo del componente afectado, suponiendo un retraso en los tiempos del resto de prototipos y entregables
Probabilidad	Baja
Impacto	Medio
Plan de mitigación	▪ Optar por una implementación modular, debido a que un cambio de diseño puede afectar a unos pocos módulos y no a todo código. ▪ Estudiar los puntos del proyecto que puedan ser más propensos a recibir muchos cambios de requisitos y no implementarlos hasta que sea estrictamente necesario. ▪ Hacer uso de herramientas de generación de código que puedan asistir con la modificación del código.
Plan de contingencia	▪ Descartar cambios de requisitos cuyo aporte al proyecto frente al coste que suponga implementarlos sea muy negativo. ▪ Eliminar requisitos menos importantes en favor de cambios a requisitos importantes. ▪ En caso de falta de tiempo, adaptar los requisitos al código y no al revés.

Tabla 2.10: Riesgo R05

Riesgo R06	**Fallo en el uso de herramientas de inteligencia artificial**
Descripción	El avance en las herramientas de inteligencia artificial es claro y muy útil para el desarrollo de software, pero, al igual que otras herramientas, la inteligencia artificial puede fallar y entorpecer el desarrollo en lugar de acelerarlo.
Probabilidad	Baja
Impacto	Bajo
Plan de mitigación	▪ No emplear las herramientas para la generación de código, hacer uso de ellas únicamente para búsqueda de fallos o asistencia con la refactorización del código. ▪ Supervisar el trabajo realizado por la inteligencia artificial. ▪ Si se emplea en la refactorización de grandes bloques de código, implementar la refactorización de manera gradual y probando si el nuevo código no falla.
Plan de contingencia	▪ Si se detectan fallos continuados sin mejora, dejar de hacer uso de las herramientas.

Tabla 2.12: Riesgo R06

Riesgo R07	**Fallos de entendimiento a la hora de definir los requerimientos**
Descripción	Toda comunicación humana es propensa a que se produzcan malentendidos. Una mala comunicación entre tutor y alumno puede suponer retrasos en el proyecto debido a reescrituras de código que, de no haberse producido el malentendido, no habrían sido necesarias.
Probabilidad	Baja
Impacto	Muy bajo
Plan de mitigación	▪ Asegurar la claridad de la idea antes de proponer requerimientos. ▪ Estudiar todas las posibles ramificaciones que puedan surgir de un requerimiento.
Plan de contingencia	▪ Adaptar un punto medio entre la idea original y el código producido. ▪ Eliminar requerimientos no esenciales para el proyecto en el caso de que produzcan mucha confusión.

Tabla 2.14: Riesgo R07

Riesgo R08	**Dificultad en la integración de diferentes lenguajes de programación**
Descripción	El proyecto se implementa haciendo uso de dos lenguajes de programación distintos. Estos lenguajes no son interoperables entre sí de forma nativa. Por ello, la incorrecta integración entre ambos dentro del proyecto puede suponer el fallo completo del mismo.
Probabilidad	Baja
Impacto	Muy alto
Plan de mitigación	▪ Definir el área de uso de cada uno de los lenguajes. Evitando su coincidencia en una misma parte del proyecto. ▪ Emplear protocolos de comunicación agnósticos al lenguaje de programación. ▪ Realizar pruebas de comunicación entre los diferentes lenguajes de programación antes de comenzar la implementación completa de una parte del proyecto.
Plan de contingencia	▪ Reescribir la parte del proyecto que se encuentre en un estado menos avanzado en el lenguaje empleado para implementar las otras partes.

Tabla 2.16: Riesgo R08

2.5. Costes

Además del análisis de riesgos de un proyecto, es crucial realizar también un análisis de costes del mismo. Dicho análisis tiene que tener en cuenta los costes acarreados en la etapa de desarrollo del proyecto y los costes que va a tener el producto final durante su vida útil.

Cada una de las etapas cuenta con costes comunes entre sí y costes que son específicos de la etapa concreta. A su vez, los costes de cada etapa se pueden dividir en costes recurrentes o costes fijos. Un coste recurrente es todo aquel coste que se produzca de manera periódica, por ejemplo una subscripción a un servicio, mientras que un coste fijo es aquel que solo se produce en un momento concreto y, una vez realizado el desembolso, no hay necesidad de que se vuelva a producir, por ejemplo la compra de un equipo.

2.5.1. Desarrollo

Durante el desarrollo del proyecto, se producen unos costes relacionados con la mano de obra, el equipo empleado, el coste de alojamiento del control de versiones del código, las licencias de software necesarias y el alojamiento web.

Mano de obra

El gobierno de España establece el salario mínimo mensual en 1184 euros en 14 pagas [5]. Si se tiene en cuenta que semanalmente se le han dedicado 16 horas al proyecto de media, se puede aproximar el sueldo mensual a 518 euros [6]. Habiéndose alargado el desarrollo del proyecto hasta los 12 meses, el coste aproximado de mano de obra es de 6216 euros.

Se emplea el salario mínimo interprofesional con el fin de simplificar los cálculos en este documento. En la realidad, el salario de un ingeniero informático junior puede variar mucho dependiendo del lugar en el que se encuentre la empresa y el sueldo que ofrezca dicha empresa contratante. Por ese motivo y, debido a que el SMI se encuentra dentro de la horquilla de posibles salarios para un ingeniero informático junior, se hace uso del ya mencionado SMI en el cálculo de la mano de obra.

Equipo empleado

Para el desarrollo del proyecto se han empleado dos equipos, un ordenador fijo de sobremesa, para el desarrollo diario del mismo, y un ordenador portátil, para aquellas ocasiones en las que se necesitara la portabilidad que ofrece un equipo de estas características. El coste del ordenador de sobremesa es de

1600 euros mientras que el portátil se valora en 700 euros. Estos costes se toman del precio pagado en el momento de la compra de los mismos.

En total, el coste en equipo de trabajo es de 2300 euros. Este coste no es el coste real de lo equipos, ya que al finalizar el proyecto estos no dejan de ser empleados, el coste real es el desgaste producido en estos durante la realización del mismo. Este coste se conoce como amortización y, en este caso, se calcula empleando la siguiente fórmula, ya que, por simplicidad, se emplea la amortización lineal [7]:

$$Coste = (Precio * TiempoUso)/VidaUtil$$

La Agencia Tributaria proporciona una tabla para el cálculo de amortización lineal [8]. Según esta tabla los equipos informáticos cuentan con una vida útil de 8 años y, por lo tanto, el coste de los equipos de trabajo durante el periodo de ejecución del proyecto resultan en 287,5 euros.

Control de versiones

El código se aloja en un servicio de control de versiones en la nube llamado Github. El coste de alojamiento de Github se basa en un modelo por asientos. Existe una versión no profesional gratuita y dos profesionales, una enfocada a equipos más pequeños y otra enfocada a grandes empresas. En este caso, el modelo más adecuado para el proyecto es el profesional para equipos pequeños, el cual cuesta 4 dólares estadounidenses por miembro al mes. Para el proyecto el coste será, por lo tanto, de unos 4 euros aproximados al mes, ya que la conversión entre dólar y euro es cambiante y, en este caso, se estima de manera pesimista la conversión.

El desarrollo del proyecto comienza en febrero del año 2025 y finaliza un año más tarde en febrero del año 2026. Teniendo en cuenta, que el único contribuidor al código del proyecto es el alumno, el coste total del alojamiento del proyecto durante el desarrollo del mismo en Github asciende a 48 euros aproximadamente.

Licencias de software

El uso del lenguaje de programación Elixir es gratuito, al igual que el empleo de Docker y Postgresql. Por otro lado, Java tiene un modelo de financiación variado en el cual el empleo del lenguaje de programación en su variante empresarial es de pago, siguiendo la misma estrategia de cobro que Github. En el caso de Java, el coste por usuario por mes de 15 dólares estadounidenses, que, por simplicidad de cálculos, se aproximan a 15 euros al mes. Por último, el editor empleado para el desarrollo del proyecto es Visual Studio Code, editor de texto creado por Microsoft y que está orientado a la programación, supone un coste nulo, ya que es gratuito.

Con todo esto en cuenta, el coste de licencias de software es de 180 euros al calcular el coste Java de 15 euros al mes por el año de desarrollo necesitado para el proyecto.

Alojamiento web

Una de las partes del proyecto es una página web pensada para todo aquel desarrollador de videojuegos que quiera hacer uso del motor Hephaestus. Esta web tiene que ser alojada en un servidor web para que sea visible públicamente. Github ofrece alojamiento de páginas web estáticas sin ningún coste para repositorios de código públicos. Siendo este el caso el coste del alojamiento web es de 0 euros.

2.5.2. Producción

Una vez finalizado el proyecto la etapa de producción del mismo puede tener unos costes asociados. Al igual que a la etapa de desarrollo, a esta etapa también le afectan los costes de licencias de software, alojamiento web y alojamiento del control de versiones. Estos costes ya han sido establecidos en el apartado anterior.

Los costes propios de la etapa de producción son el alojamiento de las imágenes Docker del proyecto y el equipo necesario para ponerlo en funcionamiento.

Alojamiento de imágenes

Las imágenes Docker del proyecto se pueden construir a partir del código disponible en el repositorio de Github, pero, en caso de querer almacenar las imágenes ya construidas en un repositorio público para un acceso más sencillo, existen productos que ofrecen estos servicios. DockerHub es el repositorio en la nube más conocido. Propiedad también de los dueños de Docker, su modelo de precios permite el alojamiento de una imagen privada, en caso de necesitar más se tiene que optar por una opción de pago, y alojamiento ilimitado de imágenes públicas.

En el caso del proyecto las imágenes son públicas, con el fin de que los alumnos puedan tener acceso a ellas, por lo tanto no tienen un coste asociado a su alojamiento.

Hardware

El proyecto completo está pensado para poder ser usado en una sola máquina y, como se estableció en el apartado de desarrollo, un ordenador valorado en 700 euros es capaz de ejecutarlo.

A parte de poder ser usado en una sola máquina, la naturaleza del proyecto es distribuida y por lo tanto se puede desplegar de esa manera. Para ello son necesarias tres máquinas independientes, si se opta por el empleo de VPS (Servidor Virtual Privado, por sus siglas en inglés) el coste se puede estimar en 4 euros mensuales por máquina.

Por lo tanto, si se opta por la primera opción, el coste de la producción del proyecto es únicamente de un mínimo de 700 euros como coste fijo. En caso de optar por la segunda opción el coste sería recurrente costando 12 euros mensuales, dependiendo siempre del distribuidor de VPS elegido.

2.5.3. Costes totales

Coste	Tipo de coste	Valor (€)
Mano de obra	Mensual	518
Equipo hardware de desarrollo	Fijo amortizado	287,5
Control de versiones (Github)	Mensual por contribuidor	4
Licencias de software (Java)	Mensual por desarrollador	15

Tabla 2.18: Tabla de costes de desarrollo

Coste	Tipo de coste	Valor (€)
Control de versiones (Github)	Mensual por contribuidor	4
Equipo hardware	Fijo	700
Equipo hardware VPS	Mensual variable (depende de la máquina contratada)	4

Tabla 2.20: Tabla de costes en producción

Resumiendo lo visto en los dos apartados anteriores las Tablas 2.18 y 2.20 muestran un resumen de los gastos que se pueden producir a lo largo del desarrollo y de la producción del proyecto. Teniendo en cuenta que el tiempo de desarrollo estimado es de un año, se puede realizar el cálculo de costes de desarrollo. Respecto a producción el coste varía de múltiples maneras mensualmente, por lo tanto, la estimación dada corresponde a un coste mensual aproximado, con los menores costes posibles.

En primer lugar, el coste total de desarrollo consiste en la suma total, durante un año, de todos los costes vistos en la tabla. La mano de obra aporta un total de 6216 euros al coste final del desarrollo. A esto se le tienen que sumar el control de versiones y las licencias de software, teniendo en cuenta que solo hay que pagar por una asiento en cada una de las mensualidades, lo cual aportan un total de 228 euros. Finalmente a todo esto se le suma el coste amortizado por un año del equipo hardware dando un total de 6731,5 euros en el año de desarrollo que lleva el proyecto.

Para finalizar el coste total de producción depende del tiempo que se tenga en funcionamiento el proyecto, por ello lo más adecuado es proporcionar el coste mensual del proyecto una vez a salido de la fase de desarrollo. En primer lugar, el coste de alojamiento en Github continua siendo el mismo. A este coste hay que añadirle el coste del equipo hardware, este puede ser mediante un equipo físico propio o haciendo uso de VPS para conseguir auténtica arquitectura distribuida. Si se escoge el equipo hardware propio hay que calcular la amortización del sistema mínimo conocido en el cual funciona. Esto constituye un gasto de 7,29 euros al mes dando un total de 11,29 euros mensuales. Si se escoge la opción de las VPS es necesario alquilar tres, por lo tanto el coste mensual aproximado es de 16 euros.

Resumiendo el coste total de desarrollo es de 6731,5 euros mientras que la aproximación mensual de costes en producción depende de la elección de hardware donde se despliega, siendo de 7,29 euros al mes en caso de hardware propio o de 16 euros al mes en caso de desplegarlo haciendo uso de varias VPS.

Capítulo 3

Estado del arte

Aunque una idea sea original existen otros muchos trabajos que pueden servir para aportar nuevas ideas que complementen a la original, encontrar soluciones a problemas que pueda llegar a tener el proyecto o simplemente afianzar la idea inicial al ver otro proyecto que comparte un concepto que se va a emplear en el nuevo. La construcción del proyecto depende enteramente de las tecnologías que lo conforman, sin estas el proyecto no tendría entidad y solamente existiría como concepto abstracto.

3.1. Antecedentes

Dado que el foco principal del proyecto es el motor de videojuegos distribuidos Hephaestus, el foco de la investigación inicial es la búsqueda de otros ejemplos de videojuegos distribuidos y, por lo tanto, que cuenten con un motor distribuido. Teniendo esto en cuenta, es también importante adquirir nociones básicas de desarrollo de videojuegos y de videojuegos de rol ya que el juego didáctico forma parte de esta categoría.

3.1.1. Second Life

Second Life es un mundo virtual multijugador que permite a los usuarios crearse un avatar para ellos mismos e interactuar a través del avatar con otros usuarios y con el contenido creado por estos. Fue lanzado en junio de 2003 por la compañía Linden Lab. Se puede acceder al mundo virtual mediante el cliente software propio de la empresa o mediante clientes creados por terceros, ya que la tecnología del cliente es de código abierto.

La tecnología sobre la que se construye Second Life está formada por el cliente, ejecutado en la máquina del usuario, y miles de servidores bajo el mando de Linden Lab [9].

Cliente

El cliente renderiza gráficos 3D utilizando la tecnología OpenGL y su código fuente está licenciado bajo LGPL (Lesser General Public License) desde 2010, permitiendo la creación de clientes por parte de terceros los cuales añaden funcionalidades que el oficial no tiene.

Servidor

Cada región del mundo virtual (256x256 metros de área) se ejecuta en un núcleo en un servidor con un procesador multinúcleo. Además del alojamiento de la región del mundo correspondiente los servidores se encargan de la comunicación entre usuarios y objetos presentes en la región de la que se encargan. Además realizan los cálculos de colisiones entre avatares y objetos.

Figura 3.1: Algunos avatares dentro del juego. Imagen por HyacintheLuynes

De este proyecto se puede obtener la confirmación del uso de servidores distribuidos encargados cada uno de un elemento concreto. Aunque todos corren el mismo conjunto de operaciones, cada uno se especializa en una de las regiones del mundo virtual y se comunica con el resto de servidores y con los clientes para conservar la integridad de dicho mundo.

Esto se puede extrapolar al proyecto confirmando que el motor puede ser distribuido sin realizar modificaciones al código del mismo y que cada servidor se puede especializar en un elemento concreto del juego.

3.1.2. JominiEngine

El JominiEngine es un motor de videojuegos distribuidos creado por David Alexander Bond en 2015 como parte del estudio *Design and implementation of a massively multi-player online historical role-playing game*. En este estudio se presenta la funcionalidad imprescindible del motor así como guías para su futuro desarrollo [10].

Este proyecto permite confirmar la posibilidad de hacer algo parecido con este proyecto. Pese a que la premisa de motor de videojuegos distribuidos de rol es igual para ambos proyectos, el JominiEngine está especializado en juegos de rol históricos, mientras que Hephaestus se enfoca en la fantasía clásica popularizada por juegos de rol de mesa como Dungeons and Dragons.

Se pueden encontrar otras muchas diferencias entre motores debido a las diferencias entre requisitos de un proyecto a otro, mientras que JominiEngine enfoca su evolución hacia los juegos MMORPG, Hephaestus tiene un enfoque más didáctico y de un jugador, aunque esto no impide que se utilice en juegos multijugador.

Pese a las diferencias JominiEngine es un buen punto de conocimiento para el desarrollo de motores de videojuegos que permitan la creación de juegos distribuidos.

3.1.3. Design patterns of successful role-playing games

Design patterns of successful role-playing games es un documento publicado por Whitson John Kirk III en el año 2006 en el cual ahonda en el diseño de juegos de rol. En este caso el estudio realizado por Kirk no se centra únicamente en los videojuegos si no que profundiza en todas las mecánicas que puedan ser de utilidad a la hora de diseñar un juego de rol en cualquier formato, físico o virtual [11].

El nivel de profundidad que llega a tener este estudio sobrepasa por mucho el objetivo de este proyecto y, en concreto, la complejidad del juego didáctico diseñado en el mismo. Pero, aún así, el documento concentra en un mismo lugar todos los elementos que conforman un juego de rol, facilitando el diseño del juego didáctico y aportando ideas nuevas que podrían pasar desapercibidas de otra forma.

3.2. Tecnologías empleadas

La elección de tecnologías se realiza teniendo en cuenta las necesidades del proyecto. Estas necesidades que se necesitan suplir son los lenguajes de programación en los que se va a realizar el proyecto, el gestor de bases de datos empleado y otras herramientas o tecnologías necesarias para el correcto desarrollo del mismo.

3.2.1. Lenguajes de programación

Un proyecto software está sostenido por los lenguajes de programación en los que se construye. La elección de lenguajes de programación se guía por las características que estos tengan o por requisitos que tenga el proyecto.

Elixir

Elixir [12] es un lenguaje de programación que ofrece unas características que lo hacen especialmente apto para el diseño y programación de sistemas distribuidos. Sigue el paradigma de programación funcional y está diseñado para ser implementado en una arquitectura modular.

Es un lenguaje basado en Erlang [13] y emplea la misma máquina virtual que este. De esta manera se puede hacer uso de bibliotecas originalmente pensadas para Erlang desde el propio Elixir de forma nativa. Además, Elixir ofrece otra serie de características, algunas de ellas heredadas de Erlang, que lo siguen posicionando como un lenguaje muy interesante para el desarrollo de sistemas distribuidos y desarrollo web:

- **Tolerancia a los fallos**. Si algún componente del sistema falla, es posible recuperar la parte del sistema en la que se encuentra este fallo sin poner en riesgo el funcionamiento del resto del sistema. Esto se debe al sistema de supervisores que Elixir hereda de Erlang.
- **Distribuido**. Elixir puede ejecutar partes del código en procesos ligeros que están aislados y se comunican mediante mensajes. Estos procesos se pueden ejecutar en cualquier máquina que cuente con una máquina virtual de Erlang y el propio lenguaje cuenta con mecanismos nativos para facilitar la comunicación entre procesos localizados en máquinas diferentes, siempre que estas se puedan comunicar entre sí.
- **Escalable**. Gracias a la característica del punto anterior, el código de Elixir puede lanzar nuevos procesos sin afectar el funcionamiento del resto. Estos nuevos procesos se pueden lanzar en máquinas diferentes si se requiere.

Elixir cuenta con el conjunto de patrones de diseño y comportamientos OTP (Open Telecom Platform) [14]. Diseñado para proporcionar la estructura necesaria para aplicar la filosofía de dejar que falle, se traduce en un conjunto de bibliotecas estándar del lenguaje de Elixir que facilitan el desarrollo software. En el contexto de este proyecto, el conjunto proporciona una serie de elementos interesantes para cumplir con los objetivos planteados.

En primer lugar, Elixir cuenta con una biblioteca estándar encargada de abstraer la implementación de un proceso cliente-servidor. Dicha biblioteca se llama "GenServerz permite levantar procesos que admiten peticiones síncronas y asíncronas, haciendola idónea para implementar actores en un sistema distribuido.

Acompañando a los "GenServer", Elixir cuenta con árboles de supervisión como parte del conjunto OTP. Implementados mediante las bibliotecas "Supervisorz "DynamicSupervisor", los árboles de supervisión controlan los procesos que se les ha encargado supervisar, de manera que, si uno falla, el supervisor puede reiniciarlo en un estado anterior estable. Todos los supervisores implementados en Elixir son procesos, por lo tanto se puede encargar a un supervisor que supervise a otro, el cual a su vez tiene otros procesos a los que supervisar, de ahí que se denominen árboles de supervisión.

La máquina virtual sobre la que la se ejecuta el código Elixir se conoce como BEAM. Surge con el nacimiento de Erlang y permite a este lenguaje de programación, y también a Elixir, aprovecha varias de sus características. Ya se ha mencionado la característica distribuida de la máquina virtual, gracias a sus procesos ligeros y aislados. Además de esta característica, la máquina virtual ofrece otras dos características que aumentan el interés por el empleo de Elixir para este proyecto.

La primera de las características consiste en el tipo de planificación preferente, mediante la cual se distribuyen los recursos de la máquina física para el programa. BEAM emplea un planificador por cada núcleo de CPU disponible, cada uno de estos planificadores otorga un número de pasos finito para cada proceso ejecutado en ese planificador. Una vez agotados los pasos disponibles, el planificador pausa el proceso forzosamente y permite la ejecución del siguiente proceso. Esta característica es especialmente interesante en el contexto de los videojuegos debido a que evita que el servidor se quede bloqueado mucho tiempo si se tiene que ejecutar un proceso pesado, como puede ser la carga de un mapa o un elemento no jugable buscando una ruta.

La segunda de las características se trata de la gestión de memoria. A diferencia de otros lenguajes como Java, la máquina virtual BEAM tiene un "Garbage Collector"(GC) por proceso, lo que permite liberar la memoria no utilizada sin bloquear todo el sistema, únicamente el proceso que necesita liberarla.

Todas estas características, tanto del propio Elixir como de la máquina virtual BEAM, le han hecho posicionarse como un lenguaje muy interesante en el mundo de la programación web, gracias al framework de desarrollo de aplicaciones web Phoenix [15] y lo hacen el lenguaje elegido para la implementación del motor de videojuegos y el juego didáctico.

La programación de un servidor TCP en Elixir es una tarea trivial que no requiere de mucho esfuerzo, incluso forma parte de la documentación oficial del propio lenguaje. Esta sencillez solo se traduce en un servidor simple que puede servir para probar conceptos o visualizar comportamientos del sistema, pero no está preparado para ser utilizado en producción. Para este cometido existen bibliotecas preparadas para los retos que presenta ejecutar el servidor en producción.

Phoenix emplea Cowboy/Ranch como servidor HTTP preparado para producción. Aunque, en el caso del proyecto, la biblioteca empleada como servidor TCP/IP es Thousand Island [16], servidor de sockets moderno preparado para su uso en producción.

Java

Java [17] es un lenguaje de programación diseñado para seguir el paradigma de programación orientada a objetos y cuyas fortalezas residen en:

- **Independiente de la plataforma**. Java es un lenguaje de programación compilado. El resultado de los lenguaje compilados, tradicionalmente, está sujeto a la arquitectura del sistema operativo en el que se compila y, por ejemplo, un binario de un lenguaje compilado en Windows no se puede ejecutar en Linux. Java toma otro acercamiento y se ejecuta sobre una máquina virtual, la JVM (Java Virtual Machine). Gracias a esto, el resultado de la compilación es un binario que se puede ejecutar sobre cualquier JVM independientemente del sistema operativo sobre el que se ejecute. El único requisito es que el sistema operativo sobre el que se ejecuta el programa tiene que tener instalada la máquina virtual de Java.
- **Fuertemente tipado**. Los lenguajes de programación fuertemente tipados evitan la asignación de valores a variables cuando el tipo de ambos no coincide y obligan a asignarle un tipo a cada variable, no pudiendo haber variables sin tipo hasta que se las asigna un valor. Esto evita errores durante la programación que puedan llevar al programa a fallar durante la ejecución.
- **Recolector de basura**. Java cuenta con un recolector de basura. El recolector de basura es un mecanismo que tienen algunos lenguajes de programación que permite liberar espacio de memoria de manera automática en tiempo de ejecución. Para ello el recolector de basura libera el espacio ocupado por variables que ya han sido utilizadas y no tienen uso en el futuro de la ejecución.
- **Amplia comunidad**. Es un lenguaje utilizado en multitud de sistemas que aún siguen en funcionamiento y, pese a su antigüedad, sigue siendo escogido por multitud de empresas y personas. Esto hace que Java cuente con una gran comunidad de programadores e ingenieros y por consiguiente se pueden encontrar una gran cantidad de recursos para cualquier necesidad que pueda surgir durante un proyecto.

Estas características hacen de Java un lenguaje muy asequible para los alumnos que se inician en la carrera universitaria de la ingeniería informática. Por este motivo y debido a que es el lenguaje en el que se realizan el resto de prácticas de la asignatura, se emplea Java para proporcionar las soluciones a las prácticas de los alumnos.

SQL

Structured Query Language [18], más conocido como SQL, es el lenguaje de programación de dominio específico diseñado para gestionar datos estructurados en sistemas de gestión de bases de datos o RDBMS por sus siglas en inglés.

SQL permite al motor Hephaestus comunicarse con la base de datos relacional elegida y su uso es obligatorio debido a que dicha base de datos no admite otro lenguaje. Para ello se hace uso de una biblioteca de Elixir llamada Ecto, la cual permite hacer consultas a la base de datos mediante una abstracción propia o mediante código SQL directamente. Esta biblioteca emplea SQL para sus consultas aunque no se haga uso de la opción de código SQL directo.

JSON

JSON (JavaScript Object Notation) [19] es un formato de intercambio de datos diseñado para poder ser legible para los humanos. Basado en un subconjunto de JavaScript [20], es completamente independiente a este y puede ser empleado para transmitir datos entre diferentes lenguajes de programación.

Es el lenguaje empleado por el protocolo admitido por el juego didáctico, el cual se verá en el apartado de otras tecnologías de esta misma sección, y también se usa para almacenar los estados de los actores en la base de datos, por lo tanto es el lenguaje que define el juego.

3.2.2. Gestor de bases de datos

El gestor de bases de datos es el encargado de procesar las consultas realizadas en SQL y modificar la base de datos acorde a esas consultas de ser necesario.

Postgresql

Postgresql [21] es un gestor de bases de datos relacionales de código abierto. Cuenta con su propia extensión del lenguaje SQL y admite tanto la extensión como SQL estándar. Se caracteriza por su robustez, seguridad y rendimiento. Asegura la integridad de los datos al realizar operaciones sobre estos y cuenta con una gran variedad de tipos de datos que pueden ser almacenados. Además cuenta con la posibilidad de extender sus capacidades mediante plugins.

Se elige este gestor debido a sus grandes capacidades y a su extenso catálogo de tipos de datos, el cual incluye JSONB un tipo de dato que permite el almacenamiento de documentos JSON en formato binario y la capacidad de hacer cálculos sobre estos.

3.2.3. Herramientas de documentación

Existen numerosas posibilidades para crear una página web, dependiendo de las necesidades del proyecto se pueden emplear distintas tecnologías como pueden ser HTML y CSS puros, usar bibliotecas

JavaScript como Angular o React, o hacer uso de generadores estáticos. Este último grupo presenta una ventaja frente al resto, el creador de la página web no se tiene que preocupar del estilo de la página ni de otros elementos técnicos del desarrollo web, simplemente tiene que escribir el contenido. Esta ventaja los hace especialmente adecuados para generar documentación web.

Docsify

Docsify [3] es una biblioteca Javascript que genera documentación a partir de documentos Markdown. La generación del contenido web se realiza de manera dinámica, la biblioteca se encarga de cargar los documentos en la página web y realizar las transformaciones necesarias para que el lenguaje Markdown se pueda visualizar como un documento HTML.

Además, Docsify presenta varios temas para personalizar la página web generada al gusto del creador e incluye, por defecto, varios elementos imprescindibles en un página dedicada a la documentación como pueden ser las barras de navegación.

La elección de Docsify para el proyecto surge de la necesidad de crear una página web estática, requisito imprescindible para poder ser alojada en Github Pages. El empleo de otras tecnologías web, como puede ser Angular, suponían el empleo del sistema de CI/CD de Github para poder ser desplegados. Por lo tanto, la elección más lógica significa emplear Docsify, por su despliegue inmediato y su sencillez a la hora de desarrollar la documentación.

3.2.4. Otras tecnologías

Docker

Docker [22] es un sistema de gestión y ejecución de contenedores. Un contenedor es una pieza de software que simula un sistema operativo reducido dentro del sistema operativo de la máquina que ejecuta Docker, permitiendo ejecutar programas dentro de este sistema operativo en miniatura empleando los recursos del sistema operativo donde se ejecuta el contenedor. Para la construcción o ejecución de un contenedor se emplean imágenes, estas imágenes contienen la definición del contenedor y del código que se ejecuta dentro de este. Esto le permite a Docker contar con varias ventajas a la hora de distribuir programas a servidores o usuarios:

- **Control de versiones**. Al crear un contenedor este tiene la capacidad de otorgarle una etiqueta con la versión de la que se trata. Esto permite que el cambio de versiones consista únicamente en crear una nueva imagen con la nueva versión del código y una nueva etiqueta de versión.
- **Portabilidad**. Los contenedores se pueden ejecutar en cualquier sistema con Docker instalado.

- **Escalabilidad**. Existen tecnologías que permiten la duplicación de contenedores, el balance de carga sobre contenedores duplicados y creación y destrucción automática de contenedores. Una de las tecnologías más populares que permite esto es Kubernetes [23].

Aunque la portabilidad de Docker no es de mucha utilidad ya que no proporciona nada nuevo a lo que ya ofrecen Elixir y Java, Docker permite simular un sistema distribuido en la misma máquina física sin necesidad de configurar máquinas virtuales. Esto supone una ventaja a la hora de que los alumnos instalen el proyecto al solo necesitar un archivo *docker-compose* para poner todo el sistema en funcionamiento.

JSON-RPC

JSON-RPC [24] es un protocolo de hilo basado en JSON para realizar llamadas de procedimiento remoto (RPC por sus siglas en inglés). Este protocolo permite notificaciones al servidor y el envío de múltiples llamadas al servidor, las cuales pueden ser respondidas asíncronamente o síncronamente.

El protocolo es independiente del protocolo de transporte empleado, en este proyecto se transporta empleando el protocolo TCP. Otra de las características es que no incluye ningún tipo de autenticación o autorización, se tiene que implementar aparte.

Se trata del protocolo empleado en el proyecto para la comunicación entre los clientes Java y el juego didáctico. Transportado, como ya se ha mencionado, mediante TCP.

Capítulo 4

Análisis

El proyecto cuenta con tres grandes bloques de contenido con código: el motor Hephaestus, el juego didáctico y las prácticas para los alumnos. El código de las prácticas para los alumnos, al tratarse de las soluciones de estas, es sencillo y orientativo, por lo tanto queda excluido del análisis formal del proyecto.

4.1. Motor Hephaestus

Se trata del foco principal del proyecto, por lo tanto el grueso del análisis se concentra en el motor de videojuegos distribuidos Hephaestus.

4.1.1. Análisis inicial

En la planificación del proyecto se mostraron los entregables planeados para la ejecución del proyecto. Estos entregables se dividen en las cinco partes con la que cuenta el proyecto y marcan la pauta de desarrollo del mismo. Por lo tanto, al realizar el análisis de cada una de las partes se requiere de un análisis inicial de todos los entregables que contienen código y que forman cada una de estas, con el objetivo de determinar los requisitos y casos de uso necesarios para poder finalizar el proyecto. El motor Hephaestus cuenta con nueve entregables los cuales, en conjunto, conforman el motor al completo.

Módulo Personaje

El personaje representa la entidad jugable del sistema, toda modificación del estado del juego que se produzca por la acción de un jugador tiene que pasar por el personaje de ese mismo jugador y será esta situación la única que provoque que el personaje realice una acción. Esto no implica que el escenario

no pueda cambiar por acciones externas al jugador, otros tipos de elementos del juego pueden disparar cambios en el estado del juego sin necesidad de recibir una acción por parte del jugador.

Módulo Enemigo

Un enemigo representa una entidad opuesta al personaje, al entrar ambos en contacto dentro del juego estos pueden, a elección del desarrollador del videojuego, entrar en un estado de combate y reducir al vida que tienen ambos a base de golpes al contrario. Por ello estos dos actores son los únicos que cuentan con una estadística de vida. Por estos motivos, tanto el personaje como el enemigo son actores que deben contar con la posibilidad de recibir mensajes que permitan al contrario modificar su estado interno.

Módulo NPC

Un NPC representa un personaje no jugable con el cual se puede interactuar. Por su definición el NPC tiene que permitir a los actores que se lo soliciten iniciar una interacción con él. El resultado de dicha interacción dependerá totalmente del desarrollador que haga uso del motor, por lo tanto el código ha de estar preparado para admitir una gran variedad de posibilidades como resultado a la acción de interactuar. Además de la flexibilidad a la hora de resolver la interacción, cabe la posibilidad de que el desarrollador limite el número de interacciones que permite el NPC.

Módulo Objeto

Un objeto es una entidad que puede ser entregada por otra entidad y que se puede almacenar en el inventario del actor personaje. Además un objeto puede ser volátil, una vez usado desaparece, o equipable, no desaparece una vez empleado y puede desequiparse y volver al inventario. Dependiendo del tipo de objeto utilizará una estadísticas u otras para desempeñar su papel, teniendo en cuenta que el tipo y utilidad del objeto se dejan a valoración del desarrollador del videojuego.

Módulo Ubicación

Una ubicación define los emplazamientos sobre los que transcurre el juego. Por ello el resto de actores están colocados en una de estas ubicaciones. A elección del desarrollador un actor puede cambiar su ubicación durante el transcurso de la partida, el ejemplo más claro es el personaje, el cual necesita de esta característica para avanzar por el mapa del juego.

Módulo Conexión

Una conexión permite viajar entre ubicaciones, pudiendo tener restricciones para realizar dicho viaje. El viaje se realiza entre dos ubicaciones contiguas cuya contigüidad se determina por el hecho de ser ambas dependientes de la misma conexión.

Persistencia

La persistencia incluye todas las funciones necesarias para transferir el estado del juego al almacenamiento permanente de la base de datos. Además del guardado y carga de la partida, se necesitan otras funciones relacionadas con la base de datos, las cuales facilitan todo el control de acceso a los personajes por parte de los jugadores.

Utilidades

En el entregable de utilidades se agrupan todas las funciones que puedan ser de utilidad para el desarrollador de videojuegos y ahorren tiempo de desarrollo de los proyectos que empleen Hephaestus.

4.1.2. Requisitos

Hephaestus se describe como un motor para el desarrollo de videojuegos distribuidos escrito en Elixir. Por lo tanto dicho motor deberá contar, fundamentalmente, con: la capacidad de ser distribuido; la posibilidad de emplearlo para manejar el almacenamiento del estado del juego, ya sea de manera permanente o efímera; la capacidad de simplificar el desarrollo de videojuegos y la modularidad suficiente para admitir futuras extensiones de capacidades.

Con esta descripción como base, a continuación se muestran las tablas de requisitos funcionales y no funcionales del motor Hephaestus.

Identificador	Requisito
RF-01	Tiene que tener la capacidad de autenticar y autorizar a los usuarios.
RF-02	Cada pieza que actúe como actor debe tener un identificador único que sirva para su identificación en todo el sistema.
RF-03	Tiene que permitir la creación de los siguientes actores: Personaje, conexión, enemigo, ubicación, NPC y objeto.
RF-04	Los actores personaje, NPC y enemigo tienen que tener las siguientes estadísticas: Carisma, sabiduría, inteligencia, constitución, destreza y fuerza.

RF-05	Todos los actores tienen que poder devolver el estado actual en el que se encuentran.
RF-06	El actor personaje tiene que poder almacenar objetos en un inventario.
RF-07	El actor personaje tiene que poder equiparse objetos de su inventario para aumentar o disminuir sus estadísticas.
RF-08	El actor personaje tiene que poder subir de nivel tras llegar a un umbral de experiencia conseguida.
RF-09	El actor personaje, el actor NPC y el actor enemigo deben contar con un nivel con el fin de realizar resoluciones de conflictos.
RF-10	El actor personaje y el actor enemigo deben contar con un contador de vida.
RF-11	El actor personaje tiene que indicar si se encuentra en combate o no.
RF-12	El actor personaje tiene que tener la capacidad de recordar la ubicación en la que se encuentra y cambiarla si se requiere.
RF-13	Tiene que tener los mecanismos necesarios para asociar un personaje a un usuario.
RF-14	El actor enemigo tiene que proporcionar una cantidad de experiencia arbitraria si su vida ha llegado a cero.
RF-15	El actor enemigo tiene que contar con una estadística a partir de la cual se derive el daño de ataque que tiene.
RF-16	El actor NPC tiene que contar con un resultado para una interacción exitosa.
RF-17	El actor NPC tiene que contar con un límite de interacciones que pueda ser opcional. En caso de no ser opcional, el resultado de obtener la interacción debe ser exitoso hasta que se consuman todos los intentos.
RF-18	El actor conexión debe almacenar exactamente dos ubicaciones.
RF-19	El actor conexión debe mostrar la siguiente ubicación en función de la ubicación actual.
RF-20	El actor conexión debe tener un nivel fijo.
RF-21	El actor conexión debe tener la capacidad de tener un objeto que se pueda emplear desde el juego para no permitir el cruce hasta la obtención del objeto.
RF-22	El actor objeto tiene que tener un nivel fijo.
RF-23	El actor objeto tiene que tener el tipo de objeto del que se trata.
RF-24	El actor objeto tiene que tener un valor numérico que represente diferentes cosas dependiendo del tipo de objeto.
RF-25	El actor objeto tiene que tener la opción de añadir una estadística sobre la que actúe el objeto.
RF-26	El estado del juego se tiene que poder almacenar de forma permanente en una base de datos.
RF-27	Tiene que proporcionar utilidades para simplificar el enrutamiento de actores en arquitecturas distribuidas.
RF-28	Tiene que proporcionar utilidades para la supervisión de procesos.
RF-29	Tiene que poder crear nuevos usuarios.
RF-30	El actor ubicación debe poder añadir y eliminar actores objetos

RF-31	El actor ubicación debe poder añadir actores conexión
RF-32	El actor ubicación debe poder añadir actores NPC
RF-33	El actor ubicación debe poder añadir actores enemigo
RF-34	El actor ubicación debe devolver los actores enemigos que se encuentren en el actor ubicación y cuya vida sea mayor a cero

Tabla 4.1: Hephaestus. Requisitos funcionales

Identificador	**Requisito**
RNF-01	Tiene que estar escrito en el lenguaje de programación Elixir.
RNF-02	Tiene que estar especializado en juegos de rol en mundos de fantasía.
RNF-03	Las contraseñas deben ser almacenadas empleando una función hash.
RNF-04	Tiene que estar pensado para su futura modificación haciendo uso de una arquitectura de módulos.
RNF-05	La definición de tipos de objetos tiene que ser dependiente del juego que emplea el motor.
RNF-06	La base de datos empleada debe ser Postgresql.
RNF-07	El control de los actores debe residir en el motor.

Tabla 4.2: Hephaestus. Requisitos no funcionales

4.1.3. Casos de uso

Una vez se han determinado los requisitos de esta parte del proyecto, se acotan los casos de uso a implementar durante el transcurso de todo el proyecto. Las Figuras 4.1, 4.2 y 4.3 muestran los casos de uso del motor Hephaestus. Se divide el diagrama en varios subdiagramas debido al tamaño que tendría un único diagrama con todos los casos de uso de Hephaestus, de esta manera se consigue una mejor legibilidad de los casos de uso. Gracias a los casos de uso se puede tomar un métrica del estado del proyecto, si todos los casos de uso están implementados este se puede dar por finalizado.

Además de los diagramas mostrados se proporciona una serie de tablas que describen en profundidad cada uno de los casos de uso. Existen múltiples opciones para realizar este tipo de tablas, pero en este caso se opta por tablas pequeñas, que no incluyen mucha información sobre el caso de uso, debido a la cantidad de casos de uso presentes en el proyecto. Las tablas cuentan con el identificador y el nombre del caso de uso, un listado de referencias a los requisitos funcionales u otros casos de uso afectados por el caso de uso en cuestión y una descripción del caso de uso. De este forma se consigue resumir la información más importante que no pudiera quedar clara mediante los diagramas mencionados anteriormente.

Caso de uso	CU-01	Crear usuario
Referencias	RF-13, RF-29	
Descripción	El juego solicita crear un nuevo usuario proporcionando el nombre de usuario y la definición del nuevo personaje asociado al nuevo usuario.	

Tabla 4.4: Caso de uso de Hephaestus CU-01

Caso de uso	CU-02	Autenticar usuario
Referencias	RF-01	
Descripción	El juego solicita comprobar si la identidad del usuario proporcionado existe en el sistema. Esta comprobación se hace mediante el uso de usuario y contraseña.	

Tabla 4.6: Caso de uso de Hephaestus CU-02

Caso de uso	CU-03	Guardar estado
Referencias	RF-26	
Descripción	El juego solicita guardar su estado actual en la base de datos. Debe solicitar el estado de todos los actores del sistema para poder guardarlos en la base de datos.	

Tabla 4.8: Caso de uso de Hephaestus CU-03

Caso de uso	CU-04	Cargar estado
Referencias	RF-03	
Descripción	El juego solicita el estado de las entidades que conforman el juego almacenados en la base de datos.	

Tabla 4.10: Caso de uso de Hephaestus CU-04

Caso de uso	CU-05	Crear entidad
Referencias	RF-03, CU-04	
Descripción	El juego solicita la creación y puesta en funcionamiento de las entidades que conforman el juego.	

Tabla 4.12: Caso de uso de Hephaestus CU-05

Caso de uso	CU-06	Consultar estado
Referencias	RF-05	
Descripción	El juego solicita el estado actual del actor entidad correspondiente al tipo e identificador proporcionados por el juego.	

Tabla 4.14: Caso de uso de Hephaestus CU-06

Caso de uso	CU-07	Obtener daño de ataque
Referencias	RF-05, RF-15	
Descripción	El juego solicita el valor de ataque de una entidad perteneciente al juego que cuente con vida (Enemigo o Personaje).	

Tabla 4.16: Caso de uso de Hephaestus CU-07

Caso de uso	CU-08	Consultar si está vivo
Referencias	RF-05, RF-10	
Descripción	El juego consulta si la entidad solicitada cuenta con un valor de vida superior a cero y por lo tanto si se considera que está con vida. Solo se asocia a entidades que tienen vida (Enemigo o Personaje).	

Tabla 4.18: Caso de uso de Hephaestus CU-08

Caso de uso	CU-09	Herir
Referencias	RF-10	
Descripción	El juego indica la entidad con vida (Enemigo o Personaje) que debe reducir una cantidad, también indicada por el juego, de su campo de vida.	

Tabla 4.20: Caso de uso de Hephaestus CU-09

Caso de uso	CU-10	Consultar recompensa
Referencias	RF-05, RF-14, CU-08	
Descripción	El juego consulta la cantidad correspondiente a la recompensa entregada por la entidad Enemigo si se cumple la condición de que su vida se encuentra a cero.	

Tabla 4.22: Caso de uso de Hephaestus CU-10

Caso de uso	CU-11	Consultar siguiente ubicación
Referencias	RF-05, RF-19	
Descripción	El juego consulta la siguiente ubicación de una entidad Conexión dada una de las dos ubicaciones que almacena esta.	

Tabla 4.24: Caso de uso de Hephaestus CU-11

Caso de uso	CU-12	Consultar valor de la estadística
Referencias	RF-05, RF-24, RF-25	
Descripción	El juego consulta el valor de la estadística asociada a la entidad Objeto objetivo.	

Tabla 4.26: Caso de uso de Hephaestus CU-12

Caso de uso	CU-13	Equipar objeto
Referencias	RF-06, RF-07	
Descripción	El juego proporciona el identificador de una entidad Objeto en el inventario de la entidad Personaje para equiparlo en el espacio correspondiente.	

Tabla 4.28: Caso de uso de Hephaestus CU-13

Caso de uso	CU-14	Cambiar ubicación
Referencias	RF-12	
Descripción	El juego indica la entidad Ubicación a la cual la entidad Personaje ha de moverse.	

Tabla 4.30: Caso de uso de Hephaestus CU-14

Caso de uso	CU-15	Entrar en combate
Referencias	RF-11	
Descripción	El motor cambia el estado de la entidad Personaje para que esta se encuentre en estado de combate.	

Tabla 4.32: Caso de uso de Hephaestus CU-15

Caso de uso	CU-16	Añadir experiencia
Referencias	RF-08, RF-09	
Descripción	El juego proporciona una cantidad de experiencia a la entidad Personaje, la cual puede subir de nivel si sobrepasa el umbral del nivel.	

Tabla 4.34: Caso de uso de Hephaestus CU-16

Caso de uso	CU-17	Añadir al inventario
Referencias	RF-06	
Descripción	El juego proporciona una entidad Objeto la cual se añade al inventario de la entidad Personaje.	

Tabla 4.36: Caso de uso de Hephaestus CU-17

Caso de uso	CU-18	Consultar enemigos
Referencias	RF-05, RF-34	
Descripción	El juego solicita el listado de entidades Enemigo que se encuentran en la entidad Ubicación sobre la cual se piden los datos.	

Tabla 4.38: Caso de uso de Hephaestus CU-18

Caso de uso	CU-19	Añadir objeto
Referencias	RF-30	
Descripción	El juego proporciona una entidad Objeto la cual se añade a la entidad Ubicación objetivo.	

Tabla 4.40: Caso de uso de Hephaestus CU-19

Caso de uso	CU-20	Eliminar objeto
Referencias	RF-30	
Descripción	El juego proporciona una entidad Objeto la cual se elimina de la entidad Ubicación objetivo.	

Tabla 4.42: Caso de uso de Hephaestus CU-20

Caso de uso	CU-21	Añadir conexión
Referencias	RF-31	
Descripción	El juego proporciona una entidad Conexión la cual se añade a la entidad Ubicación objetivo.	

Tabla 4.44: Caso de uso de Hephaestus CU-21

Caso de uso	CU-22	Añadir conexión
Referencias	RF-32	
Descripción	El juego proporciona una entidad NPC la cual se añade a la entidad Ubicación objetivo.	

Tabla 4.46: Caso de uso de Hephaestus CU-22

Caso de uso	CU-23	Añadir enemigo
Referencias	RF-33	
Descripción	El juego proporciona una entidad Enemigo la cual se añade a la entidad Ubicación objetivo.	

Tabla 4.48: Caso de uso de Hephaestus CU-23

Caso de uso	CU-24	Interactuar
Referencias	RF-16	
Descripción	El juego solicita interactuar con una entidad NPC.	

Tabla 4.50: Caso de uso de Hephaestus CU-24

Caso de uso	CU-25	Modificar límite de interacción
Referencias	RF-17	
Descripción	El juego solicita la modificación de los límites puestos por la entidad NPC para que una interacción con dicha entidad sea considerada como válida.	

Tabla 4.52: Caso de uso de Hephaestus CU-25

4.1.4. Modelos

Aunque durante todo el transcurso del análisis y del diseño se van a utilizar diagramas de UML con notación de clases por estandarización, se tiene que hacer notar que Elixir no cuenta con estos mecanismos y a la hora de implementar la solución diseñada se tiene que hacer uso de módulos, structs y comportamientos, conceptos propios de Elixir. De la misma manera se emplea herencia, tampoco

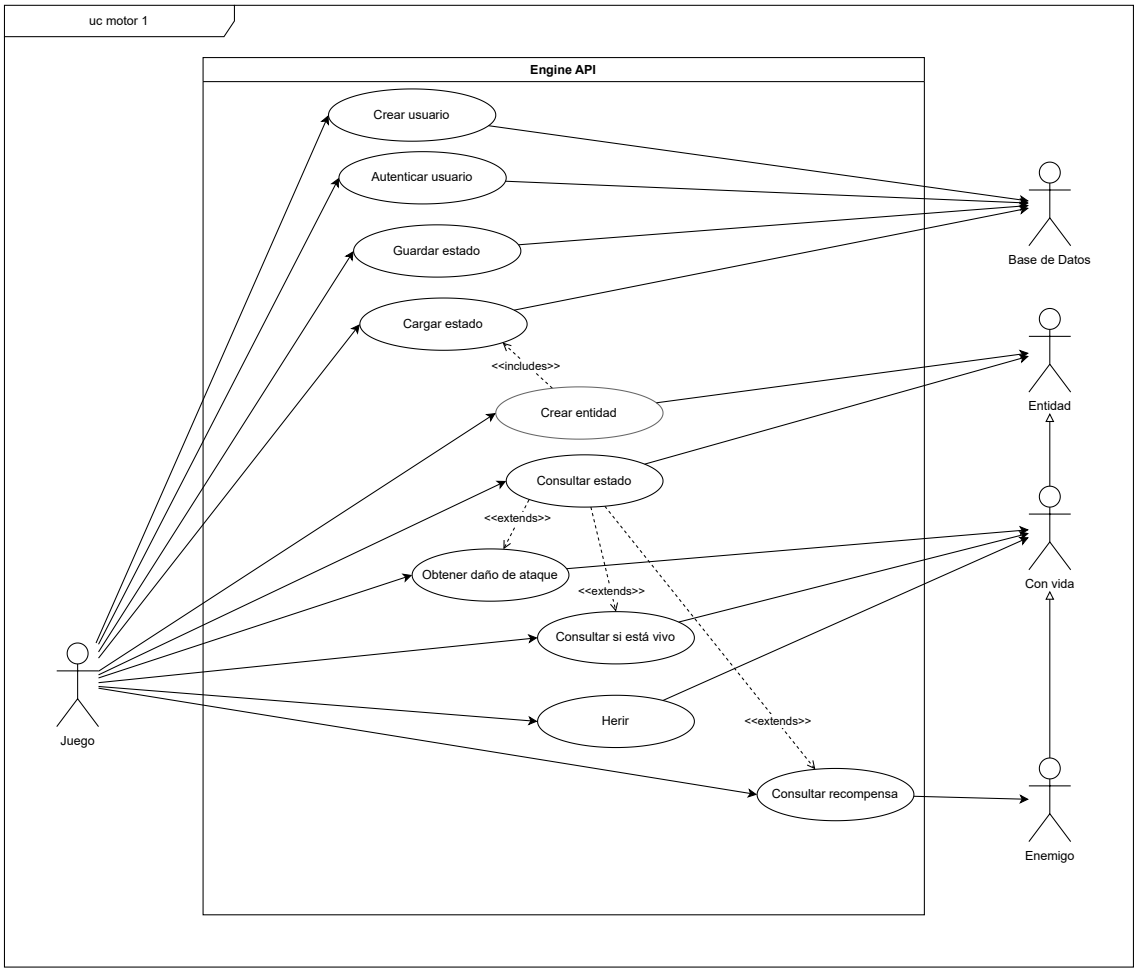


Figura 4.1: Diagrama de casos de uso de Hephaestus (Base de datos y Enemigo)

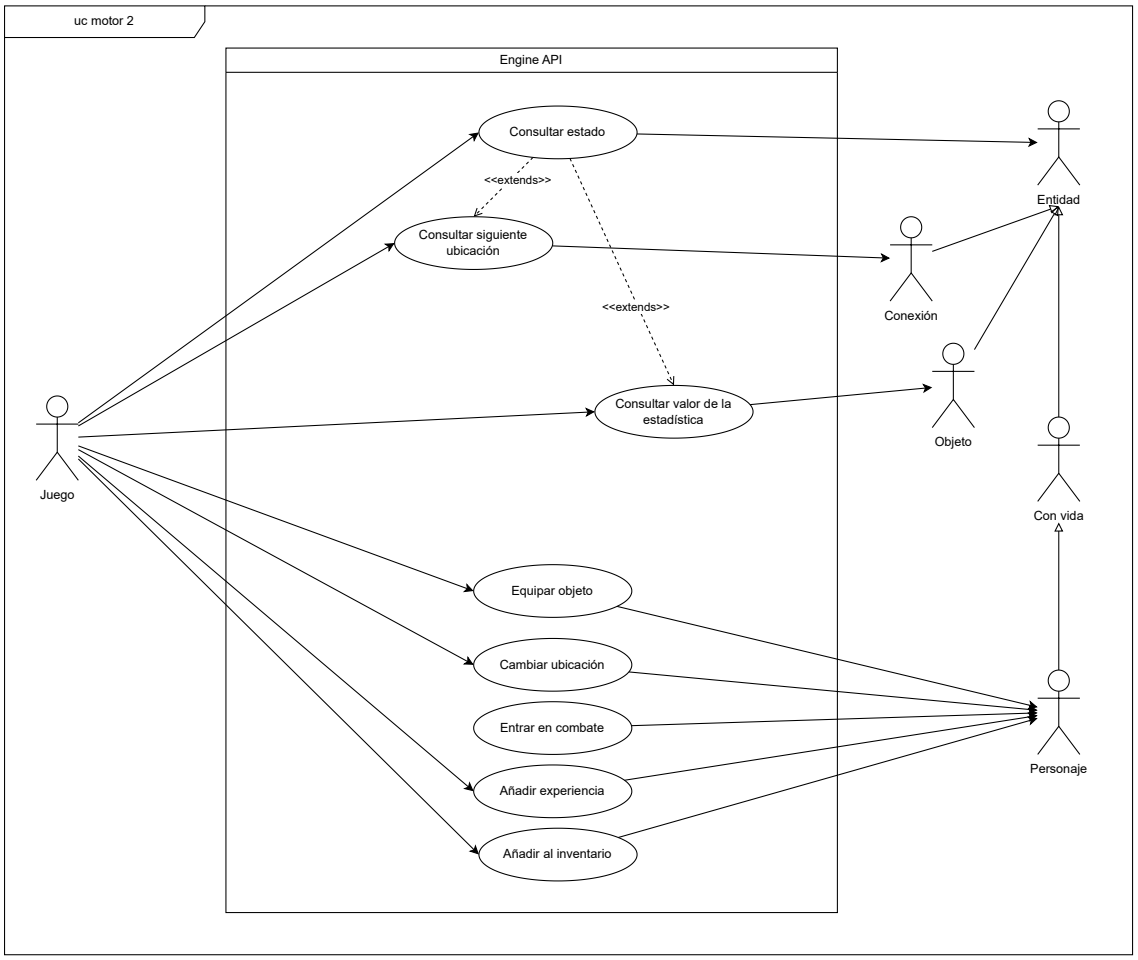


Figura 4.2: Diagrama de casos de uso de Hephaestus (Conexión, Objeto y Personaje)

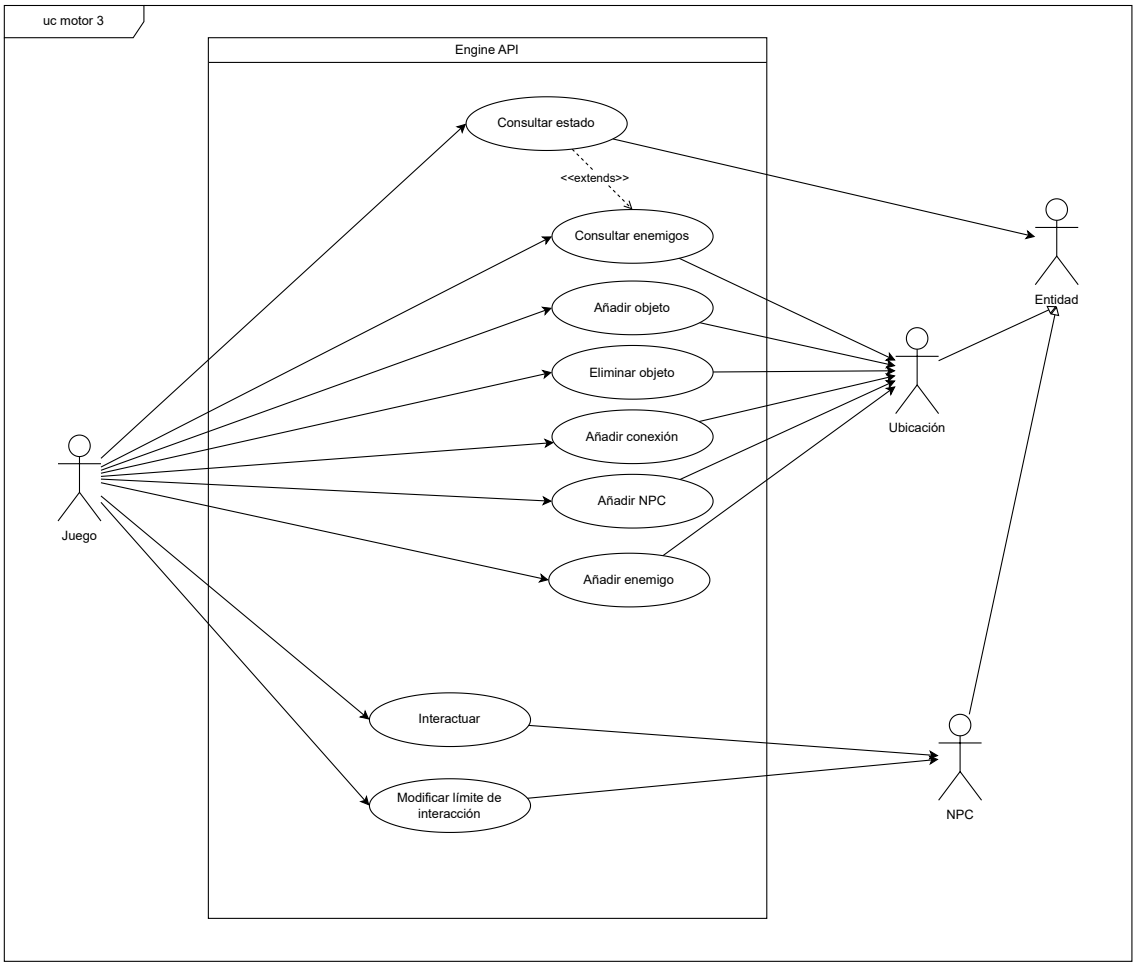


Figura 4.3: Diagrama de casos de uso de Hephaestus (Ubicación y NPC)

estando esta disponible en Elixir, y por los mismos motivos se traduce una vez llegada la implementación. De esta forma el análisis y el diseño se pueden emplear para recrear el proyecto en otros lenguajes de programación.

En primer lugar, en el diagrama 4.4 se muestra un concepto inicial, tomando en cuenta más casos de uso que los que se mostraron anteriormente y que proporciona una imagen mayor de los componentes internos del motor de videojuegos Hephaestus. Desarrollando la información proporcionada por este diagrama se pueden realizar modificaciones al motor en un futuro.

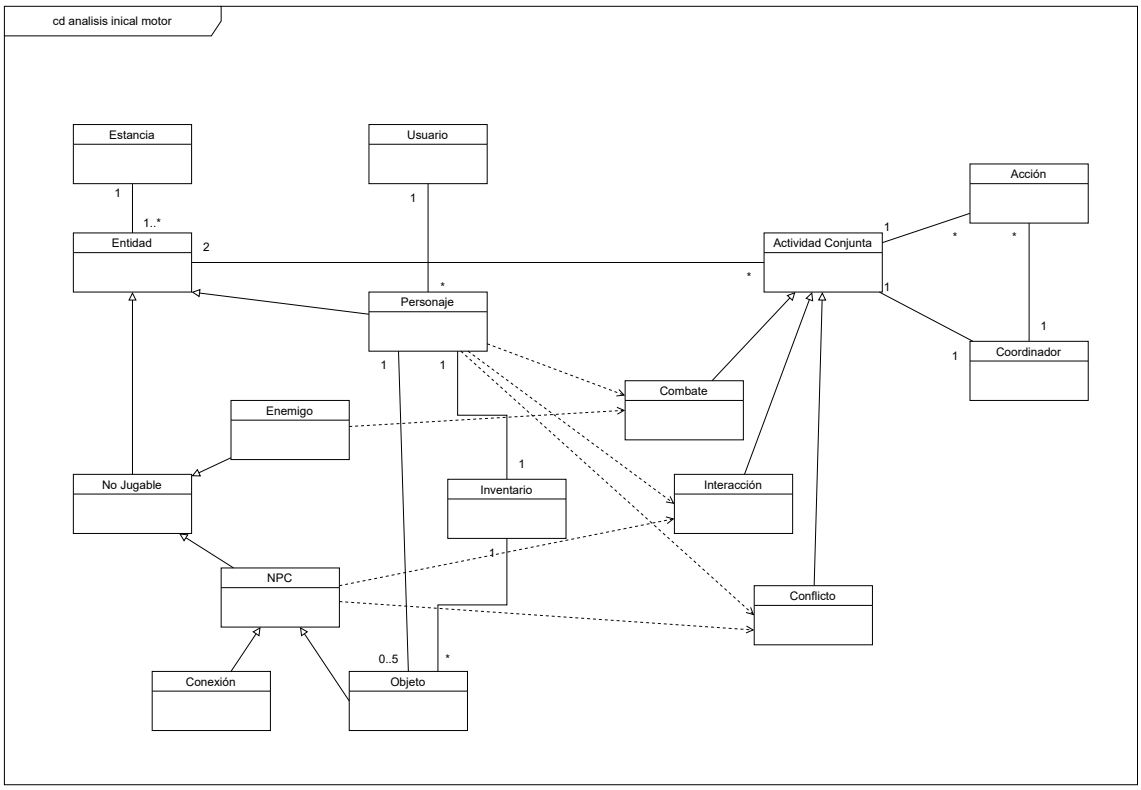


Figura 4.4: Diagrama de análisis Hephaestus completo

De este análisis inicial se eliminan elementos opcionales, cuya ausencia no impide la creación de juegos funcionales, debido a la falta de tiempo que supone la implementación completa del motor, además del resto del proyecto. Los elementos eliminados son los siguientes:

- Acción.
- Coordinador.
- Acción conjunta.
- Combate.
- Interacción.

- Conflicto.

Al eliminar estos elementos la implementación de la lógica que proporcionan estos elementos se delega en el juego que hace uso del motor, dándole a este mayor flexibilidad.

A partir de la eliminación de estos elementos se crea el diagrama de análisis final, mostrado en la figura 4.5. Este diagrama deja atrás la simpleza del diagrama inicial, puesto que este sirve como punto de partida pero no se encuentra desarrollado por completo, y desarrolla el análisis completo del motor.

Se puede apreciar el cambio que sufre el análisis final, mostrado en el diagrama de la figura 4.5, respecto al diagrama inicial, mostrado en la figura 4.4. El análisis inicial del motor, al tratarse de una primera iteración sobre lo que se quiere conseguir al finalizar el proyecto, cuenta con una distribución de clases la cual difiere de la distribución final. Este análisis final cuenta ya con una estructura completamente adaptada a los requisitos funcionales y no funcionales de esta parte del proyecto. Por ende, prescinde de clases no necesarias y adapta la herencia de todas ellas para cumplir con las restricciones del lenguaje empleado.

El mayor cambio conceptual que se aprecia en el análisis final sobre el inicial es la transformación del concepto de entidad. Al iniciar el análisis *Entity* hace referencia a los elementos del videojuego, de los cuales el motor está encargado, que se encuentran en una ubicación (*Location*). Una vez se profundiza el análisis el concepto de *Entity* se transforma y pasa a representar todo elemento del cual el motor está a cargo. Por ello, toda entidad cuenta con un identificador único, por el cual puede ser referenciada desde el contexto del videojuego y, que el motor sepa sobre que entidad se debe ejecutar la acción solicitada por dicho contexto.

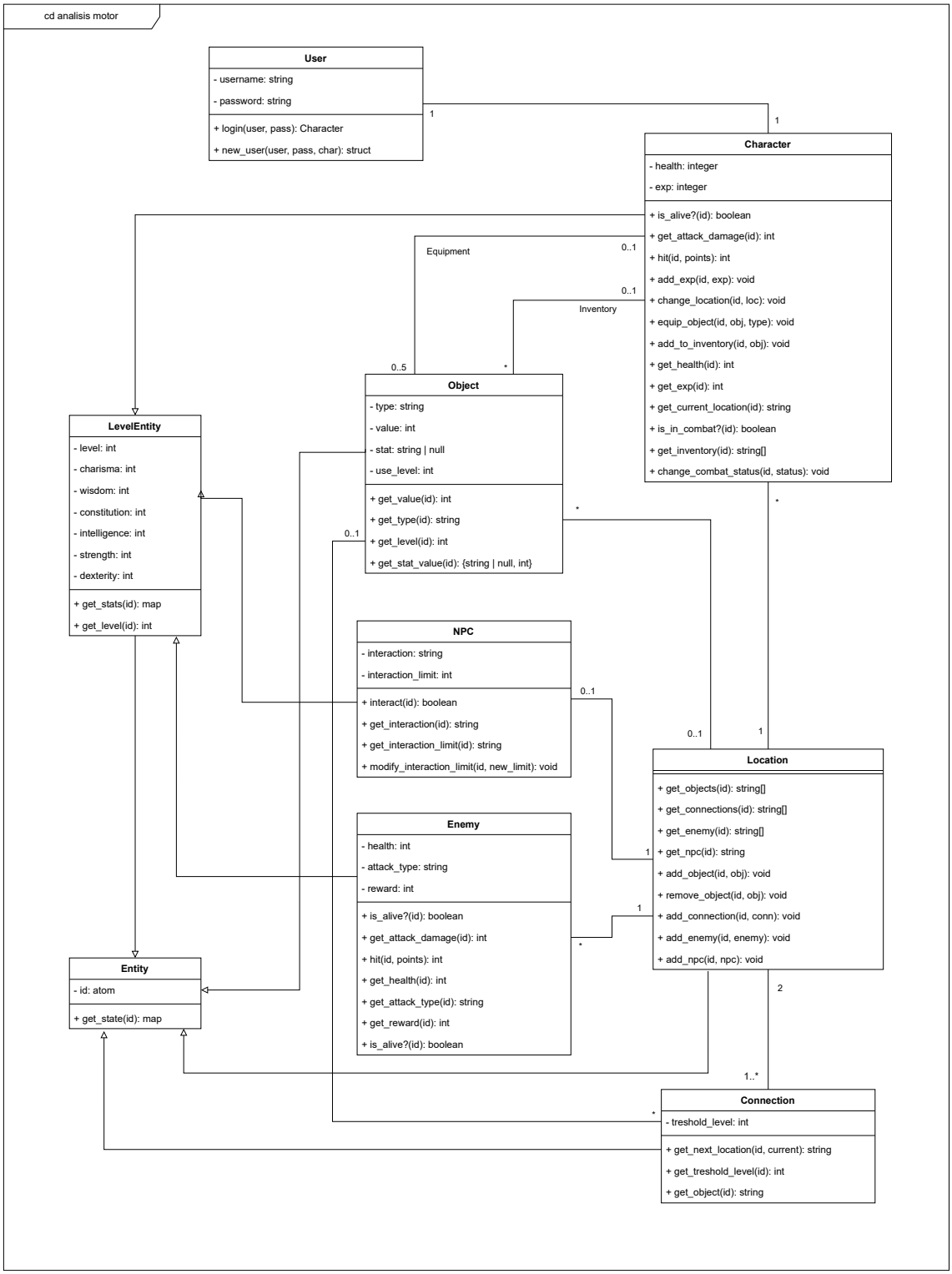


Figura 4.5: Diagrama de análisis de Hephaestus

4.2. Juego didáctico

Al tratarse de un foco menor respecto a Hephaestus, el análisis del juego didáctico no es tan extenso y profundo como lo es el del motor. Pese a ello, el análisis es suficiente para que el diseño posterior cumpla con las condiciones suficientes para el correcto funcionamiento de las prácticas.

4.2.1. Análisis inicial

Tal y como se ha realizado con Hephaestus, el juego didáctico requiere de un análisis inicial de los entregables planificados en el Capítulo 2, mediante el cual definir los requisitos y casos de uso necesarios para la correcta ejecución de esta parte del proyecto.

Servidor

La estructura del proyecto obliga al jugador a emplear uno o varios clientes para comunicarse con el juego. Por lo tanto el juego ha de ser un servidor que pueda recibir instrucciones por parte de los clientes. Las características de este servidor incluyen admitir peticiones mediante TCP/IP, emplear el protocolo JSON-RPC y estar preparado para un entorno profesional con varias peticiones concurrentes y múltiples usuarios.

Handlers

Los handlers son los encargados de resolver un mensaje concreto recibido por el servidor, dados los datos necesarios para la labor. El servidor es el encargado de solicitar la resolución de un mensaje al handler específico y estos devuelven el resultado al servidor, por lo tanto, toda la lógica de negocio para poder superar el videojuego recae sobre los handlers. A la hora de resolver el resultado los handlers tienen que interactuar con el motor y mandarle instrucciones para modificar el estado de la partida.

Gestores de elementos remotos

Uno de los objetivos del juego es enseñar a los alumnos el funcionamiento de sistemas distribuidos y, para ello, el juego se trata como uno de estos sistemas. Los actores del juego están distribuidos entre varias máquinas independientes, tal y como se requiere en un sistema distribuido, y para poder mantener la comunicación con los actores alojados en máquinas diferentes a la que está alojado el juego este requiere de la ayuda de un gestor de elementos remotos. Estos gestores se encargan de buscar a los actores por la

red que se les indique, comprobar que estos están conectados, enviar la petición que requiera el handler y devolver la respuesta que reciban del elemento remoto.

4.2.2. Requisitos

El juego didáctico se caracteriza por actuar como servidor para clientes creados por los alumnos, mediante los cuales se puede resolver el juego. Es fundamental que se adapte a las necesidades del curso en el que se emplea.

Por lo tanto las siguientes listas muestran los requisitos funcionales y no funcionales del juego didáctico.

Identificador	Requisito
RF-01	Tiene que implementar un servidor de tipo TCP que admita comunicaciones concurrentes.
RF-02	El cliente tiene que poder conectarse empleando el puerto 3000 TCP.
RF-03	Tiene que admitir, únicamente, comunicaciones que empleen el protocolo JSON-RPC.
RF-04	Tiene que controlar todos los actores de tipo personaje y enemigo de forma remota.
RF-05	Tiene que definir las rutas de las máquinas en la que los actores de tipo personaje y enemigo se van a ejecutar.
RF-06	Tiene que contar con los suficientes procedimientos remotos para ser completado.
RF-07	Tiene que contar con un procedimiento remoto que devuelva la información correcta para la ejecución del resto de procedimientos.
RF-08	Toda gestión de la base de datos se tiene que delegar al motor Hephaestus.
RF-09	Tiene que implementar un procedimiento que ejecute un guardado del estado del juego de manera permanente.
RF-10	Los procedimientos tienen que delegar la gestión de estados de los actores al motor Hephaestus.
RF-11	Tiene que desarrollarse sobre un mundo compartido. Esto es, en caso de que se conecten dos personajes, las acciones de uno de los personajes sobre el mundo afecta al mundo que ve el otro personaje, es decir, es el mismo mundo.
RF-12	Tiene que implementar un procedimiento para la autenticación de usuarios.
RF-13	Al iniciar su ciclo de vida tiene que cargar de forma automática todos los actores no correspondientes a un personaje.
RF-14	Tiene que contar con un procedimiento mediante el cual el jugador se pueda autenticar

Tabla 4.53: Juego didáctico. Requisitos funcionales

Identificador	Requisito
RNF-01	Tiene que estar escrito en el lenguaje de programación Elixir.
RNF-02	Tiene que hacer uso del motor Hephaestus.
RNF-03	La definición del juego tiene que estar escrita en JSON.
RNF-04	Su longitud tiene que estar adaptada para el contexto de la asignatura en el que es utilizado.

Tabla 4.54: Juego didáctico. Requisitos no funcionales

4.2.3. Casos de uso

Al igual que se ha hecho con Hephaestus, esta parte del proyecto cuenta con su diagrama de casos de uso derivado de los requisitos acotados para el juego didáctico y a partir del cual se realiza el análisis completo del mismo en la siguiente sección. En la figura 4.6 se muestra dicho diagrama.

Caso de uso	CU-01	Iniciar sesión de juego
Referencias	RF-13, RF-08	
Descripción	El juego, de forma automática, obtiene todos los datos guardados en la base de datos sobre el último mundo de juego e inicia todos las entidades del motor que conforma dicho mundo.	

Tabla 4.56: Caso de uso del juego didáctico CU-01

Caso de uso	CU-02	Iniciar sesión
Referencias	RF-01, RF-02, RF-08, RF-14	
Descripción	El actor jugador realiza una llamada, la cual contiene su nombre de usuario y contraseña, mediante la cual inicia sesión en el mundo del juego con su personaje.	

Tabla 4.58: Caso de uso del juego didáctico CU-02

Caso de uso	CU-03	Guardar partida
Referencias	RF-01, RF-02, RF-08, RF-09	
Descripción	El actor jugador realiza una llamada, a partir de la cual se realiza un proceso para guardar el estado actual del juego y de la entidad Personaje asociada a ese jugador.	

Tabla 4.60: Caso de uso del juego didáctico CU-03

Caso de uso	CU-04	Inspeccionar personaje
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual se le devuelve la información sobre su entidad Personaje.	

Tabla 4.62: Caso de uso del juego didáctico CU-04

Caso de uso	CU-05	Inspeccionar ubicación
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual se le devuelve la información sobre la entidad Ubicación en la que se encuentra la entidad Personaje del jugador.	

Tabla 4.64: Caso de uso del juego didáctico CU-05

Caso de uso	CU-06	Inspeccionar conexión
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual se le devuelve la información sobre la entidad Conexión solicitada.	

Tabla 4.66: Caso de uso del juego didáctico CU-06

Caso de uso	CU-07	Equipar objeto
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual la entidad Personaje del jugador se equipa una de las entidades Objeto que se encuentran en su inventario.	

Tabla 4.68: Caso de uso del juego didáctico CU-07

Caso de uso	CU-08	Coger objeto
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual la entidad Personaje del jugador recoge una entidad Objeto y la almacena en su inventario. La entidad Objeto se ha de encontrar en la misma entidad Ubicación que el personaje y, una vez recogida, se debe eliminar de esta.	

Tabla 4.70: Caso de uso del juego didáctico CU-08

Caso de uso	CU-09	Moverse
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual la entidad Personaje del jugador se mueve a una nueva entidad Ubicación adyacente a la que se encuentra. Si en la nueva ubicación se haya una o varias entidades Enemigo se modifica el estado del personaje del jugador a estar en combate.	

Tabla 4.72: Caso de uso del juego didáctico CU-09

Caso de uso	CU-10	Interactuar con NPC
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual se interactúa con una entidad NPC ubicada en la misma entidad Ubicación que el personaje del jugador.	

Tabla 4.74: Caso de uso del juego didáctico CU-10

Caso de uso	CU-10	Interactuar con NPC
Referencias	RF-01, RF-02, RF-06, RF-08	
Descripción	El actor jugador realiza una llamada mediante la cual la entidad Personaje del jugador ataca a una de las entidades Enemigo que se encuentren en la misma ubicación que el personaje. Los enemigos de la ubicación también realizan su ataque el cual resta puntos de vida al personaje.	

Tabla 4.76: Caso de uso del juego didáctico CU-11

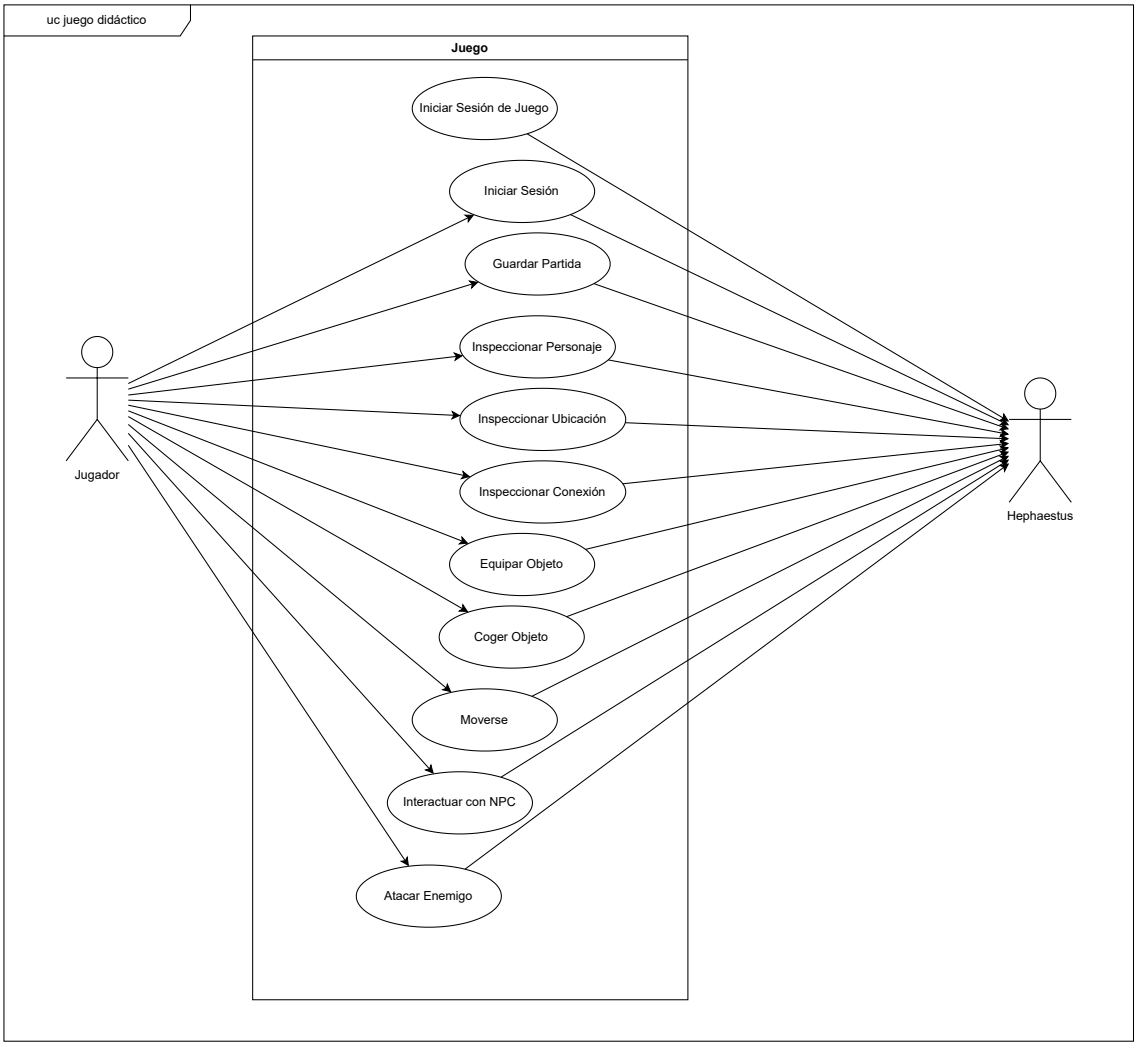


Figura 4.6: Diagrama de casos de uso del juego didáctico

4.2.4. Modelos

Teniendo en cuenta las características y los requisitos anteriores. El análisis completo del juego didáctico se muestra en el diagrama representado en la figura 4.7.

En este diagrama se aprecia cómo el juego no supone un análisis tan profundo como el motor debido a su sencillez. Además se aprecia como este depende de los diferentes módulos que proporciona el motor Hephaestus.

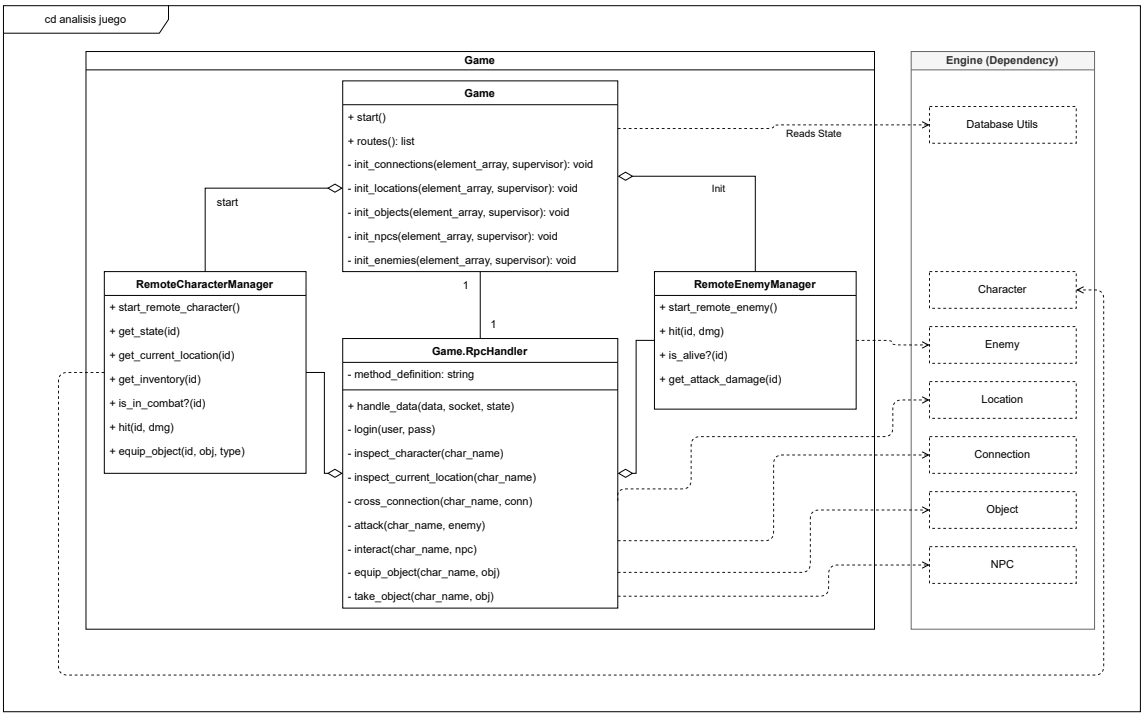


Figura 4.7: Diagrama de análisis juego didáctico

4.3. Otras partes del proyecto

Llegados a este punto del análisis del proyecto completo quedan por ser analizados todos los elementos cuyo valor al proyecto no incluye programación, véase la documentación o las prácticas para los alumnos, o cuya programación es tan sencilla, véase la solución de las prácticas, que someter el entregable a un proceso de análisis con requisitos y casos de uso conllevaría más tiempo que la propia ejecución del entregable. No por ello se tiene que obviar el análisis de estos entregables y partes del proyecto, por lo tanto en esta sección se realiza un análisis sencillo y básico de todos los componentes del proyecto que no han sido mencionados en las dos secciones anteriores.

4.3.1. Hephaestus

Aunque Hephaestus cuenta, en su mayor parte, con entregables de código, existe un entregable de esta parte del proyecto que no se ubica dentro de esta categoría. Por lo tanto, se requiere de un análisis básico para guiar el diseño posterior. Este entregable mencionado es la imagen Docker del propio motor.

Imagen de Hephaestus

Con el fin de levantar un contenedor y que ejecute el programa correspondiente es necesario crear una imagen de dicho contenedor primero. Esta imagen permite a Docker saber que comandos tiene que ejecutar y en que orden con el fin de poner en funcionamiento el contenedor. La imagen contiene también el código necesario para la ejecución. Para poder crear esta imagen se requiere de un fichero especial que contiene las instrucciones que debe seguir Docker a la hora de construir la imagen. Este fichero se denomina Dockerfile.

En el caso de Hephaestus, la imagen final debe tener una serie de características que permita levantar un contenedor Docker con un motor de Hephaestus funcional. Además la imagen debe ocupar el menor espacio posible para facilitar su distribución. Por ello la técnica recomendada en estos casos es compilar el código en una etapa auxiliar y transferir el código compilado a la imagen final ejecutable. Todo ello en el entorno .ªlpine", el cual se trata de una distribución de Linux diseñada específicamente para estos casos poniendo como prioridad una ocupación de disco mínima.

4.3.2. Juego didáctico

Al igual que el motor Hephaestus, el juego didáctico contiene dos entregables que siguen la misma filosofía que la imagen del motor al tratarse de dos imágenes Docker. En este caso la imagen del juego didáctico y la imagen de la base de datos necesaria para almacenar el estado del juego de permanentemente.

Imagen del juego

Al igual que sucede con la imagen del motor, la imagen del juego didáctico tiene como función poder distribuir de forma sencilla el juego y que el despliegue en los equipos de los alumnos pueda ser fácilmente reproducible. Sigue la misma filosofía que la imagen ya mencionada y, por lo tanto, se emplean las mismas técnicas ya mencionadas con el fin de reducir el tamaño final al mínimo posible.

Imagen de la base de datos

El juego didáctico requiere de una base de datos para poder almacenar de forma permanente el estado del juego. El estado almacenado consiste en el estado inicial del juego, el estado del último punto de guardado de la partida de los jugadores y toda la información necesaria para la autenticación de los ya mencionado jugadores. Por lo tanto es necesario modificar la imagen por defecto que proporcionan los desarrolladores de Postgresql. Esta labor es trivial y está ampliamente documentada.

4.3.3. Prácticas

Hasta este punto todo el análisis se ha centrado en las dos partes principales del proyecto: Hephaestus y el juego didáctico. Pero todavía falta una tercera parte, menor en importancia y tamaño, que complementa a las dos anteriores y facilita el trabajo del personal docente que haga uso de este proyecto. Las prácticas proporcionadas en esta parte pretenden simplificar el trabajo del docente proporcionando un punto de partida para que los alumnos realicen en los laboratorios de la asignatura. Aunque el docente tiene en su poder el modificar las prácticas como sea más conveniente, estas están pensadas para poder ser usadas sin hacer ninguna modificación, siguiendo un hilo conductor entre ellas y con una dificultad incremental.

Práctica introductoria

La primera práctica con la que se encuentra el alumno no se trata de una práctica per se. Consiste en un documento introductorio que incluye un resumen del resto de las prácticas, dando una visión global al alumno de lo que se va a encontrar a lo largo de las sesiones. Complementando la introducción a las prácticas, este documento ofrece un listado de todas las tecnologías necesarias para poder desarrollar las prácticas, así como las instrucciones para operar el despliegue del juego en una máquina local.

Práctica 1

En esta práctica se le presentan al alumno elementos fundamentales de la comunicación cliente-servidor que se emplea durante todas las prácticas, de esta manera el alumno adquiere los conocimientos básicos necesarios para superarlas. Entre estos conocimientos básicos se encuentra también el funcionamiento del protocolo JSON-RPC, la otra pieza fundamental en la comunicación entre los cliente Java y el juego, que actúa como servidor.

Práctica 2

Continuando con el hilo de la primera práctica se le muestra a los alumnos el funcionamiento interno de los actores en un sistema distribuido. Esto enmarca la segunda práctica y el segundo entregable del proyecto. Un actor en un sistema distribuido son unidades de cómputo que representan una parte de la lógica de la aplicación. El control de estos actores recae sobre el motor y el juego hace uso de dicho motor para enviar mensajes a los actores. Son un elemento fundamental en muchos sistemas distribuidos y, por lo tanto, parte del temario de la asignatura. Mediante esta práctica se pretende que el alumno afiance los conocimientos obtenidos sobre los actores mediante la puesta en práctica.

Práctica 3

En esta última práctica se pretende enseñar al alumno un poco más sobre el funcionamiento interno de Hephaestus. Esto permite una ligera introducción al lenguaje de programación Elixir, al que el alumno no se ha enfrentado previamente, con un paradigma completamente diferente a lo habitual durante la carrera. Además muestra al alumno como se implementan los actores en un caso real, poniendo en perspectiva lo aprendido y mostrando la utilidad real de la teoría vista en clase.

Soluciones

Al igual que se proporciona documentación para proponer unas prácticas de laboratorio a los alumnos, es también necesario proporcionar una posible solución al docente con el fin de facilitar la labor de corrección. Al igual que con las prácticas, las soluciones son simples guías para poder solucionar la práctica, esto no significa que sean las únicas soluciones correctas. Cada práctica cuenta con su solución, la cual consiste en un cliente Java que envía los mensajes necesarios para solucionar la práctica.

La elección de Java como lenguaje de programación para estas soluciones se realiza para que la realización de estas prácticas no suponga un cambio radical para el alumno. Hasta este momento el lenguaje empleado por las prácticas de la asignatura es Java y, al tratarse estas prácticas parte de todo el conjunto de prácticas de la asignatura, de esta manera se continua haciendo uso del mismo lenguaje.

4.3.4. Documentación web

Una vez analizadas las partes que conforman el conjunto funcional del proyecto, queda por analizar las partes del proyecto que complementan a las partes anteriores. El primer componente de estos complementos es la página web destinada a los desarrolladores de videojuegos que vayan a hacer uso del motor Hephaestus para desarrollar sus videojuegos.

Documentación para el desarrollo de videojuegos

Esta parte del proyecto no consiste en documentar todo el código de Hephaestus, sino seleccionar las partes que son necesarias para el desarrollo de un videojuego y describir sus capacidades y el funcionamiento. No se pretende detallar el funcionamiento exacto a nivel técnico de todos y cada uno de los módulos que componen el motor.

El empleo de la biblioteca *docsify* permite la escritura de la documentación en markdown, además todo lo relacionado con la navegación propia de una página web también lo autogenera la propia biblioteca. Esto reduce en gran medida las decisiones a tomar en áreas como aspecto de la página web o reactividad

de los diferentes componentes que conforman la página web, decisiones naturales a la hora de enfrentar un desarrollo web. Todo lo relacionado con la parte web se genera mediante la biblioteca, dejando únicamente como incertidumbre la estructura que va a seguir la documentación.

4.3.5. Distribución

Finalmente, el otro componente complementario al resto del proyecto consiste en distribuir al alumno los archivos necesarios, que no forman parte de otras partes del proyecto, para poner en funcionamiento el sistema del juego y el motor, simulando un sistema distribuido y de manera sencilla. Con este objetivo se proporciona un archivo que permite realizar todas estas acciones mediante un único comando.

Archivo docker-compose

La tecnología que permite a Docker ser reproducible en cualquier entorno que tenga instalado su motor se denomina Docker Compose. Con el fin de que los alumnos tengan todos el mismo punto de inicio para realizar las prácticas, ya configurado y preparado para estas mismas, se tiene que crear un fichero docker-compose que reproduzca la misma secuencia de inicio de contenedores para todos los alumnos. Este fichero debe levantar y configurar dos contenedores del motor que actúen como máquinas remotas al juego, un contenedor del juego que permita recibir peticiones por el puerto correspondiente y un contenedor de la base de datos para almacenar el estado del juego de forma permanente.

Capítulo 5

Diseño

5.1. Hephaestus

Tras el análisis realizado en el capítulo 4 se pasa a la fase de diseño del sistema. En concreto, este apartado trata sobre el diseño del motor Hephaestus. Al tratarse de la parte del proyecto más complicada, el apartado sobre el diseño se centra principalmente en este.

En primer lugar se realiza el diagrama de clases del diseño completo del motor. Este diagrama, mostrado en la figura 5.1, completa el análisis realizado en el capítulo ya mencionado en el párrafo anterior.

Una vez el diagrama de clases está completo se necesita establecer el comportamiento del sistema. Para ello, se cuenta con varios tipos de diagrama que permiten modelar el comportamiento del sistema. En este caso se opta por realizar diagramas de secuencia, por ser estos sobre los que mayor conocimiento se tiene.

Los diagramas de secuencia modelan la secuencia, valga la redundancia, que ocurre a la hora de ejecutar un caso de uso. Debido a la cantidad de casos de uso, no corresponde colocar todos los diagramas dentro de este documento, todos los diagramas se pueden encontrar en el repositorio del proyecto, el cual se puede encontrar en los anexos de este documento.

El primer diagrama, representado en la figura 5.2, muestra el diseño para solucionar el caso de uso cambiar ubicación. Este caso de uso le corresponde a la pieza actor dentro del motor y es una acción asíncrona, tal y como se aprecia en el diagrama.

El proceso de comunicación entre el juego y las piezas que conforman el motor se realiza mediante la biblioteca estándar de Elixir "GenServer", esta biblioteca proporciona procesos observables, con un estado interno modificable y persistentes durante la ejecución del programa, esto significa que el colector

de basura no elimina estos procesos aunque estén en un estado de inactividad. Los "GenServers.están formados por una API pública, empleada por el juego para enviar órdenes al motor, y el proceso interno del "GenServer", el cual se encarga de realizar las operaciones necesarias y actualiza el estado interno del proceso.

En el diseño anterior el caso de uso era completamente asíncrono. En el caso de uso de la figura 5.3, el mensaje enviado al "GenServer.es síncrono, por lo tanto el juego no puede continuar por ese hilo de ejecución hasta que reciba el mensaje de respuesta del servidor.

Hay que destacar en este punto también, el comportamiento del "GenServer", el cual cuenta con las características de un sistema servidor en la arquitectura cliente-servidor. Recibe mensajes a través de la API pública que ya se ha mencionado, los guarda en una cola de mensajes y los va procesando por orden. La cola que emplea es una cola FIFO (First In First Out).

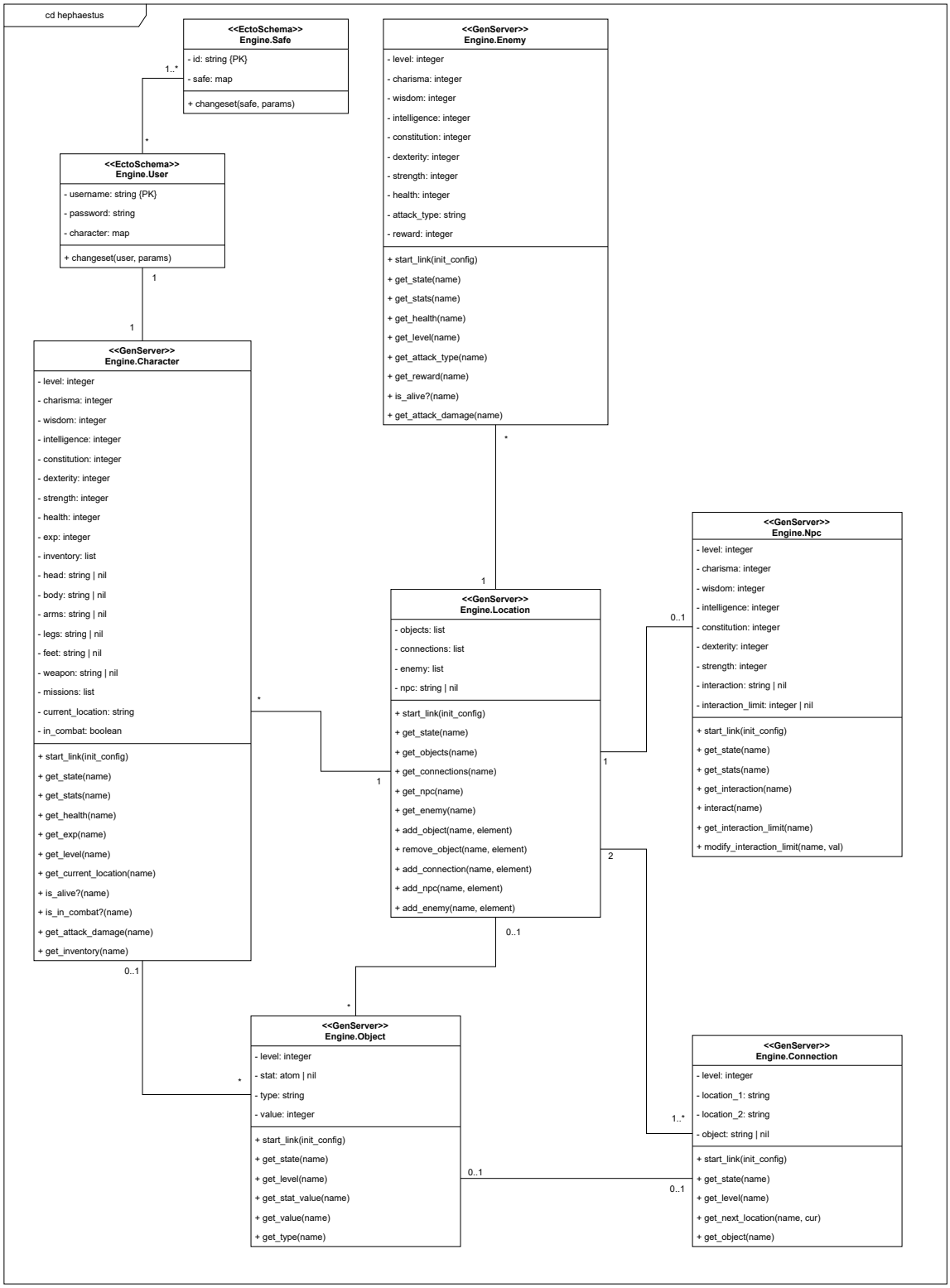


Figura 5.1: Diagrama de clases del motor Hephaestus

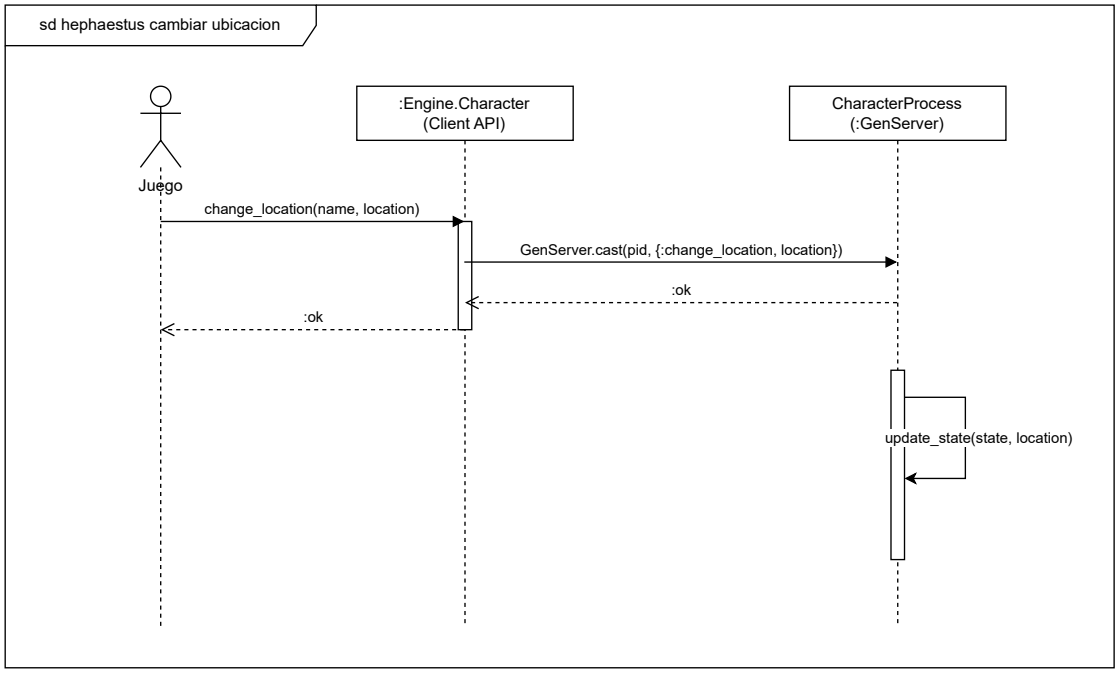


Figura 5.2: Diagrama de secuencia del caso de uso cambiar ubicación

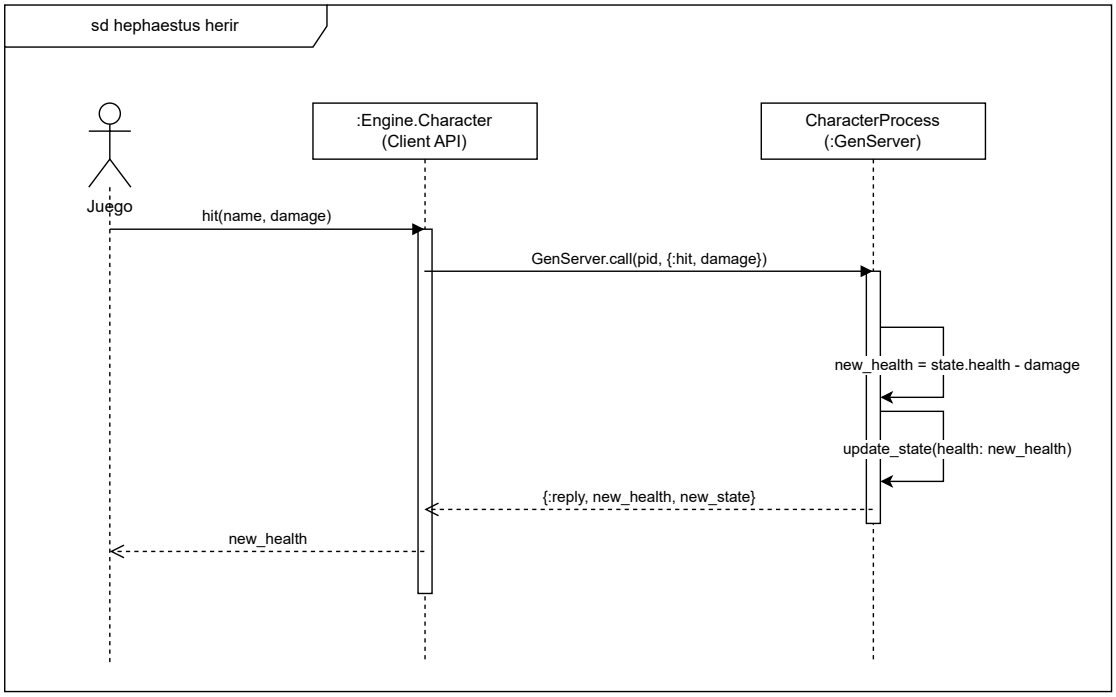


Figura 5.3: Diagrama de secuencia del caso de uso herir

5.2. Base de datos

En este proyecto la base de datos toma un rol sencillo, almacena la información sobre el estado del juego actualmente y sobre los jugadores que tienen acceso al juego.

El diseño de la misma, por lo tanto, refleja la sencillez de esta labor a la vez que muestra la flexibilidad de la que dispone el desarrollador de los videojuegos que emplean Hephaestus para adaptarla a sus necesidades.

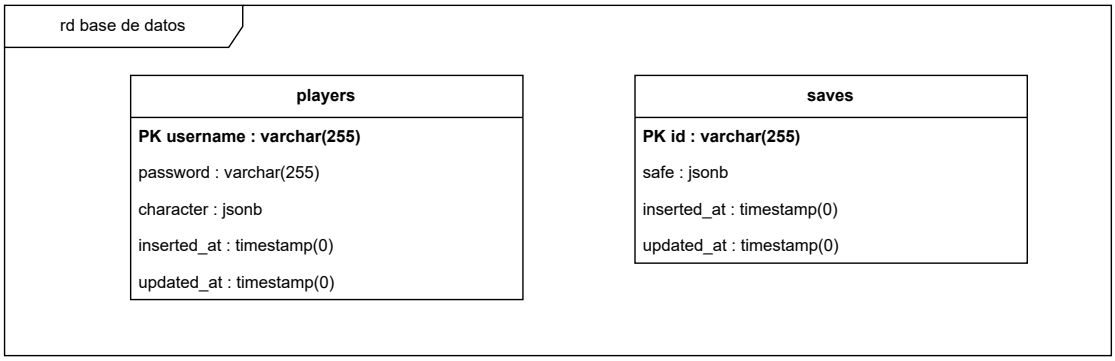


Figura 5.4: Diagrama entidad-relación de la base de datos

5.3. Juego didáctico

Una vez realizados el diseño del motor y de la base de datos, el siguiente diseño es el correspondiente al jeugo didáctico. Al igual que ha sucedido con el motor el diagrama mostrado en la figura 5.5 muestra el resultado final del diseño derivado del análisis realizado en el capítulo anterior.

Hasta este momento el juego didáctico se ha nombrado así, como juego didáctico, en esta fase el juego toma mayor forma y ya no es un juego didáctico genérico, toma forma y, por lo tanto, se le nombra como Alfa desde este punto en adelante.

El diseño de Alfa requiere de un servidor con el que el jugador se pueda comunicar. Alfa emplea una librería externa que proporciona un servidor ya preparado para un contexto de producción, de esta manera el diseño solo se tiene que centrar en las primitivas que se proporcionan al usuario.

Al igual que sucede con Hephaestus, Alfa requiere de varios diagramas de secuencia para definir su comportamiento completo. En este caso, todos los casos de uso corresponden a una primitiva a la que tiene acceso el jugador. Como en la sección anterior, todos los diagramas están disponibles en el repositorio, no siendo viable mostrarlos todos en esta documentación. La dirección al repositorio se encuentra en los anexos de este mismo documento.

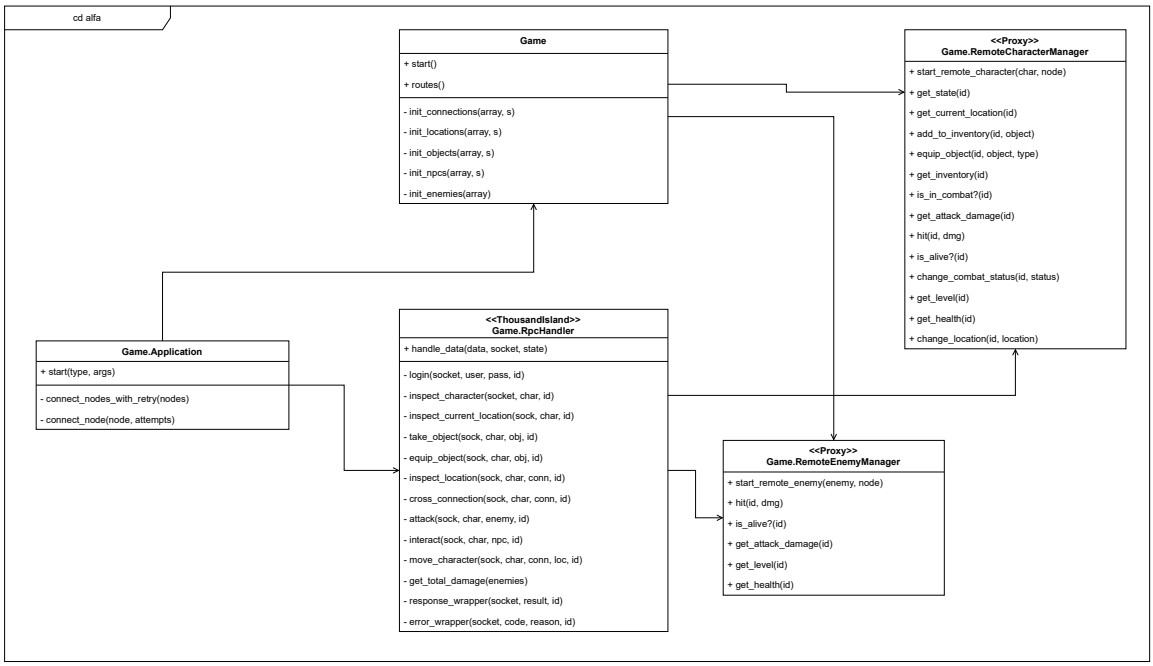


Figura 5.5: Diagrama de clases de Alfa

La figura 5.2 muestra el diagrama de secuencia mediante el cual se modela el caso de uso ïnspeccionar ubicación actual". Alfa se comporta como un servidor para el jugador, el cual tiene que realizar peticiones mediante un cliente que envíe una petición, empleando el protocolo JSON-RPC, al servidor. Este, una vez recibida la petición, evalúa la primitiva que pretende emplear el usuario y deriva el flujo del programa hacia la sección del código dedicada a resolver dicha primitiva. Durante la resolución de la primitiva, y no antes, el juego hace uso de las piezas proporcionadas por el motor Hephaestus para gestionar el estado en tiempo real de la partida, este estado es volátil y no se almacena en la base de datos hasta que no se le indica al motor. Todo este proceso se muestra en el diagrama mencionado anteriormente.

Uno de los puntos claves de cualquier videojuego es el inicio del propio juego, se tienen que cargar todos los elementos necesarios para jugar la partida previo al inicio de la misma. Existen múltiples estrategias donde elegir para realizar la carga de elementos, en el caso de Alfa se opta por cargar todos los actores relacionados con la partida pero no con el jugador, estos son los enemigos, ubicaciones, conexiones, objetos y NPCs, de esta manera el único actor que falta es el que dirige el rumbo del juego, el personaje. Como la partida puede estar empezado y encontrarse en un punto intermedio, la carga del estado de los actores se realiza desde la base de datos, la cual almacena el estado en el último punto de guardado. En la figura 5.7 se muestra como es el diseño del proceso descrito arriba.

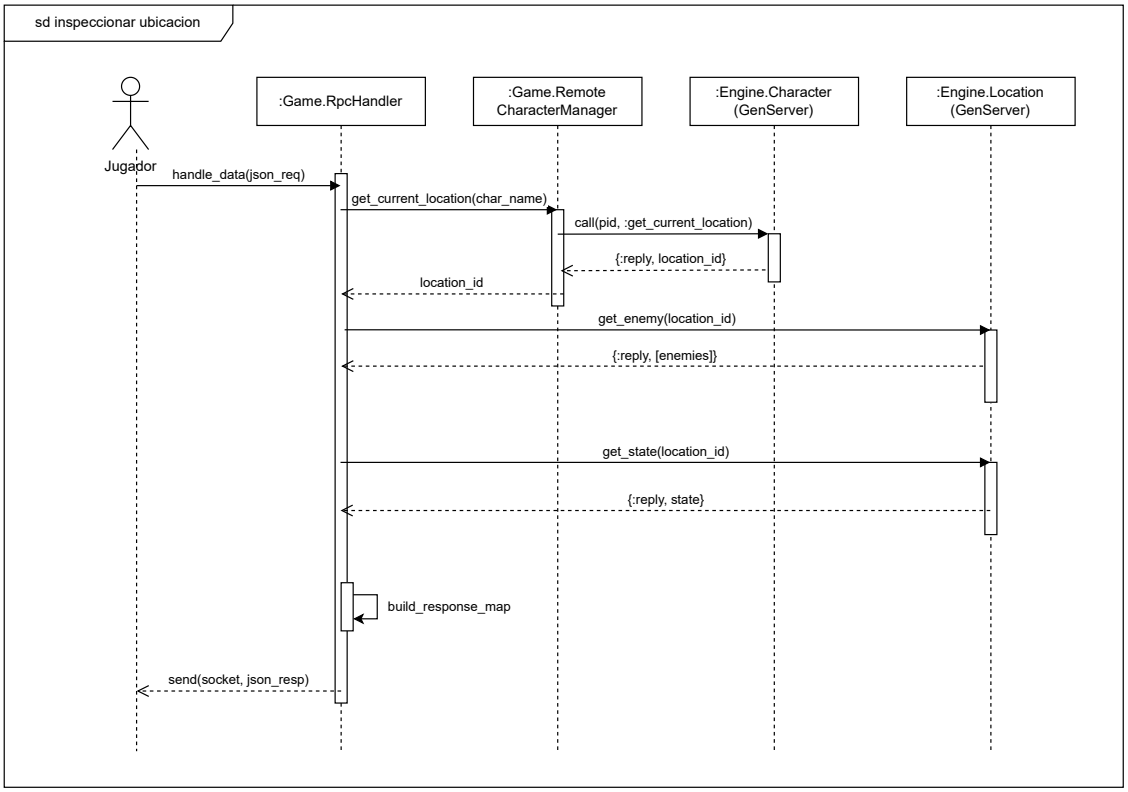


Figura 5.6: Diagrama de secuencia del caso de uso inspeccionar ubicación actual

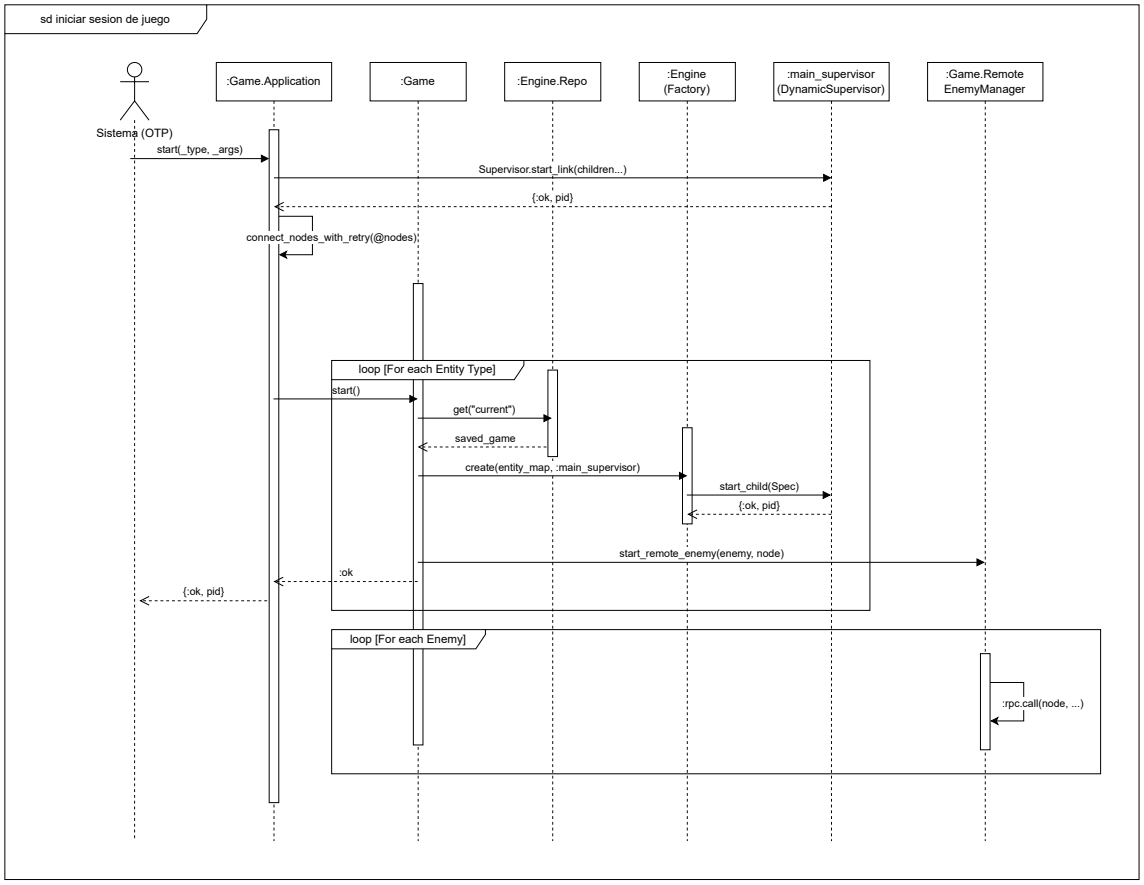


Figura 5.7: Diagrama de secuencia del caso de uso iniciar sesión de juego

5.4. Prácticas

Al contrario que las secciones anteriores, la prácticas para los alumnos no cuentan con un análisis debido a la naturaleza de estas. El objetivo de las prácticas es crear documentos que ayuden al alumno a asimilar el conocimiento adquirido en las clases de teoría. Por lo tanto, el entregable principal de las prácticas para los alumnos son los documentos descriptivos con las instrucciones que los alumnos tienen que seguir para completar las prácticas, incluyendo un reto a completar por el alumno con todo lo aprendido durante la práctica.

Se decide hacer tres prácticas las cuales están relacionadas entre sí, continúan a raíz de la práctica anterior, e incrementan su dificultad siendo la primera la más fácil y la última la más difícil. A estas tres prácticas se añade un cuarto documento que actúa como introducción a las prácticas.

A estas tres prácticas se les añade sus soluciones y otros programas escritos en Java que complementan las prácticas escritas. Todo este código Java está pensado para su uso por el profesor de la asignatura ya sea para tener una solución ya construida para entregar a los alumnos una vez terminado el periodo de la práctica, o para mostrar el funcionamiento de alguna de las partes del enunciado mediante el resto de programas que no son soluciones.

Por lo tanto, el diseño de las prácticas se trata de una labor de diseño didáctico sin código lo suficiente complejo para justificar un análisis y diseño exhaustivo. A continuación se describe el diseño didáctico de cada una de las prácticas y como se relacionan entre sí.

5.4.1. Introducción a las prácticas

Como ya se ha mencionado se crea un documento diseñado para introducir las tres prácticas al alumno. Para ello el documento cuenta con varias secciones. En estas secciones el alumno cuenta con las instrucciones para instalar todas las tecnologías necesarias para desarrollar las prácticas, las instrucciones necesarias para poner en funcionamiento el juego y un resumen de los objetivos de las prácticas a las que el alumno va a enfrentarse.

Con este documento el alumno es capaz de encontrar soluciones para los problemas más comunes que se pueda encontrar durante el transcurso de las prácticas, como, por ejemplo, levantar los contenedores del proyecto, modificar la base de datos o reiniciar el sistema si surgen algún problema.

5.4.2. Primera práctica

Durante la primera práctica se presenta la base del funcionamiento del sistema. Debido al nivel de los alumnos para los que están pensadas las prácticas, la primera sección de la práctica introduce la

arquitectura para sistemas distribuidos cliente-servidor, de esta manera el alumno se introduce en la estructura que van a seguir los clientes que va a programar para resolver las prácticas.

Tras la introducción a la arquitectura distribuida cliente-servidor, la otra pieza fundamental para que el alumno pueda resolver las prácticas es el protocolo JSON-RPC. Por este motivo la siguiente sección de la práctica consiste en una introducción a este protocolo. En esta introducción se añaden ejemplos del formato de las peticiones que admite el protocolo al igual que el formato que sigue el protocolo para las respuesta que se dan en el mismo.

Una vez se hacen las introducciones a los dos conceptos anteriores, la práctica finaliza con el ejercicio a realizar por el alumno. Pero, antes de presentar el enunciado, es necesario realizar un ejemplo de un cliente Java para que el alumno tenga un apoyo visual de lo que se le pide, ilustrado en 5.1. A partir de este cliente, la premisa de esta primera práctica es emular el cliente puesto como ejemplo y, realizar una serie de acciones dentro del juego mediante un solo cliente de Java o varios de ellos.

Con el fin de facilitar al alumno la resolución de la práctica, tanto en esta como en las siguientes prácticas, se proporciona un listado con todas las primitivas necesarias para la resolución de la misma. A su vez, como ya se vio en el capitulo de análisis (4) y en la sección anterior sobre el diseño del juego didáctico, este cuenta con una primitiva la cual devuelve todas las primitivas disponibles en el juego y como llamarlas.

```
public class JavaClient {
  public static void main(String[] args) throws IOException {
    Gson jsonParser = new Gson();
    Socket socket = new Socket("localhost", 3000);

    // Abrir el canal de comunicacion con el servidor
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(new InputStreamReader(socket.
        getInputStream()));

    // Preparar los datos
    HashMap<String, Object> mapa = new HashMap<String, Object>();
    mapa.put("jsonrpc", "2.0");
    mapa.put("method", "info");
    mapa.put("id", "1");

    // Enviar datos
    out.println(jsonParser.toJson(mapa));

    // Obtener respuesta
    HashMap<String, Object> respuesta = jsonParser.fromJson(in.readLine(), new
        TypeToken<HashMap<String, Object>>(){}.getType());

    printServerResult(respuesta); // Imprimir respuesta
    socket.close(); // Cerrar el canal
```

```
  }

  private static void printServerResult(HashMap<String, Object> mapa) {
    Object resultado = mapa.get("result");
    Object error = mapa.get("error");
    Object id = mapa.get("id");

    if (resultado == null) {
      if (error == null) {
        System.out.println("Algo salio mal en el servidor \n");
      } else {
        System.out.println("Error al realizar la accion del mensaje con id " + id
            + ": " + error + "\n");
      }
    } else {
      System.out.println("Resultado del mensaje con id " + id + ": " + resultado +
          "\n");
    }
  }
}
```

Código 5.1: Snippet cliente Java entregado a los alumnos

5.4.3. Segunda práctica

En esta segunda práctica se profundiza en uno de los conceptos vistos en teoría, los actores. Además se profundiza en una de las partes de los actores como son las colas de mensajes. Con este objetivo, se comienza la práctica recordando lo visto en teoría sobre los actores, esto ubica a los alumnos en el nuevo contexto y les ayuda a encontrar la información necesaria si lo requieren en la teoría.

Tras la introducción a los actores, la práctica explora en profundidad el concepto de las colas de mensaje, un concepto informático empleado en muchos tipos de sistemas y que, en este caso, forman parte de los actores. Tras estas introducciones se le muestra al alumno cómo se emplean estos conceptos en el desarrollo y contexto del sistema empleado para las prácticas, el motor Hephaestus y el juego didáctico.

Con el contexto de estos conceptos, se prepara la práctica para los alumnos. Al ser la segunda práctica la dificultad de la misma tiene que aumentar y, como ya se ha mencionado, continúa con el hilo de la práctica anterior, jugar al juego didáctico. Para ello se diseña la práctica empleando programación asíncrona. El concepto de la programación asíncrona es un concepto nuevo para el nivel de los alumnos a los que va dirigido el proyecto, por lo tanto, y teniendo en cuenta que ya se ha mostrado en la práctica anterior como se construye un cliente para jugar al juego, se le da al alumno el esqueleto base para poder realizar la práctica mediante programación asíncrona, tal y como se muestra en el código 5.2.

Con todo lo entregado en esta práctica y lo mostrado en la práctica anterior el alumno debe ser capaz

de resolver la premisa de la práctica. Los conocimientos que el alumno adquiere de esta práctica son el funcionamiento y uso de los actores y las colas de mensajes, dentro de estos, en un sistema distribuido y la programación asíncrona.

```
private static void interactWithNpc(Gson jsonParser) throws IOException,
    InterruptedException, ExecutionException {
  // Parametros comunes a todas las llamadas
  HashMap<String, String> params = new HashMap<String, String>();
  params.put("npc", "sabio");
  params.put("character_name", "tim");

  // Ejecutor de tareas asincronas
  ExecutorService executor = Executors.newFixedThreadPool(10);
  // Lista de tareas asincronas
  List<Future<HashMap<String, Object>>> futures = new ArrayList<>();

  // Bucle de creacion y lanzamiento de tareas asincronas
  for (int i = 1; i < 7; i++) {
        HashMap<String, Object> mapa = new HashMap<String, Object>();
        mapa.put("jsonrpc", "2.0");
        mapa.put("method", "interact");
        mapa.put("params", params);
        mapa.put("id", String.valueOf(i));

        Callable<HashMap<String, Object>> task = () -> {
          // Implementacion de un cliente
          return respuesta;
        };

        futures.add(executor.submit(task));
  }

  // Muestra de resultados por pantalla
  for (Future<HashMap<String, Object>> future : futures) {
    printServerResult(future.get());
  }

  executor.shutdown();
}
```

Código 5.2: Snippet codigo asíncrono Java entregado a los alumnos

5.4.4. Tercera práctica

En la tercera y última práctica se diseña una práctica basada no tanto en conceptos vistos en teoría, sino en el desarrollo de sistemas distribuidos y cómo funciona, en concreto, el sistema empleado en las

prácticas.

Con este objetivo la práctica comienza con una introducción a la programación en Elixir, debido al cambio que supone este lenguaje frente a Java. Se explican los fundamentos necesarios para realizar la práctica, esto incluye el funcionamiento del "pattern matching.º las construcciones condicionales, entre otros conceptos.

Una vez se muestra el funcionamiento de Elixir, la práctica continúa con la premisa a resolver por el alumno, en esta práctica dicha premisa consiste en emplear ambos lenguajes para resolver lo planteado. En primer lugar se emplea Elixir para modificar el motor del juego Hephaestus y, en segundo lugar, se emplea Java para completar el juego, dando por finalizada la práctica.

Mediante este diseño de la práctica se incrementa la dificultad de la misma, mientras que se muestra el funcionamiento de un sistema distribuido y se continúa con el hilo de todas estas prácticas que es la finalización del juego didáctico.

Capítulo 6

Pruebas

La gestión de la calidad del software se asegura que el desarrollo de los sistemas software desarrollados se encuentran en un estado correcto para el propósito para el que fueron diseñados. Esto implica que el sistema debe asegurar las necesidades de los clientes que tiene y ser eficiente y fiable durante su periodo de funcionamiento [4].

Existen diferentes tipos de pruebas para determinar la calidad del software, las cuales se enfocan en diferentes propiedades de un sistema. De esta manera se puede asegurar que el sistema no contenga errores que impidan su utilización. Además, existen técnicas que permiten asegurar que cada parte del sistema funcione correctamente sin producir resultados inesperados. La división de estas partes puede ser a nivel más general o más concreta, siendo por ejemplo una parte general un componente del sistema y una parte más concreta una función dentro de un componente del sistema.

Durante el proyecto se emplean tres tipos diferentes de pruebas para los distintos entregables. Debido al tiempo disponible, el rango de actuación y la cantidad de tipos de pruebas realizadas a cada una de las partes se ve disminuido. Por ello se prioriza el correcto funcionamiento del sistema y una experiencia como mínimo adecuada para el alumno, se deja de lado pruebas más profundas y se centra el mayor esfuerzo en el motor, como pieza central del proyecto.

La cantidad de pruebas y profundidad de las mismas se ve afectado por el tiempo disponible. Por ello y debido a que la posibilidad de recuperar el sistema y dejarlo en un estado anterior es muy sencillo, gracias al empleo de Docker, se hicieron recortes en este punto del proyecto, siendo las pruebas realizadas suficientes para asegurar el correcto funcionamiento del sistema, pero dejando de lado la integridad del mismo.

6.1. Pruebas unitarias

Las pruebas unitarias en el desarrollo de software sirven para verificar el funcionamiento de las partes más pequeñas del sistema que está siendo probado. Se aseguran de que las partes verificadas se comporten como se espera y no surjan ningún error.

Por lo general las pruebas unitarias requieren de un tiempo mayor a otro tipo de pruebas para su implementación ya que, al centrarse en las partes más pequeñas del sistema evaluado, requieren de múltiples pruebas por unidad del sistema de manera que se abarquen todos los escenarios posibles.

Debido a su naturaleza, las pruebas unitarias son muy útiles para probar sistemas o aplicaciones software que normalmente no cuentan con interacción directa del usuario, sino que son empleados por otros sistemas, por ejemplo las bibliotecas de algún lenguaje. Por ello, y debido al recorte por tiempo ya mencionado, estas pruebas solo se emplean para asegurar el correcto funcionamiento del motor Hephaestus.

Al tratarse de Hephaestus el lenguaje para las pruebas del motor es Elixir, por lo tanto se hace uso de las herramientas proporcionadas por el propio lenguaje para hacer las pruebas, tal y como se muestra en el ejemplo 6.1. Mediante estas pruebas se asegura el correcto funcionamiento de las piezas del motor necesarias para crear un videojuego, incluyendo todo lo implementado mediante *GenServer*, para lo cual las pruebas se centran en el código que produce algún cambio en el estado, esto es se excluye de las pruebas todo código que no modifique el comportamiento de la biblioteca *GenServer* como puede ser un *getter* de un campo del estado.

```
defmodule ConnectionTest do
  use ExUnit.Case, async: true

  setup do
    connection =
    start_supervised!(
      {Engine.Connection,
        %Engine.GameEntity{
              name: "test_connection",
              state: %Engine.Connection{
                    level: 1,
                    location_1: "location_test",
                    location_2: "second_location_test",
                    object: nil
              }
      }}
    )

    %{test_connection: connection}
  end
```

```
  test "get location from location_1",
       %{test_connection: test_connection} do
    assert Engine.Connection.get_next_location(test_connection,
           "location_test") === "second_location_test"
  end
end
```

Código 6.1: Snippet test unitario Elixir

6.2. Pruebas end-to-end

Las pruebas end-to-end simulan el flujo completo del sistema desde la perspectiva del usuario final. De esta manera se prueba la experiencia de usuario obtenida al interactuar con el sistema. Por lo tanto estas pruebas son las más adecuadas para probar el juego didáctico, puesto que el alumno, el cual actúa como usuario final en este proyecto, interactúa con dicha parte del proyecto.

Para la realización de las pruebas end-to-end se crean una serie de escenarios los cuales el juego debe cumplir para poder ser considerado exitoso. Dichos escenarios derivan, más adelante en el desarrollo del proyecto, en las soluciones a las prácticas presentadas a los alumnos, puesto que el juego está diseñado específicamente para cumplir con ese propósito.

El juego didáctico actúa como hilo conductor de las prácticas y, por lo tanto, es natural que se enlace las pruebas end-to-end del mismo con las prácticas proporcionadas a los alumnos, en este caso como solución de las mismas.

Capítulo 7

Conclusión

Una vez finalizado el proyecto, se analizan los problemas encontrados durante el desarrollo del mismo y el desarrollo futuro del mismo tras el estado en el que termina con este documento. Se evalúan de nuevo los riesgos planteados en la planificación, enfocándose en aquellos riesgos que han sucedido y las medidas tomadas para su resolución.

7.1. Evaluación de los riesgos

Los riesgos se muestran en el documento mediante tablas en el Capítulo 2 y, una vez finalizado el proyecto, se pueden evaluar los riesgos que han sucedido durante el desarrollo del mismo. El primer riesgo planificado se describe en la Tabla 2.2 y ocurre durante el desarrollo, en este riesgo se plantea la posibilidad de que surjan imprevistos que retrasen el desarrollo del proyecto. La mayor parte de los imprevistos surgidos durante el desarrollo afectaron a las reuniones programadas, no pudiéndose hacer en el momento planeado, siendo solucionado mediante el adelantamiento o el retraso de dicha reunión. Pese a esta mitigación del riesgo, estos no fueron suficientes para no afectar a la planificación inicial.

El siguiente riesgo surgido durante el desarrollo se muestra en la Tabla 2.8. Este riesgo abarca la novedad de algunas de las tecnologías empleadas como Elixir. Debido a la novedad, el mayor efecto surge a la hora de identificar los errores que surgen en la ejecución y la frecuencia de estos, debido al escaso conocimiento en el lenguaje. Al final, aunque el riesgo surge en contadas ocasiones, no provoca un retraso perceptible al total del tiempo de desarrollo.

Finalmente el mayor causante de retrasos en el proyecto es el riesgo descrito en la Tabla 2.6, el cual tiene en cuenta los errores de estimación que pudieran darse en la planificación del proyecto. Ha sido con diferencia el mayor causante de retrasos en el proyecto debido a la gran inexperiencia del alumno, lo que da lugar a malas estimaciones de tiempo, no teniendo en cuenta los tiempos disponibles en la realidad y siendo demasiado optimista, en general, a la hora de asignar prototipos a las diferentes partes del proyecto.

Todo esto en conjunto provoca el mayor retraso de todo el proyecto, pudiendo haber sido peor de no haberse aplicado el plan de contingencia, y, aún con eso, se produce un retraso de varias semanas sobre las fecha estimada de finalización del proyecto.

7.2. Problemas en la implementación del código

Una vez se concluye el análisis y el diseño del proyecto se procede a la implementación del código de acuerdo a lo diseñado. Durante este proceso, como en todo proceso de implementación, se encuentran varios problemas los cuales se tienen que solucionar.

El proyecto cuenta con varias partes de las cuales dos de ellas están escritas en Elixir mientras que la última está escrita en Java. Ya se ha mencionado en capítulos anteriores que el código escrito en Java forma parte de las soluciones para las prácticas, por lo tanto es código sencillo que no requiere de análisis y diseños profundos y que, además, no supone un problema su implementación.

Por lo tanto, los focos de problemas durante la implementación del código se concentran en Elixir y Docker. Elixir supone un reto debido a la novedad del lenguaje para el alumno, mientras que el reto que supone Docker deriva de Elixir y la construcción de las imágenes para la ejecución de programas en este lenguaje.

7.2.1. Elixir

Al ser Elixir un lenguaje de programación que sigue el paradigma de programación funcional y no cuenta con herramientas para la programación orientada a objetos. Estas dos características del diseño del lenguaje desarrollan los dos primeros problemas a la hora de implementar el código. El cambio de paradigma del imperativo al funcional supone el mayor coste temporal al proyecto, debido a que las diferencias son demasiado pronunciadas. Respecto a la programación orientada a objetos, se adapta el código empleando los elementos que ofrece el lenguaje, esto no supone tanto costo temporal al proyecto.

El siguiente elemento del lenguaje que se presenta problemático son los *GenServer*. Estos elementos presentan unas características que los hacen buenos candidatos a representar actores distribuidos dentro del sistema. Tras algunas vueltas se acaba por decidir por ellos para hacer esta labor, además de ser la solución a la adaptación del problema de la programación orientada a objetos. Debido a las características de los *GenServer* pueden actuar como una especie de objetos tradicionales, de esta manera se consigue una simulación parcial, no pudiendo conseguir la composición y la herencia características de este paradigma. Por lo tanto no se puede evitar cierta repetición de código.

Una vez se superan estos problemas ya descritos, el siguiente problema que surge es la transformación de estructuras de Elixir a JSON. Debido a que el estado del juego se almacena como JSON en la base

de datos, es fundamental poder transformar estructuras de Elixir a JSON. Elixir cuenta con mapas como estructura para el almacenamiento de datos en tiempo de ejecución, estos mapas tienen transformación directa a JSON al emplear la biblioteca con la que se hace uso de la base de datos desde el motor. Por lo tanto es un problema con fácil solución y que no supone ningún retraso.

Finalmente, el último problema que se presenta por usar Elixir son los monitores. Los monitores son un elemento de Elixir que sirven para vigilar la ejecución de procesos hijos, en el caso de Hephaestus se emplean para vigilar la ejecución de los *GenServer*. Los monitores no solo vigilan los procesos sino que actúan sobre ellos en caso de que se produzca un error, pudiendo reiniciarlos o eliminarlos dependiendo de las instrucciones que tengan. El problema surge de los diferentes tipos de monitores que existen y de como se relacionan entre sí. Se evalúan todas las opciones y se opta por emplear monitores dinámicos por la sencillez que supone su uso. Este tipo de monitores es adecuado para juegos pequeños como lo es el juego didáctico, pero para su uso en juegos más grandes el desarrollador debe evaluar si merece la pena emplear monitores estáticos, los cuales permiten más opciones.

7.2.2. Docker

Los problemas surgidos por el empleo de Docker son derivados de Elixir. La distribución de Elixir difiere en gran medida del proceso de desarrollo, por lo tanto la construcción de las imágenes es más complicado de lo que sería en caso de ser un entorno de desarrollo. Otro factor determinante es la falta de documentación, Elixir es un lenguaje relativamente pequeño en lo que respecta a comunidad y, por lo tanto, hay ciertas tareas que no son tan populares y que no cuentan con la suficiente documentación para su ejecución.

En este caso la solución pasa por adaptar los pocos ejemplos existentes, sin relación con la tarea específica, y emplear herramientas de inteligencia artificial para generar el código restante. Finalmente, se hace uso de estos métodos para solucionar el problema, costando bastante tiempo de proyecto.

7.3. Desarrollo a futuro

Una vez se finaliza el desarrollo del proyecto descrito en este documento dicho proyecto puede seguir siendo desarrollado. Para ello se presentan tareas o partes enteras del proyecto que no se han podido llegar a realizar. Mediante este listado, cualquier futuro desarrollador cuenta con una imagen global de lo que falta, así como de ideas lanzadas que puedan ser interesantes.

7.3.1. Mejora en el proceso de pruebas

Como ya se ha mencionado en el Capítulo 6, debido a la falta de tiempo, las pruebas realizadas sobre el motor Hephaestus y el juego didáctico solamente aseguran el correcto funcionamiento de estos.

Con el fin de mejorar la calidad del código se plantea la introducción de un mayor número de pruebas unitarias que mejoren la integridad de Hephaestus, así como la integración de este tipo de pruebas en el juego didáctico.

7.3.2. Ampliación del motor

Desde el principio se ha declarado que el motor debe ser ampliable y tener la capacidad de añadir más módulos para crear juegos más complejos o incluir una mayor cantidad de utilidades para los desarrolladores.

En el Capítulo 4 se muestra una serie de elementos que fueron descartados por la falta de tiempo. Estos elementos se pueden ver en la figura 4.4 y son un buen punto de inicio para una futura expansión del motor Hephaestus.

7.3.3. Separación en motores más pequeños

Aunque el motor se ha diseñado para ser distribuido y actuar como base de ejecución para diferentes piezas en diferentes máquinas, existe la posibilidad de especializar aún más esta labor.

Se propone la división del propio motor en motores más pequeños con los módulos del motor original necesarios para ejecutar una pieza o piezas específicas en cada máquina.

Este aumento de la complejidad de la arquitectura y la mantenibilidad del código solo es justificable si el tamaño del motor y los recursos necesarios para hacerlo funcionar aumentan, de tal manera que su ejecución en cada máquina suponga un coste operacional tan alto que desincentive su uso en sistemas distribuidos.

7.3.4. Generalizar y aumentar la especialidad del motor

Al final del proyecto, el motor Hephaestus es un motor diseñado para crear videojuegos distribuidos de tipo RPG basados en Dungeons and Dragons. Se trata de una especialización muy concreta que reduce el mercado en el que se puede emplear dicho motor.

Una futura modificación del motor puede generalizar tanto el género de juegos RPG al que afecta, no solo a los que están basados en Dungeons and Dragons, cómo el tipo de juego que se puede construir con dicho motor. De esta manera se avanza hacia un motor más generalista como pueden ser Godot, Unity o Unreal Engine.

7.3.5. Entorno de desarrollo gráfico

Los motores de videojuegos modernos cuentan con entornos de desarrollo gráficos los cuales aportan mejoras a la experiencia de desarrollo de los desarrolladores de videojuegos. Entre estas mejores se pueden mencionar: Desarrollo gráfico mediante cuadros de comandos, bibliotecas de *assets*, editor integrado de código, *debugger* integrado...

Como proyecto se puede plantear crear un entorno con alguna de estas características para el motor Hephaestus. Para el cual no serían necesarias todas las que ofrecen motores comerciales como Unity o Unreal, ya que estos soportan gráficos en 2D y 3D entre otras muchas características, con el fin de poder ser empleados para la mayor variedad de tipos de videojuegos.

7.4. Tiempo de desarrollo

En el Capítulo 2 se desarrolla el proceso y resultado de la planificación del proyecto. Durante este proceso se menciona que se realizan una serie de reuniones durante el desarrollo que no pueden ser representadas en el diagrama de Gantt al igual que los periodos de vacaciones impuestos por la universidad. Con el fin de expandir la gestión temporal acometida durante el proceso de desarrollo del proyecto la Tabla 7.1 muestra las reuniones planeadas teniendo en cuenta además los periodos de pausa. De esta forma, además, se explica porque, en el diagrama de Gantt, algunos entregables se alargan más en el tiempo, puesto que coinciden con periodos vacacionales.

Evento	**Fecha Inicio**	**Fecha Fin**
Reunión inicial sobre el proyecto	13/02/2025	
Investigación inicial	13/02/2025	27/02/2025
Reunión	27/02/2025	
Hephaestus, Módulo personaje	27/02/2025	03/04/2025
Reunión	06/03/2025	
Reunión	13/03/2025	

Reunión	20/03/2025	
Reunión	27/03/2025	
Reunión	03/04/2025	
Hephaestus, Módulo enemigo	03/04/2025	24/04/2025
Reunión	10/04/2025	
Reunión	24/04/2025	
Hephaestus, Módulo NPC	24/04/2025	08/05/2025
Reunión	30/04/2025	
Reunión	08/05/2025	
Hephaestus, Módulo objeto	08/05/2025	22/05/2025
Reunión	15/05/2025	
Reunión	22/05/2025	
Hephaestus, Módulo ubicación	22/05/2025	05/06/2025
Reunión	29/05/2025	
Reunión	05/06/2025	
Hephaestus, Módulo conexión	05/06/2025	19/06/2025
Reunión	12/06/2025	
Reunión	19/06/2025	
Hephaestus, Persistencia	19/06/2025	10/07/2025
Reunión	26/06/2025	
Reunión	03/06/2025	
Reunión	10/07/2025	
Hephaestus, Utilidades	10/07/2025	24/07/2025
Reunión	17/06/2025	
Reunión	24/07/2025	
Juego didáctico, Servidor	24/07/2025	11/09/2025
Reunión	31/07/2025	

Reunión	11/09/2025	
Parón de verano	01/08/2025	09/09/2025
Juego didáctico, Handlers	11/09/2025	09/10/2025
Reunión	18/09/2025	
Reunión	25/09/2025	
Reunión	02/10/2025	
Reunión	09/10/2025	
Juego didáctico, Gestores elementos remotos	09/10/2025	30/10/2025
Reunión	16/10/2025	
Reunión	23/10/2025	
Reunión	30/10/2025	
Prácticas, Práctica introductoria	30/10/2025	13/11/2025
Reunión	06/11/2025	
Reunión	13/11/2025	
Prácticas, Práctica 1	13/11/2025	20/11/2025
Reunión	20/11/2025	
Prácticas, Práctica 2	20/11/2025	27/11/2025
Reunión	27/11/2025	
Prácticas, Práctica 3	27/11/2025	04/12/2025
Reunión	04/12/2025	
Prácticas, Soluciones	04/12/2025	18/12/2025
Reunión	11/12/2025	
Reunión	18/12/2025	
Parón de Navidad	22/12/2025	08/01/2026
Prácticas, Soluciones	18/12/2025	08/01/2026
Reunión	08/01/2026	
Documentación web, Documentación	08/01/2026	22/01/2026

Reunión	15/01/2026	
Reunión	22/01/2026	
Hephaestus, Imagen Hephaestus	22/01/2026	05/02/2026
Reunión	29/01/2026	
Reunión	05/02/2026	
Juego didáctico, Imagen juego didáctico	05/02/2026	12/02/2026
Reunión	12/02/2026	
Juego didáctico, Imagen base de datos	12/02/2026	19/02/2026
Reunión	19/02/2026	
Distribución, Archivo docker-compose	19/02/2026	26/02/2026
Reunión	26/02/2026	

Tabla 7.1: Planificación en el tiempo del proyecto

Apéndice A

Repositorio de código

A.1. Ubicación del repositorio

El código de este proyecto se encuentra disponible públicamente en el repositorio GitHub del alumno en el siguiente enlace:

```
https://github.com/ThAlsay/Hephaestus
```

A.2. Organización del repositorio

El código del repositorio se organiza en las siguientes carpetas:

- Database: Código SQL para la creación del esquema de la base de datos y la inserción de los datos necesarios para jugar a Alfa.
- Docs: Página web para el desarrollo de juegos haciendo uso de Hephaestus.
- Document: Este documento.
- Manuals: Documentos de las prácticas.
- Stages: Código de ejemplo y soluciones para las prácticas. Todo ello escrito en Java. Contiene también la biblioteca necesaria para que se pueda ejecutar.
- Tfg_umbrella: Contiene todo el código Elixir. Esto incluye al motor Hephaestus y al juego didáctico Alfa.

Apéndice B

Prácticas para los alumnos

B.1. Práctica 0

Práctica 0

Documento:	Módulo-0: Presentación de las prácticas
Curso:	SISTEMAS DISTRIBUIDOS - Grado en Ingeniería Informática
Libro:	Enunciado 0: Presentación de las prácticas
Autor:	SAYALERO BLAZQUEZ, ALONSO
Fecha:	Viernes, 28 de Noviembre de 2025
Versión:	0.2

Tabla de contenidos

- Práctica 0
  - Tabla de contenidos
  - Objetivos
  - El escenario
  - La presentación de las páginas
    - Práctica 1
    - Práctica 2
    - Práctica 3
  - Elementos necesarios
    - Docker
      - Windows / MacOs / Linux (Docker Desktop)
      - Linux (Docker Engine) / WSL
    - Java
    - Elixir

Objetivos

En este documento encontramos los objetivos que vamos a intentar cumplir a lo largo de las siguientes tres prácticas.
También contiente una guía para instalar todo lo que vamos a necesitar para poder realizar las prácticas.

El escenario

Con la excusa de aprender sobre sistemas distribuidos vamos a jugar a un pequeño juego de rol por la línea de comandos. En este juego somos Tim, un aventurero
que se adentra en una cueva para intentar encontrar algún objeto de valor.

En este contexto vamos a utilizar Java y Elixir para jugar, ya veremos cómo.

La presentación de las páginas

Práctica 1

En esta primera práctica tenemos como objetivo aprender sobre la arquitectura de sistemas distribuidos cliente-servidor y el protocolo JSON-RPC. Para ello vamos a
hacer uso de Java para programar un cliente que se comunique con nuestro juego.

Práctica 2

En la segunda práctica avanzaremos en elementos de los sistemas distribuidos, como son los actores y las colas de mensajes. En esta práctica seguiremos usando
Java para, en este caso, múltiples clientes que se comunican de manera simultánea con el juego.

Práctica 3

En la última vamos a profundizar en los conceptos de la práctica anterior modificando el propio juego, esta vez empleando Elixir. Veremos el empleo de los actores,
programación en Elixir y finalizaremos el juego con otro cliente de Java.

Elementos necesarios

Docker

Windows / MacOs / Linux (Docker Desktop)

En Windows y MacOs docker solo está disponible mediante Docker Desktop. Para instalarlo sigue las instrucciones oficiales: https://docs.docker.com/get-started/get-docker/

Existe también una versión para Linux, aunque si trabajas desde WSL hay que seguir las instrucciones del siguiente apartado.

Linux (Docker Engine) / WSL

Este es el único método mediante el cuál podemos tener docker en WSL. Docker Engine instala únicamente los comandos por terminal sin aplicación de escritorio y solo
está disponible para Linux.

Al igual que con Docker Desktop, Docker proporciona instrucciones oficiales y detalladas sobre como instalarlo: https://docs.docker.com/engine/install/

Java

Para programar en Java vamos a necesitar el JDK de Oracle, el cuál se puede descargar desde este enlace: https://www.oracle.com/es/java/technologies/downloads/#jdk25-linux

Si nos encontramos en Linux/WSL el enlace anterior contiene paquetes para Debian y derivados y RedHat y derivados (.deb y .rpm), pero el JDK se puede encontrar
normalmente en los repositorios de las distintas distribuciones. Esta suele ser una forma más sencilla de instalarlo.

Elixir

La página web oficial de Elixir proporciona guías especificas para cada sistema operativo. https://elixir-lang.org/install.html

B.2. Práctica 1

Práctica 1

Documento:	Módulo-1: Arquitectura cliente-servidor
Curso:	SISTEMAS DISTRIBUIDOS - Grado en Ingeniería Informática
Libro:	Enunciado 1: Introducción a los sistemas distribuidos. Arquitectura de distribución cliente-servidor
Autor:	SAYALERO BLAZQUEZ, ALONSO
Fecha:	Viernes, 21 de Noviembre de 2025
Versión:	1.0

Tabla de contenidos

- Práctica 1
  - Tabla de contenidos
  - Objetivo
  - La arquitectura de distribución cliente-servidor
  - JSON-RPC
    - Sintáxis de JSON-RPC
      - Petición
      - Respuesta
  - Poner la teoría en práctica
    - El servidor Elixir
      - Arquitectura del servidor
    - El cliente Java
      - Petición con parámetros
      - La función "printServerResult"
    - Ahora es tu turno
      - Tabla de primitivas

Objetivo

Al finalizar esta práctica sabremos los fundamentos de la comunicación entre un cliente y un servidor, así como una idea del protocolo JSON-RPC. Con todos estos
fundamentos programaremos un cliente en Java que nos va a permitir jugar a un juego distribuido.

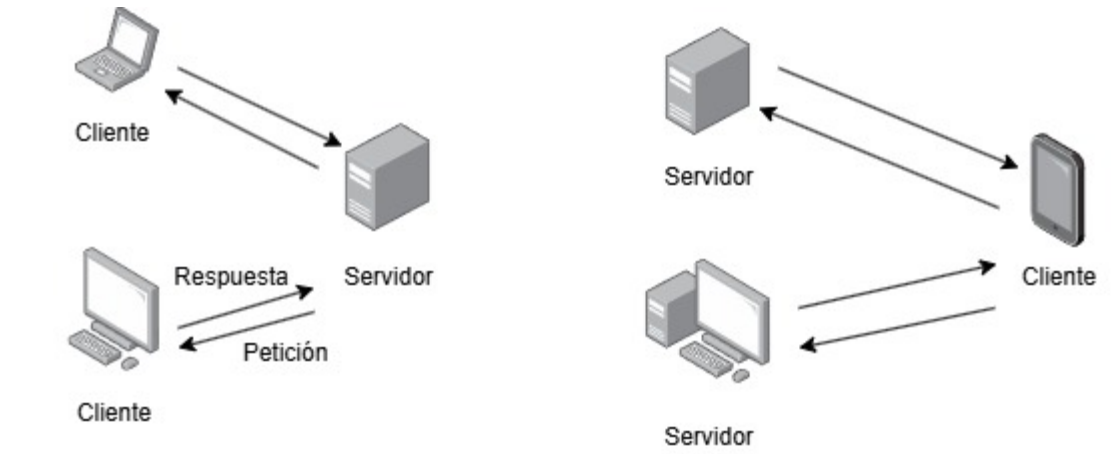
La arquitectura de distribución cliente-servidor

En el contexto de los sistemas distribuidos, un sistema construido en base a una arquitectura cliente-servidor consta de dos elementos, bien nombrados en el propio nombre
de la arquitectura, el cliente y el servidor. Hay que tener en cuenta que esto abarca cualquier multiplicidad de los elementos, un sistema con arquitectura cliente-servidor
puede estar formado por un cliente y un servidor o dos clientes y un servidor o un cliente y tres servidores, y así con todas las combinaciones posibles dependiendo de las
necesidades globales del sistema.

El servidor es la máquina o máquinas físicas o virtuales cuyo trabajo es recibir peticiones a través de un punto de acceso. La comunicación con el punto de acceso
proporcionado por el servidor se realiza mediante uno o varios protocolos de transporte, los más conocidos son TCP (sobre el que se construye HTTP) y UDP.

Todo servidor implementa un servicio, al cuál acceden los clientes mediante una interfaz que presenta el servidor satisfaciendo la función del servicio mediante
funciones accesibles de diferentes formas (RPC, RMI, primitivas...).

Al igual que el servidor el cliente puede ser también una máquina física o virtual o varias de ellas. Su trabajo es realizar peticiones al servidor que provocan
una respuesta del mismo, estas respuestas pueden incluir una gran variedad de tipos, entre ellos respuestas de éxito, de rechazo o de error.



La imagen superior muestra un par de ejemplos de arquitecturas distribuidas cliente-servidor. Se puede observar como tanto los clientes como los servidores pueden
ser cualquier tipo de dispositivo con las capacidad necesarias.

El patrón cliente-servidor se basa en el patrón de interacción petición-respuesta (R-R del inglés *request-response*). Dicho patrón puede ser síncrono o asíncrono,
aunque, durante las prácticas, todo se desarrolla siguiendo el patrón R-R síncrono.

JSON-RPC

JSON-RPC es un protocolo de llamada de procedimientos remotos (RPC, remote procedure call). Normalmente los protocolos que implementan RPC utilizan estructuras de datos
codificadas de manera que la salida de las mismas son binarios no legibles para el ser humano. JSON-RPC trata evita esto empleando la estructura de JSON como método de
codificación para los mensajes entre cliente y servidor.

Durante esta y las siguientes lecciones estaremos trabajando con peticiones por parte del cliente y respuestas por parte del servidor.
El estándar del protocolo define como debe ser la estructura de los objetos petición y respuesta con los que trabajaremos.

Ambos objetos constan de un campo de versión, necesario en este caso ya que estaremos trabajando con la versión 2 de este protocolo.

Sintáxis de JSON-RPC

Petición

El objeto petición implementa una petición al servidor en una estructura JSON con el formato estándar de JSON-RPC.
Consta de un campo método (method) obligatorio en el que se indica el método que se quiere invocar en el servidor, un campo parámetros (params) opcional
en el que se pasan elementos necesarios para el método invocado, el equivalente a los parámetros de una función en programación, y un campo identificador (id) el cual
nosotros tomaremos como obligatorio y que, también en nuestro caso, será un `string` de caracteres.

```
{
  "version": "2.0",
  "method": "travel",
  "params": {
    "location": "madrid",
    "days": 3
  },
  "id": "100"
}
```

Respuesta

El objeto respuesta implementa la respuesta que envía el servidor al cliente una vez la petición ha concluido, al ser un servidor síncrono siempre obtendremos una
respuesta. Consta de un campo resultado (result) si la petición ha sido correcta o, si la petición es incorrecta, un campo error con los motivos del mismo.
Además devuelve el mismo identificador (id) que había recibido del objeto petición.

```
{
  "version": "2.0",
  "result": "has viajado 3 dias a madrid",
  "id": "100"
}
```

Si la respuesta es errónea, el objeto respuesta tendrá la siguiente estructura:

```
{
  "version": "2.0",
  "error": {
    "code": -32600,
    "message": "algo salio mal en el servidor"
  },
  "id": "100"
}
```

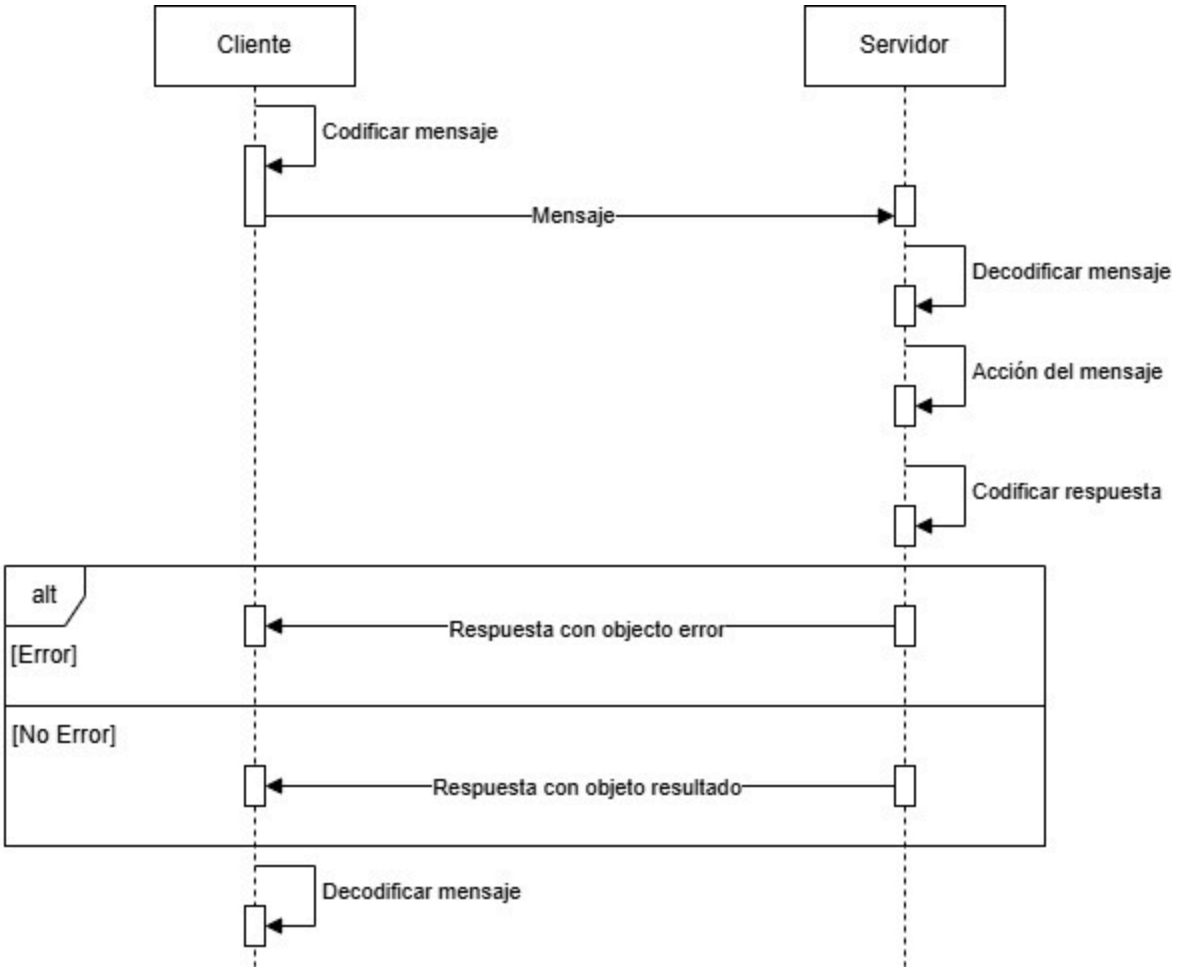
Para profundizar sobre lo que se ha resumido arriba la especificación del mismo es corta y asequible y se puede encontrar en el siguiente enlace:
https://www.jsonrpc.org/specification (inglés).

Para saber más sobre JSON en el siguiente enlace se encuentra la especificación del mismo: https://www.json.org/json-es.html.

Sobre RPC la página de wikipedia en inglés da una buena descripción del mismo: https://en.wikipedia.org/wiki/Remote_procedure_call (inglés).

Poner la teoría en práctica

Con todo lo aprendido anteriormente podemos deducir que la interacción entre el cliente y el servidor sigue la premisa del diagrama de secuencia que se muestra
a continuación.



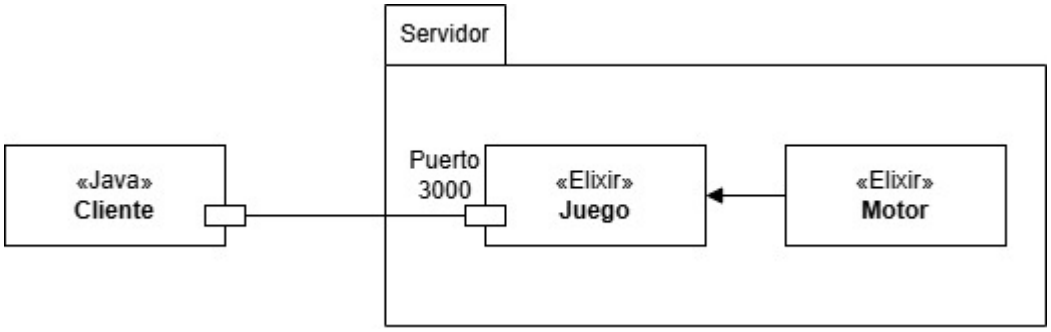
Una vez entendidos los conceptos vamos a ponerlos en práctica. Durante las siguientes prácticas vamos a pasarnos un juego escrito en Elixir y que va a actuar como servidor
(o servidores como veremos en prácticas posteriores) con el que nos comunicaremos mediantes clientes de Java.

El servidor Elixir

Lo que llamamos servidor en realidad es la combinación de dos sistemas, un motor de juego que contiene toda la lógica de las partes que componen un juego y el juego, que
es el servidor con el que nos comunicaremos durante las prácticas que lo requieran.

El servidor es un sistema multienhebrado, es decir, permite múltiples conexiones al mismo tiempo, cuenta con un sistema de autenticación para los usuarios, que nosotros
emplearemos para iniciar nuestros personajes, y maneja como cambia el estado del juego.

Arquitectura del servidor



El cliente Java

Sabiendo que el servidor acepta peticiones TCP/IP y que funciona mediante el protocolo JSON-RPC podemos hacer un cliente en Java que se conecte al puerto 3000
por el que está escuchando el juego y lanzar una petición para saber todos los métodos que acepta el servidor.

```
public class JavaClient {
  public static void main(String[] args) throws IOException {
    Gson jsonParser = new Gson();
    Socket socket = new Socket("localhost", 3000);

    // Abrir el canal de comunicación con el servidor
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

    // Preparar los datos
    HashMap<String, Object> mapa = new HashMap<String, Object>();
    mapa.put("jsonrpc", "2.0");
    mapa.put("method", "info");
    mapa.put("id", "1");

    // Enviar datos
    out.println(jsonParser.toJson(mapa));

    // Obtener respuesta
```

```
    HashMap<String, Object> respuesta = jsonParser.fromJson(in.readLine(), new
TypeToken<HashMap<String, Object>>() {}.getType());

    printServerResult(respuesta); // Imprimir respuesta
    socket.close(); // Cerrar el canal
  }

  private static void printServerResult(HashMap<String, Object> mapa) {
    Object resultado = mapa.get("result");
    Object error = mapa.get("error");
    Object id = mapa.get("id");

    if (resultado == null) {
      if (error == null) {
        System.out.println("Algo salio mal en el servidor \n");
      } else {
        System.out.println("Error al realizar la accion del mensaje con id " + id + ": " +
error + "\n");
      }
    } else {
      System.out.println("Resultado del mensaje con id " + id + ": " + resultado + "\n");
    }
  }
}
```

Vamos a profundizar en lo que estamos haciendo en el código de arriba.

En primer lugar creamos un parser para transformar mapas de Java en JSON y viceversa. Su función es serializar/deserializar los mensajes enviados por red.

```
Gson jsonParser = new Gson();
```

A continuación creamos un socket que nos conecta con el servidor mediante el puerto 3000 y también inicializamos el escritor y el lector del socket.
El escritor nos permite mandar la petición al juego mientras que el lector atrapa la respuesta que este nos dé.

```
Socket socket = new Socket("localhost", 3000);

PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
```

Una vez tenemos el socket conectado el siguiente paso natural es realizar la petición, para ello tenemos que contruirla en un mapa de Java y
dárselo al parser de JSON para que nos lo transforme y lo podamos enviar.

```
HashMap<String, Object> mapa = new HashMap<String, Object>();
mapa.put("jsonrpc", "2.0");
mapa.put("method", "info");
mapa.put("id", "1");

out.println(jsonParser.toJson(mapa));
```

Por último, obtenemos la respuesta del servidor, la imprimimos en consola y cerramos el socket.

```
HashMap<String, Object> respuesta = jsonParser.fromJson(in.readLine(), new
TypeToken<HashMap<String, Object>>() {}.getType());

printServerResult(respuesta);
socket.close();
```

Petición con parámetros

Si queremos mandar parámetros al juego como parte de la petición simplemente habría que incluir el campo en el mapa de Java que hemos creado para la petición.

```
// Petición anterior
HashMap<String, Object> mapa = new HashMap<String, Object>();
mapa.put("jsonrpc", "2.0");
mapa.put("method", "info");
mapa.put("id", "1");

// HashMap de parámetros que queremos enviar
HashMap<String, String> params = new HashMap<String, String>();
params.put("about", "character");

// Añadimos los parámetros a la petición
mapa.put("params", params);
```

El resto de programa es idéntico al cliente mostrado antes. Hay que tener en cuenta que la petición mostrada en este ejemplo falla si se ejecuta contra el juego.
Es un simple ejemplo de como se añadirían parámetros a una petición que los requiera.

La función "printServerResult"

Esta función formatea el resultado que llega desde el servidor, comprobando si ha sido correcta o no.
Siguiendo el estándar de JSON-RPC podemos darnos cuenta de
que hay dos posibilidades:

1. Que el objeto traiga un campo resultado
2. que traiga un campo error.

Teniendo en cuenta esto el formateador simplemente extraerá ambos campos y comprobará si hay un resultado, en caso que no halla, la petición JSON habrá fallado y
el campo error tendrá el motivo del mismo.

Se proporciona esta función para que pueda ser reutilizada por cualquier cliente que sea necesario crear.

Ahora es tu turno

Para solidificar los conocimientos que hemos adquirido y que vamos a ir adquiriendo a lo largo de las siguientes prácticas vamos a jugar al ya mencionado juego.

Como ya sabemos, nos vamos a comunicar con el juego mediante clientes de Java, ya que este actúa como un servidor. Mediante el cliente de Java mandaremos primitivas al juego en formato JSON mediante JSON-RPC para que este modifique el estado del juego de acuerdo a nuestra acciones. Para esta práctica es necesario saber que contamos con un personaje, llamado Tim, este se mueve por un mundo formado por habitaciones conectadas mediante conexiones, dentro de las habitaciones podemos encontrar NPCs, objetos o enemigos. Todos estos elementos tienen un nombre que emplearemos para referirnos a ellos en los parámetros de la petición si es necesario.

Sabiendo cómo se crea un cliente para comunicarnos con el juego, como mandar peticiones con y sin parámetros y qué hace el juego cuando enviamos los mensajes vamos a ponerlo en práctica. Empleando las primitivas de la tabla utiliza los métodos necesarios para encontrar y equiparte una espada. Una vez equipada la espada viaja a la ubicación `habitacion_cueva_3`. Dejaremos esta práctica cuando lleguemos a la localización.

Para poder jugar necesitamos escoger un personaje mediante la utilidad "login" de "Utilities.java". Durante estas prácticas vamos a jugar como Tim.

Tabla de primitivas

Primitiva	Respuesta / Acción	Parámetros	Explicación parámetro
inspect_current_location	estado de la ubicación actual	character_name	nombre del personaje con el que estamos jugando
cross_connection	cambia de estancia al personaje	connection / character_name	nombre de la conexión / nombre del personaje
take_object	recoge un objeto de la estancia	object / character_name	nombre del objeto / nombre del personaje
equip_object	equipa el objeto al personaje	object / character_name	nombre del objeto / nombre del personaje
inspect_character	estado del personaje	character_name	nombre del personaje

B.3. Práctica 2

Práctica 2.

Documento:	Módulo-2: Actores y colas de mensaje
Curso:	SISTEMAS DISTRIBUIDOS - Grado en Ingeniería Informática
Libro:	Enunciado 2: Actores y colas de mensaje.
Autor:	SAYALERO BLAZQUEZ, ALONSO
Fecha:	Jueves, 20 de Noviembre de 2025
Versión:	0.1

Tabla de contenidos

- Práctica 2.
  - Tabla de contenidos
  - Objetivo
  - Actores en los sistemas distribuidos
    - Recepción de un mensaje por un actor (imagen de la teoría)
  - Las colas de mensajes
  - Aplicación en el juego
  - Poner la teoría en práctica
    - Nuestro amigo el sabio
    - No era tan amigo el sabio
    - Abrumar al sabio
    - Finalizando la práctica
      - Tabla de primitivas

Objetivo

Durante esta práctica vamos a profundizar en los actores que ya hemos visto en clase, como funciona una cola de mensajes, la aplicación de todos estos conceptos
en el juego y, finalmente, emplearemos todo ese conocimiento y el adquirido en la práctica anterior para programar clientes simultáneos en Java.

Actores en los sistemas distribuidos

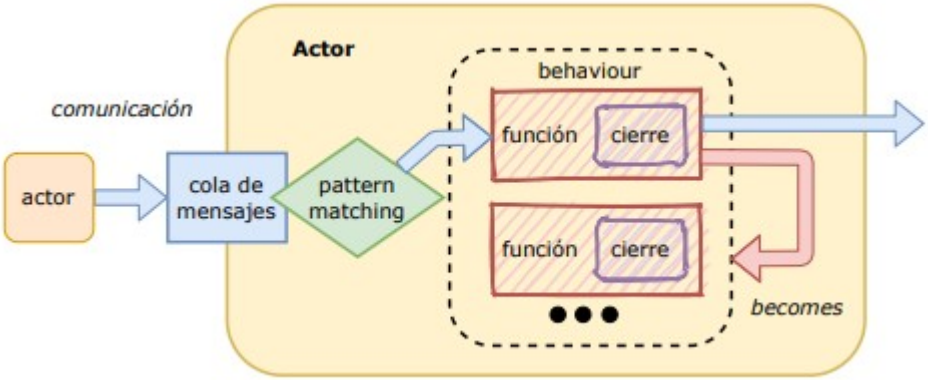
Como ya hemos visto en la teoría los actores son unidades de cómputo que representan una parte de la lógica de la aplicación.
Cada actor cuenta con una dirección para interactuar con él mediante llamadas a funciones, estas funciones mandan un mensaje asíncrono al actor
y este lo almacena en una cola de mensajes. De esta cola de mensajes el actor va procesando cada mensaje y

manda el resultado del proceso de vuelta al
remitente si este lo había solicitado en el mensaje.

Hemos visto cuatro posibles modelos de actores: el modelo clásico, el modelo de objetos activos, el modelo de procesos y los bucles de eventos de comunicación actores
y objetos. El juego al que estamos jugando está escrito en Elixir, por lo tanto estamos ante un modelo de procesos.
Es decir, cada vez que creamos un actor dentro del juego este levanta un nuevo proceso dentro del ámbito del programa y devuelve la dirección en la que el proceso
ha sido creado. Mediante esta dirección podemos enviar mensajes al actor e interactuar con él.

En el caso de Elixir podemos hacer que esta dirección se guarde en un almacén global de direcciones y cualquier programa de Elixir que forme parte de la red
puede acceder a este almacén y comunicarse de forma remota con otros programas. Esta es una de las formas que tiene Elxir de facilitar la programación
de sistemas distribuidos.

Recepción de un mensaje por un actor (imagen de la teoría)



Las colas de mensajes

Una de las partes que hemos visto de los actores es la cola de mensajes, la cual sirve para gestionar el procesamiento de varios mensajes que llegan simultáneamente o
mientras el actor está procesando otro mensaje. Una vez el actor está libre, procesa el mensaje y responde si así lo indica la acción ejecutada a raíz del mensaje.

En nuestro caso las colas de mensajes son FIFO (First In First Out) lo que quiere decir que la prioridad de procesamiento de los mensajes depende del orden en el que
llegan a la cola de mensajes, siendo los primeros en llegar los primeros que se procesan.

Aplicación en el juego

Ya sabemos que estamos utilizando actores para que el juego funcione como un sistema distribuido y que este está escrito en Elixir, también sabemos como funcionan las
colas de mensajes de los actores, ahora vamos a ver como empleamos los actores exactamente. Como ya vimos en la práctica anterior, el juego está compuesto por varias
entidades cada cual representa un elemento del juego. Una habitación, un objeto o nuestro propio personaje son ejemplos de entidades que conforman el juego.

A parte de las entidades, existen partes del juego que no actúan como actores. Estas partes pueden incluir el acceso a la base de datos o la gestión de los
propios actores. De esto podemos deducir que, al decidirnos por una arquitectura distribuida que emplee actores, no todas la partes del sistema se van a regir por las
características de estos y podemos combinarlos con otros patrones de diseño arquitectónico de sistemas distribuidos.

Poner la teoría en práctica

Nuestro amigo el sabio

Ahora vamos a poner en práctica lo visto en los apartados anteriores. Si hemos seguido la práctica anterior deberiamos estar en la habitación `habitacion_cueva_3`.
En esta habitación si empleamos la primitiva de la practica anterior `inspect_current_location` podemos observar que hay un NPC llamado `sabio` y como ya sabemos que
toda entidad del juego es un actor, el `sabio` también lo es. Esto nos permite deducir también que cuenta con una cola de mensajes.
Vamos a interactuar con él, para ello podemos emplear un cliente como el que se muestra a continuación.

```
public class EjemploInteractuar {
  public static void main(String[] args) throws IOException, InterruptedException,
ExecutionException {
    Gson jsonParser = new Gson();

    Socket socket = new Socket("localhost", 3000);
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

    interactWithNpc(jsonParser, out, in);
    socket.close();
  }

  private static void interactWithNpc(Gson jsonParser, PrintWriter out, BufferedReader in)
      throws IOException {
    HashMap<String, String> params = new HashMap<String, String>();
    params.put("npc", "sabio");
    params.put("character_name", "tim");

    HashMap<String, Object> mapa = new HashMap<String, Object>();
    mapa.put("jsonrpc", "2.0");
    mapa.put("method", "interact");
    mapa.put("params", params);
    mapa.put("id", "1");

    out.println(jsonParser.toJson(mapa));

    HashMap<String, Object> respuesta = jsonParser.fromJson(in.readLine(),
        new TypeToken<HashMap<String, Object>>() {
        }.getType());

    printServerResult(respuesta);
```

```
  }

  private static void printServerResult(HashMap<String, Object> mapa) {
    Object resultado = mapa.get("result");
    Object error = mapa.get("error");
    Object id = mapa.get("id");

    if (resultado == null) {
      if (error == null) {
        System.out.println("Algo salio mal en el servidor \n");
      } else {
        System.out.println("Error al realizar la accion del mensaje con id " + id + ": " +
error + "\n");
      }
    } else {
      System.out.println("Resultado del mensaje con id " + id + ": " + resultado + "\n");
    }
  }
}
```

Prueba a interactuar con él.

No era tan amigo el sabio

El sabio no parece muy por la labor de colaborar, habrá que insistir más. Para ello vamos a hacer uso de lo aprendido y a dejarle varios mensajes en
su cola de mensajes.

El objetivo de esta práctica es conseguir que el `sabio` no nos ignore y nos entregue la `llave` que necesitamos para avanzar a la última habitación del juego.
Para ello tenemos varias opciones. La primera y más obvia es meter el envío de mensajes en un bucle for y listo, pero, aunque funciona, no hacemos uso de
las ventajas que nos proporciona un actor ya que no enviamos el siguiente mensaje hasta que el servidor nos responde el anterior. Por lo tanto vamos a olvidarnos
de esta opción y vamos a ver que otras posibilidades existen.

Abrumar al sabio

Para solucionar el problema que nos plantea el `sabio` vamos a enviarle un montón de mensajes al mismo tiempo empleando clientes asíncronos, de esta manera vamos a
crear una situación en la que muchos mensajes llegan al mismo tiempo al actor y este los va procesando de uno en uno. Para ello vamos a emplear la biblioteca de Java
Future la cual nos permite programar código que se ejecuta de forma asíncrona.

```
private static void interactWithNpc(Gson jsonParser) throws IOException,
InterruptedException, ExecutionException {
    // Parametros comunes a todas las llamadas
    HashMap<String, String> params = new HashMap<String, String>();
    params.put("npc", "sabio");
    params.put("character_name", "tim");

    // Ejecutor de tareas asíncronas
```

```
    ExecutorService executor = Executors.newFixedThreadPool(10);
    // Lista de tareas asíncronas
    List<Future<HashMap<String, Object>>> futures = new ArrayList<>();

    // Bucle de creación y lanzamiento de tareas asíncronas
    for (int i = 1; i < 7; i++) {
      HashMap<String, Object> mapa = new HashMap<String, Object>();
      mapa.put("jsonrpc", "2.0");
      mapa.put("method", "interact");
      mapa.put("params", params);
      mapa.put("id", String.valueOf(i));

      Callable<HashMap<String, Object>> task = () -> {
        // Implementación de un cliente
        return respuesta;
      };

      futures.add(executor.submit(task));
    }

    // Muestra de resultados por pantalla
    for (Future<HashMap<String, Object>> future : futures) {
      printServerResult(future.get());
    }

    executor.shutdown();
  }
```

La función que tenemos arriba muestra como vamos a emplear los futuros para obtener la llave. Si nos fijamos bien hemos abandonado la estructura del ejemplo
anterior. Ahora ya no creamos un cliente y enviamos el mensaje mediante el mismo. Vamos a ver que novedades tenemos.

En primer lugar, como vamos a crear varios clientes, es buena práctica separar todo aquello que podamos reutilizar. En este caso los parámetros de la primitiva
son iguales para todos los clientes, por lo tanto es lo primero que vamos poner.

```
HashMap<String, String> params = new HashMap<String, String>();
params.put("npc", "sabio");
params.put("character_name", "tim");
```

A continuación empezamos con el código de Java asíncrono, creando el ejecutor de las tareas asíncronas y una lista vacía para almacenarlas.

```
ExecutorService executor = Executors.newFixedThreadPool(10);
List<Future<HashMap<String, Object>>> futures = new ArrayList<>();
```

Una vez tenemos el ejecutor de tareas solo falta crear y ejecutar los clientes. Esta labor consiste en repetir la siguiente secuencia un número finito de veces.
La secuencia es:

1. Crear el objeto mensaje tal y como lo hemos hecho en la práctica anterior.
2. Crear un objeto tarea el cual consiste en una función que ejecuta un cliente como los que ya hemos visto y devuelve el resultado de ese cliente.
3. Lanzar los objetos tarea empleando el ejecutor y almacenarlos en la lista que hemos creado.

El primer paso es muy sencillo, dentro del bucle creamos un `HashMap` como los que creamos en la práctica anterior.

```
HashMap<String, Object> mapa = new HashMap<String, Object>();
mapa.put("jsonrpc", "2.0");
mapa.put("method", "interact");
mapa.put("params", params);
mapa.put("id", String.valueOf(i));
```

Para el segundo paso hay que introducir algunos conceptos de Java que no habían aparecido hasta ahora.

```
Callable<HashMap<String, Object>> task = () -> {
  // Implementación de un cliente
  return respuesta;
};
```

El primer concepto que hemos introducido es la interfaz `Callable`, mediante esta interfaz se represnta una tarea que devuelve un resultado, puede lanzar una
excepción e implementa un solo método `call` sin argumentos. Todo esto se traduce en una función que, en nuestro caso, implementa un cliente que se comunica
con el servidor y devuelve el resultado de dicha comunicación.

El segundo concepto introducido son las expresiones lambda (`() -> {}`), las cuales no son objeto de estudio ahora mismo pero que, en este caso en concreto, nos van a servir
para crear funciones anónimas, ahorrándonos código.

Por último, el tercer paso es añadir el resultado de la función del ejecutor `submit` a la lista.

```
futures.add(executor.submit(task));
```

Para complementar todo el trabajo hecho en los puntos anteriores es importante visualizar los resultados de las tareas asíncronas que hemos lanzado.

```
for (Future<HashMap<String, Object>> future : futures) {
  printServerResult(future.get());
}
```

Finalizando la práctica

Te habrás dado cuenta que en el código presentado antes hay un comentario que dice "Implementación de un cliente", sustituyendo a este comentario vamos a tener que
implementar un cliente como los empleados en la práctica anterior. Para ello hay que abrir una conexión con el servidor y mandar el mensaje que ya hemos construido
(`mapa`). No te olvides de cerrar el socket antes de devolver el resultado.

Si hemos construido el cliente como ya sabíamos veremos seis mensajes por pantalla. ¿Notas algo raro? ¿Qué razón puede haber para que esto suceda?

Para terminar del todo con la práctica, una vez conseguida la `llave`, nos aseguraremos que efectivamente nuestro personaje tiene el objeto en su inventario y saldremos
de la habitación del `sabio` para finalizar la práctica en la siguiente sala.

Tabla de primitivas

Primitiva	Respuesta / Acción	Parámetros	Explicación parámetro
interact	interactúa con un NPC	npc / character_name	nombre del NPC / nombre del personaje con el que estamos jugando

B.4. Práctica 3

Práctica 3.

Documento:	Módulo-3: Desarrollo en Elixir
Curso:	SISTEMAS DISTRIBUIDOS - Grado en Ingeniería Informática
Libro:	Enunciado 3: Modificación del motor Hephaestus.
Autor:	SAYALERO BLAZQUEZ, ALONSO
Fecha:	Lunes, 1 de Diciembre de 2025
Versión:	1.0

Tabla de contenidos

- Práctica 3.
  - Tabla de contenidos
  - Objetivo
  - La situación
  - Elixir
  - Puesta en práctica
    - Programando en Elixir
    - Volvemos a Java
      - Tabla de primitivas

Objetivo

En esta última práctica vamos a ver un poco más de cerca con que hemos estado jugando. Tocaremos el motor para poder pasarnos el juego.

La situación

Tras la finalización de la práctica anterior nos encontrabamos en la habitación con tres conexiones tras haber conseguido la `llave` del `sabio`.

Tras la conexión bloqueada con `llave` se ubica la última sala del juego. Vamos a entrar a la sala y derrotar al enemigo empleando todo lo aprendido
hasta ahora. Si hemos hecho las cosas anteriores el enemigo debería caer derrotado al primer ataque.

Una vez derrotado el enemigo nos damos cuenta que no hay nada por el logro de haberlo derrotado. Asique, ya que el juego no nos da un trofeo, vamos
a arreglarlo para que ese no sea el caso.

Nos vamos a adentrar en el motor del juego y vamos a programar en Elixir un trofeo para que Tim recoja el fruto de sus esfuerzos.

Elixir

Elixir es un lenguaje de programción que sigue el paradigma de programación funcional. Está diseñado para ser concurrente, de propósito general e
introduce características que no se ven en Java.

Algunas de las características de Elixir incluyen *pattern matching*, es decir, el operador `=` actúa de la misma forma que en matemáticas,
ambos lados del símbolo tienen que ser lo mismo, esto permite la asignación de variables, pero estas son inmutables, una vez se asigna un valor este no
se puede cambiar. A parte de la asignación de variables, el "pattern matching" permite definir la misma función varias veces con distintos argumentos o
con guardas como en el siguiente ejemplo.

```
def fib(n) when n in [0, 1], do
  n
end

def fib(n), do
  fib(n-2) + fib(n-1)
end
```

Otra de las características que lo diferencian de lenguajes como Java es que no tiene bucles. Para poder realizar un bucle en Elixir hay que emplear técnicas
de recursividad para lograr el objetivo.

En Elixir todas las funciones devuelven el resultado de la última expresión evaluada. Esto quiere decir que no tenemos la palabra clave *return* como en Java y que
tampoco tenemos funciones que no devuelven nada. Por ejemplo:

```
def devuelve_hola_string() do
  "hola" # <- Este es el último elemento evaluado de la función por lo tanto es lo que
devuelve
end

# La función suma es tan fácil como devolver la expresión de la suma de los dos parámetros
def suma(num1, num2) do
  num1 + num2 # <- Último elemento evaluado
end

# También podemos devolver los resultados de otra función
def devuelve_resultado_funcion() do
  numero = suma(2, 4)
  numero
end

# Se puede omitir la asignación a una variable y evaluar directamente la función
def devuelve_resultado_funcion() do
  suma(2, 4)
end
```

Cabe destacar también el motivo principal por el que empleamos este lenguaje para el motor del juego y para el juego mismo y es la inclusión de `GenServer` en
la biblioteca estándar del lenguaje. Un `GenServer` es, para nosotros, el equivalente a un actor, cumple esa función dentro del juego.

Para más información visita la página oficial de Elixir en la que se pueden ver varios ejemplos de como funciona el lenguaje.
https://hexdocs.pm/elixir/1.19.1/introduction.html

Puesta en práctica

Una vez sabemos como funciona Elixir vamos a programar nuestro `trofeo` y hacer que Tim lo recoja.

Programando en Elixir

Dentro de "tfg_umbrella/apps/engine" encontramos el fichero "location.ex", aquí se encuentra el `GenServer` que define el comportamiento de una habitación.
La función que buscamos en concreto es la función `handle_call(:get_enemy, _from, state)`. En esta función ya vemos una característica de los actores y es la
comunicación entre ellos.

```
@impl true
def handle_call(:get_enemy, _from, state) do
  response =
    Enum.filter(state.enemy, fn ene ->
      Engine.Enemy.is_alive?({:global, Engine.Utilities.advanced_to_atom(ene)}) # <- En esta línea nos comunicamos con los actores que representan los enemigos
    end)

  {:reply, response, state}
end
```

Ahora vamos a modificar esta función para que cree un actor desde la misma (otra de las características que hemos visto de los actores es que pueden crear otros
actores) que va a ser el actor objeto `trofeo`. Para ello la definición del actor es la siguiente:

```
%Engine.GameEntity{
  name: "trofeo",
  state: %Engine.Object{
    level: 1,
    stat: nil,
    type: "inventory_item",
    value: nil
  }
}
```

Los pasos que tenemos que seguir son:

1. Comprobar que el número de enemigos es mayor que 0. A continuación vemos como es la estructura condicional. En caso de que no se cumpla la condición,

```
en este ejercicio en concreto, vamos a simplemente responder con el resultado `response` de
la siguiente manera `{:reply, response, state}`.
```

```
if condition do
  # Se cumple la condición
else
  # No se cumple la condición (opcional)
end
```

2. Si son mayor que 0 comprobar que el resultado de `response` arriba es igual a 0.
3. Si se cumple la premisa anterior, iniciar el objeto como hijo del supervisor dinámico del motor:

```
DynamicSupervisor.start_child(Engine.Supervisor, {Engine.Object, definicion_objeto}) # La
definición que hemos visto antes
```

4. Una vez iniciado el actor objeto, modificar el estado de la propia habitación para que muestre trofeo como un objeto que se encuentra en ella.

Para resolver el último punto vamos a hacer que cambie el estado del actor. Cuando respondemos con `{:reply, response, state}` le estamos diciendo al `Genserver` que tiene que es una llamada síncrona (`:reply`), qué es lo que tiene que responder a la llamada (`response`) y como modifica esta llamada el estado del actor.

Hasta ahora ninguno de los caminos que podía seguir modificaba el estado interno del actor, simplemente devolvía información del mismo, por eso el tercer parámetro es el mismo estado que ya tenía: `state`.

Este último camino si que nos lleva a modificar el estado interno del actor, ya que hay que incluir el `trofeo` en la habitación. Por lo tanto tenemos que saber algunas cosas nuevas. En primer lugar, el estado de todos los actores del juego son mapas clave-valor que en Elixir se representan por `%{}` y sus claves pueden ser un `atom` (`:ejemplo`) o un `string` (`"ejemplo"`). En segundo lugar, ya hemos dicho que las variables de Elixir son inmutables, esto incluye a la variable `state` que contiene el estado del actor, por consiguiente vamos a tener que crear una nueva variable `new_state` que contenga la modificación al estado.

Sabiendo que la clave que tenemos que modificar es `:objects` y el valor es `["trofeo"]`, completa la siguiente expresión: `new_state = %{...}`. Para saber como se modifica un valor de una clave de un mapa investiga en la documentación oficial (https://hexdocs.pm/elixir/Map.html).

Una vez modificado el estado, la respuesta final del camino será: `{:reply, response, new_state}`.

De esta forma la habitación en la que se encuentra nuestro personaje creará un actor `object` con nombre `trofeo` una vez derrotemos al enemigo.

Volvemos a Java

Una vez tenemos un nuevo motor que nos va a proporcionar un premio cuando completemos el juego vamos a hacer eso mismo, finalizar el juego. Para ello, en primer lugar
tenemos que construir la nueva imagen del motor y del juego tal y como se detalla en la práctica 0 y, después programar un cliente Java que ejecute las primitivas
necesarias para realizar la siguiente secuencia (empezamos desde la habitación de la práctica anterior):

1. Viajar a la localización `habitacion_cueva_2`.
2. Atacar al enemigo que se encuentra dentro de dicha habitación.
3. Comprobar el estado de nuestro personaje tras el combate.
4. Inspeccionar la habitación en busca de nuestro `trofeo`.
5. Recoger el `trofeo`.

Para completar esta última parte de la práctica contamos con la siguiente tabla con todas las primitivas que permite el servidor.

Tabla de primitivas

Primitiva	Respuesta / Acción	Parámetros	Explicación parámetro
inspect_current_location	estado de la ubicación actual	character_name	nombre del personaje con el que estamos jugando
cross_connection	cambia de estancia al personaje	connection / character_name	nombre de la conexión / nombre del personaje
take_object	recoge un objeto de la estancia	object / character_name	nombre del objeto / nombre del personaje
equip_object	equipa el objeto al personaje	object / character_name	nombre del objeto / nombre del personaje
inspect_character	estado del personaje	character_name	nombre del personaje
interact	interactúa con un NPC	npc / character_name	nombre del NPC / nombre del personaje con el que estamos jugando
attack	ataca a un enemigo de la sala	enemy / character_name	nombre del enemigo / nombre del personaje con el que estamos jugando

Bibliografía

[1] Firmantes del manifiesto. Manifesto for agile software development. https://agilemanifesto.org/, 2001. [Online; último acceso 27-Diciembre-2025].

[2] Bob Hughes y Mike Cotterell. *Software Project Management*. McGraw Hill, 2009.

[3] docsifyjs. Biblioteca docsify. https://docsify.js.org/#/, 2025. [Online; último acceso 28-Diciembre-2025].

[4] Ian Sommerville. *Software engineering 10th Edition*. Pearson, 2016.

[5] Ministerio de Trabajo y Economía Socila, Gobierno de España. El boe publica el smi para 2025 que se establece en 1.184 euros. https://www.sepe.es/HomeSepe/es/que-es-el-sepe/comunicacion-institucional/noticias/detalle-noticia.html?folder=/SEPE/2025/Febrero/&detail=boe-publica-smi-2025-se-establece-1184-euros, 2025. [Online; último acceso 31-Diciembre-2025].

[6] Taxfix. Salario mínimo interprofesional (smi). https://taxfix.com/es-es/diccionario/salario-minimo-interprofesional-smi/#:~:text=Gracias%20al%20acuerdo%20en%202025%20del%20Gobierno,trabajadores%20a%20jornada%20completa%20(40%20horas%20semanales)., 2025. [Online; último acceso 31-Diciembre-2025].

[7] Banco Santander. ¿qué es la amortización, qué tipos hay y cómo se calcula? https://www.bancosantander.es/glosario/amortizacion, 2026. [Online; último acceso 5-Febrero-2026].

[8] Agencia Tributaria, Ministerio de Hacienda, Gobierno de España. Amortizaciones. https://sede.agenciatributaria.gob.es/Sede/impuesto-sobre-sociedades/que-base-imponible-se-determina-sociedades/amortizaciones.html?faqId=42c3904421205710VgnVCM100000dc381e0aRCRD, 2026. [Online; último acceso 5-Febrero-2026].

[9] Wikipedia. Second life. https://en.wikipedia.org/wiki/Second_Life, 2025. [Online; último acceso 31-Diciembre-2025].

[10] David Alexander Bond. Design and implementation of a massively multi-player online historical role-playing game. *MSc Advanced Internet Applications. Heriot-Watt School of Mathematical and Computer Sciences*, 2015.

[11] Michael R. Cantrell y Mike Holmes Whitson John Kirk III. Design patterns of successful role-playing games. 2006.

[12] El equipo de Elixir. Elixir. `https://elixir-lang.org/`, 2025. [Online; último acceso 21-Junio-2025].

[13] El equipo de Erlang. Erlang. `https://www.erlang.org/doc/system/design_principles.html`, 2026. [Online; último acceso 06-Abril-2026].

[14] El equipo de Erlang. Otp. `https://www.erlang.org/`, 2026. [Online; último acceso 06-Abril-2026].

[15] El equipo de Phoenix. Phoenix framework. `https://www.phoenixframework.org/`, 2025. [Online; último acceso 01-Julio-2025].

[16] mtrudel. Biblioteca thousand island. `https://hexdocs.pm/thousand_island/ThousandIsland.html`, 2025. [Online; último acceso 28-Diciembre-2025].

[17] Oracle. Java. `https://www.java.com/es/`, 2025. [Online; último acceso 01-Julio-2025].

[18] Wikipedia. Sql. `https://en.wikipedia.org/wiki/SQL`, 2025. [Online; último acceso 01-Julio-2025].

[19] Douglas Crockford. Json. `https://www.json.org/json-en.html`, 2001. [Online; último acceso 01-Julio-2025].

[20] Mozilla foundation. Javascript. `https://developer.mozilla.org/es/docs/Web/JavaScript`, 2025. [Online; último acceso 01-Julio-2025].

[21] El equipo de Postgresql. Postgresql. `https://www.postgresql.org/`, 2025. [Online; último acceso 01-Julio-2025].

[22] El equipo de Docker. Docker. `https://www.docker.com/`, 2025. [Online; último acceso 01-Julio-2025].

[23] The linux foundation. Kubernetes. `https://kubernetes.io/es/`, 2025. [Online; último acceso 01-Julio-2025].

[24] Matt Morley. Json-rpc. `https://www.jsonrpc.org/`, 2005. [Online; último acceso 03-Enero-2026].